AI相册 · 全模块研发交付手册

v1.0 · 2026-10-21 北京时间 · 文档与设计合同交付

49项需求 · 155个API操作 · 65页面 · 92张表 · 44项实施任务

产品尚未编码；实际支付、微信、AI、设备与生产验证均需在实施后执行。机器合同、完整CSV和原型源文件随ZIP提供；独立文件为权威源。

手册目录

README.md

01-产品与模块基线.md

02-架构与集成合同.md

03-状态机与一致性.md

04-实施与交接索引.md

06-发布准出清单.md

07-决策与接入清单.md

contracts/API-GUIDE.md

contracts/SCHEMA-GUIDE.md

content/README.md

content/filter-kernel.md

ux/interaction-spec.md

operations/deployment-runbook.md

testing/TEST-STRATEGY.md

database/transaction-safety.md

database/data-dictionary.md

../superpowers/plans/2026-10-21-c1-editor-render.md

../superpowers/plans/2026-10-21-c2-layout.md

../superpowers/plans/2026-10-21-c3-cloud-identity-mini.md

../superpowers/plans/2026-10-21-c4-jobs-commerce.md

../superpowers/plans/2026-10-21-c5-ai-admin.md

../superpowers/plans/2026-10-21-c6-release-operations.md

AI 相册：全部模块研发交付包 v1.0

日期2026-10-21（北京时间）。本包将产品、配置、研发合同、界面、计划、测试和运维收敛到同一入口，覆盖首期全部模块。这是可交接评审的研发资料包，产品代码尚未实施；真实微信、支付、AI和生产环境尚未联调。

完整ZIP下载 · 研发手册PDF · 可编辑Word · 离线阅读 · 19页原型册 · 原型索引

从哪里开始

1. 阅读统一产品基线与共同合同，明确网页完整编辑、小程序快速成册、会员点数及AI创作的边界。

2. 按需求追踪表核对49项需求对应的页面、API、计划任务和验收；App预留是接口要求，不虚构App页面。

3. 阅读架构与集成、状态机与一致性，再使用机器合同与数据库设计。

4. 按执行索引逐单元实施；记录真实验证结果，完成发布清单。

交付清单

部分	交付内容	入口
产品与范围	全模块统一规格、默认值、范围外、权威顺序、决策／外部接入	基线、决策
需求追踪	49项需求映射页面／接口／计划／用例	requirements.csv
API	OpenAPI3.1，155操作／134路径，类型、错误、权限、CAS、幂等和回调	openapi.yaml、接入说明、操作索引
业务数据合同	文档／排版／配置／导出／provider，5份draft2020-12 schema；含21类配置值覆盖20域	schemas、语义说明、有效与无效样例
数据库	92表完整字段字典、203 FK的ERD、初始up／受保护down、事务边界及可复现SQL断言	数据库字典、up、down、ERD、事务
页面与权限	65页面／390状态、54动作×10角色、全页流程、历史15张＋新增4张原创原型	页面索引、状态CSV、权限、原型预览
实施计划	C1 – C5＋C6发布验收，44任务／220个可勾执行步骤	实施索引
内容设计	33滤镜数值／配方、5类原创模板槽位、3类原创水墨彩绘提示词草案	内容目录
测试	73业务验收＋155 API合同测试项、故障／真机／视觉／容量策略	策略、业务案例、API案例
运维	环境变量schema、无秘密.env示例、未来应用集成Compose、发布／回滚／恢复／告警手册	运行手册、环境schema、环境示例、Compose设计
可复核证据	文档／引用／样本校验、隔离PG实际执行、SHA清单	文档校验、SQL执行、manifest、校验脚本

模块交付顺序

C1网页编辑与真实导出→C2整本排版→C3身份／云素材／小程序→C4可靠队列／配置／商业→C5生成与运营。C6贯穿集成、恢复与最终发布；不在C1结束时冒充完整产品。工作区权限、资源版本、任务事实源和账本跨模块共用，不复制第二套存储路径或余额。

已完成和仍需真实执行的边界

本次完成文档与设计合同，运行了合同语法／引用／示例检查，并在隔离PostgreSQL执行迁移、39个预期拒绝断言、合法夹具与受保护down/up。具体版本、SHA和范围以verification文件为准；PG16记录是最终jpeg/zip修订前版本，最终SQL以PG17复核。

未执行产品单元／E2E／性能／真机／真实供应商测试，未构建生产镜像或上线服务。测试CSV统一planned_not_executed，实施checkbox未勾；SQL结构验证不证明跨供应商、真实并发或支付协议已实现。

后台根前缀＋用户逻辑素材目录按暂定基线写全；供应商、价格、主体资质、部署预算及正式保留政策在决策清单记录，相关能力配置不齐时有明确unconfigured行为。BYOS／BYOK、专业印刷、多人实时协同和App安装包不在首期。

Word／PDF是主研发手册，机器合同、完整数据库逐列字典、CSV和原型源文件以同包独立文件为权威，避免数万行OpenAPI／SQL被截断或只剩图片。ZIP保留docs目录结构与历史资料，离线链接可按原相对路径访问。

产品与模块统一基线 v1.0

日期2026-10-21，北京时间。此稿统一v0.1 – v0.4的评审设计，交付范围为全模块研发文档与设计合同。实际代码、账号资质、付费服务和生产部署不包含在文档完成声明中。

1. 用户、端与成功条件

摄影师／影楼要精细排版、复用客户素材和稳定交付；个人用户要手机上传后快速成册。网页负责完整图层，小程序负责上传、模板、规则排版、文本替换／裁剪、预览和任务找回。后续App复用内部用户、文档、权益及任务API；首期不交付App安装包。

普通照片成册保持照片内容，AI艺术创作明确可能改变人物和场景。用户能核对照片去向，改标题不用重新生图；预检失败给出可修复原因，排队时继续编辑不改变导出快照。大陆微信生态与人民币优先，微信／支付按实际主体与当期能力上线。

2. 模块及首期完成定义

模块	首期行为	可观察完成条件
C1 编辑与确定性导出	多跨页、封面、图片／文字／矩形／圆／线、组合、锁定、层序、对齐吸附、裁剪、33滤镜、模板、历史和保存、PNG／JPG／普通PDF	原图不可读不回退缩略图，中文不漏字、尺寸和DPI独立正确，冻结版本的实际样片可追溯
C2 智能排版	整本分配、固定页构图、3个有效候选目标、主图／顺序／必选／禁用／锁定、局部重排	all＋forbid全量一次；无解不删片，实际不足3候选就显示实际数量；应用CAS，manifest从文档重算
C3 身份／云资产／跨端	网页与微信小程序身份、绑定、工作区、上传、云版本、两类目录、分享、通知、注销、mini轻编辑	跨租户拒绝；目录改名不改对象地址；两端冲突不覆盖；unknown字段保留；原片默认私有
C4 任务／配置／商业	Outbox、worker租约与fence、重试取消、配置发布、会员／点数／兑换码、支付退款对账	数据库可重建队列；回调重放只履约一次；点数预占／结算／释放各一次；已接受权益快照可完成
C5 AI／运营	文生图／参考图水墨彩绘、模型／路由／风格／预算、报价、审核标识、素材入册、授权登记和管理	未知上游状态先查询；未审核结果隔离；非商业／许可不明预设不能上架；生成图与可编辑标题分层
OPS 发布验收	环境、秘密、监控、备份恢复、容量、安全、预发真机联调和发布回滚	P0实测、账单核对、恢复演练、渲染基准、平台准入均有真实证据；按发布检查表关闭阻断

3. 容量与默认值

文档schemaVersion1、px96、sRGB；pageSize为单页，spread宽为2倍，封面独立。1 – 50内页跨页、2 – 100内页、500照片／项目、100递归图层／面、组深度≤5；JPEG／PNG≤25_000_000字节、≤80_000_000源像素。容量为设计上限，实测不过则明确收紧配置并提示，不写成已有性能保证。

导出DPI72／150／300、≤50MP／面；先预检再冻结；600×300mm、300DPI裁切区7087×3543像素，四边3mm出血为7157×3614。普通PDF为sRGB栅格页，不承诺PDF/X、CMYK、拼版或印厂签收。缺字／文字超框阻断；有效PPI100 – 199警告、<100强警告，可在数字导出明确确认继续。

保存空闲2秒／持续10秒触发，至少100步撤销、20恢复版本；历史文件不会因新模板／滤镜发布自动升级。浏览器C1预算768MiB、headless2GiB、图片缓存128MiB，保守估算超限拒绝；运行服务有单独实测进程预算，不能仅按CPU核数开并发。

4. 配置与目录的完整边界

平台后台统一接入存储与AI；用户自定义素材逻辑文件夹；后台可改根前缀，但仅影响新写入。首期覆盖20配置域，schema有21种typed value（含相册默认值albumDefaults）。配置草稿、测试、发布、回滚、秘密版本、权限和审计均纳入；数据库／Redis／秘密服务引导地址放部署层。

目录1 – 80个Unicode码点、NFC和大小写折叠去重、最多10级、同工作区；移动防循环／CAS，使用中素材阻断整批删除，回收／恢复不覆盖同名目录。对象key用随机ID／hash，不含客户姓名或目录名。BYOS／BYOK、目录独立共享ACL、智能标签相册为后续范围。

5. 身份、商业与内容

内部用户随机ID；不按昵称／同手机号自动合并。工作区owner购买／兑换／申请退款；角色不等于设备数。影楼初期单使用者，数据保留席位／角色，邀请UI以后开放。管理员业务权限不自动获得客户照片访问权。

免费／个人／影楼会员与AI点数包按SKU版本配置，价格与额度由经营者确认。首期不自动续费；期限用天数。AI以子任务单图计价，quote建议5分钟；接受后原子预占点数和平台成本，成功审核通过结算，明确失败释放；未知状态终局建议30分钟，不自动跨供应商双提交。

会员去品牌水印不去AI来源标识；用户原片不默认用于训练或公共素材库。模板、字体、滤镜、生成风格有不可变版本和许可记录；非商业Zine Skill不安装／不调用为商业能力，参考调研保持来源证据。

可选外部标题／章节建议模型在首期仅预留关闭；未具备独立文本输入／输出与计费合同前不启用，普通标题编辑和规则排版始终不依赖它。

6. 范围外与技术假设

首期不含多人实时协同、PSD／RAW／HEIC、任意上传SVG、专业印刷、自动续费、设备离线许可证、素材公开交易市场、App安装包、BYOS／BYOK。它们不是未完成首期模块，若纳入需显式变更范围和合同。

自定义目录两类、具体供应商／价格、真实微信／支付能力、部署预算为明确决策项，见决策记录。本轮不为这些决策编造真实值；所有依赖缺失都有disabled/unconfigured行为和验收用例。

7. 权威顺序

本目录统一基线与明确的schema／OpenAPI约定优先于历史示例；业务不变量优先于UI示意。数据库SQL负责已声明约束，跨表账本、照片完整性、权限与供应商状态由事务／服务验证补足。对应变更必须同时改需求追踪、接口、schema、SQL、页面、计划与用例；禁止只改一个版本的字段名。

历史材料在同包docs相邻目录保留；旧文档中的“ 团队未提供 ”等历史状态由本轮“ 全程AI开发 ”取代，不能把旧估算区间当交期。

架构、模块依赖与供应商集成合同

1. 代码与部署职责

采用Node24 LTS、TypeScript、React／Vite／Fabric网页、原生mini TS/WXML、NestJS模块化API、PostgreSQL事实源、Redis／BullMQ调度、对象存储；导出与AI独立worker。C1精确候选版本在原计划；backend版本由技术验证锁定package-lock和镜像摘要，不把候选当已测试。首期不用微服务／Kubernetes增加运维要求。

预期代码路径：packages/document、assets、storage、render、export、layout、contracts；apps/web、admin、api、mini、export-worker、ai-worker。实际文件由各单元计划列明，计划执行前核对真实分支与依赖；不在本次文档包创建空服务冒充实现。

依赖序：C1→C2；C1文档端口→C3；C3＋C1渲染→C4；C3＋C4→C5；OPS跨单元持续记录证据，最终全链路准出。C4配置、队列、权益可独立验证，但支付需要渠道资质，AI真实调用需C5授权和预算。

2. 核心数据流

上传：成员权限／额度预占→短期上传签名→原始对象→格式／像素／方向验证→归一图／预览→ready。文档保存使用当前revision CAS，新增不可变revision。mini仅可修改允许文本、crop、页面顺序和候选选择，未知高级字段透传；非法patch拒绝。

导出：保存→预检token绑定文档／资源／选项hash→验证权益／警告确认→事务建立快照、job、pin、Outbox→worker取得递增fence→同包渲染／编码→临时对象→条件提交产物→下载鉴权。取消／重复投递／旧worker不能产生第二个合法结果。

生成：已发布预设→用途／参数验证→Quote→点数／平台预算同事务预占→Outbox→提交或查询→结果隔离解码→审核／标识→私有新素材→唯一结算。发布新模型不会更改已有报价／任务；状态未知只查原供应商，不盲重提。

配置：草稿→字段／依赖／秘密权限→绑定hash的测试→release原子指针切换；旧任务继续用冻结版本。回滚配置复验旧凭据，不改历史账本；紧急停用可阻断新请求并对已有任务明确终结／补偿。

3. 端口合同

端口	必要操作／返回	失败语义
StorageAdapter	probe、signUpload、head、read、put、copy、delete、signDownload；每项含profileVersion／objectRef	明确不存在／无权／限流／临时错误；禁止自动读缩略图代替原片
IdentityAdapter	exchangeMiniCode、createWebQr／consumeScene、verifyBinding、refresh／revoke	一次凭据单次消费；缺UnionID不合并；token不返回session_key
PaymentAdapter	createPayment、verifyRawEvent、queryPayment、refund、queryRefund	验签和金额／商户／应用／商品一致后正规化事件；ACK按渠道合同，不固定先ACK
GenerationProvider	describe、validateParameters、estimateCost、submit、query、cancel（若支持）、fetchResult	accepted／rejected／unknown区分；不能将超时当明确失败后重新付费提交
ModerationAdapter	checkInput、checkOutput、labelRequirements、appealStatus	审核缺失不发布；有来源和策略版本，timeout不能当审核通过
NotificationAdapter	canSend、send、queryReceipt、unsubscribe	用户授权／订阅次数／限流；通知失败不回滚已成功付款／导出

provider的机器可校验元数据和输入／结果见provider schema。外部签名算法、状态映射、错误重试、endpoint、限制与计费必须由具体adapter契约报告确认；统一HTTP格式不意味着业务兼容。

4. 所有第三方的准入报告

每个adapter交付：供应商／文档日期、SDK或HTTP版本及许可、主体与服务区域、数据用途／训练／保留、认证和轮换、请求与返回映射、单次费用和成本口径、幂等与查询／取消能力、回调签名／重放、限流、故障矩阵、沙箱／小额真实验收记录、退出与数据删除方式。

候选OSS／COS／S3兼容分别测试；没有实现的adapter不出现在可启用列表。AI聊天兼容不代表参考图生成兼容；风格名称不能代替模型commercial许可。支付渠道未开通或mini平台能力不足时返回capability不可用，不设计绕过购买限制的引导。

5. 渲染与字体合同

自有JSON模型权威，Fabric仅映射。EXIF方向归一化一次，filter在归一图源坐标先应用再crop；CPU kernel、seed、recipeVersion固定。固定Chromium、字体bytes／许可／hash、locale／DPR与渲染build进入manifest。CJK文字分段／禁则／度量预览与导出共用，缺字／超框阻断；不回退系统字体。

同环境重复快照比较解码像素hash，不要求ZIP时间戳或不同系统JPEG编码字节相同。PNG pHYs／JPEG JFIF密度与实际像素分别验证；普通PDF MediaBox／TrimBox／BleedBox和页序独立校验。服务端进程出站仅允许自有manifest和已批准供应商结果域；不接受用户URL／脚本／file路径。

6. 数据和运行合同

OpenAPI是传输合同；schema目录定义业务文档／布局／配置／导出／provider输入；数据库目录定义持久化；状态机定义事务边界。结构校验之外还必须执行语义验证和工作区权限。

worker部署预算必须经过实际容量测试；BullMQ锁不代替数据库fence；Redis重建不按缓存余额恢复财务。可观测日志关联requestId/jobId/orderId，不泄露token／签名URL／提示词／原片／个人资料。数据库UTC，预算日Asia/Shanghai。

状态机、事务与一致性合同

1. 任务

queued→running→packaging（导出）→succeeded；可重试错误running→retry_wait→queued；终态failed／cancelled。cancel_requested为请求状态，worker在安全边界终结；succeeded若先赢返回已有结果。claim递增fence并设置lease_until；heartbeat和结果提交必须匹配当前fence且租约未失效；旧worker只允许清理自己的临时对象。

jobs.state不含unknown_upstream；生成确认中表示running＋providerSubmissionState=unknown、stage=confirming_upstream。平台配置测试／对账／恢复演练使用服务端创建的platform工作区，公开创建工作区仅允许personal／studio。

每次任务提交有workspace＋Idempotency-Key唯一，保留7天；同key不同payload409。任务＋业务快照＋资源pin＋必要预占＋outbox同事务。投递器至少一次，消费者以jobId和数据库状态去重；不能宣称消息恰好一次。终态与有效产物唯一条件提交后才允许下载。

建议重试最多3次，退避10／30／90秒加抖动。坏schema、缺资源／权限、缺字体、文本溢出、内容拒绝不重试；OOM不能用相同资源盲试。排队公平性按工作区，免费任务有最大等待保护。重试终态失败任务为新jobId，关联parentJobId并重新验证权限／配额。

2. 文档与候选

文档CAS更新：锁项目→确认expectedRevision→校验字段／资源所属／授权／预算→追加revision和hash→更新current指针。冲突返回409和可恢复草稿；不自动合并；未知同版本字段保存原样。模板／滤镜发布不能改变旧revision。

候选生成不更新作品：固定输入、模板／算法版本和seed；可行性硬约束先行，再评分。应用时验证baseRevision、inputHash和manifest；从候选真实图片角色重新派生照片分配，对比锁定项和未选跨页→单次事务写revision。all＋forbid时任一遗漏／重复都阻断，与视觉得分无关。

3. 支付、退款与权益

v1规范将订单收款状态与履约状态分开：orders.state为pending／paid／closed／refund_pending／partially_refunded／refunded；fulfillmentState为unfulfilled／fulfilled／reversed／manual_review。创建后pending；已收款待履约为paid＋unfulfilled，完成为paid＋fulfilled；历史v0.1的created／fulfilling／fulfilled是过程说明，不再作为独立orders.state写入。付款事实不因退款抹掉。回调原始body验签，验证商户／应用／订单／金额／商品／币种，唯一payment_event；查单和回调共享履约事务。front-end success只触发查单。渠道需同步履约ACK时按其官方合同执行。

退款记录state为requested→submitted／unknown→succeeded／failed／manual_review；其中unknown只查询原退款操作。用户展示的“退款处理中／已退款”等是这些状态的映射，不与订单收款状态混用。先冻结可退款grant／lot，查询已有任务，再调用幂等退款adapter；未知退款状态查询不重复发钱。确认退款后一次撤销对应权益，其他订单权益保留。已消耗部分按公开政策人工处理或记录损失，不把余额改负数。正常退款不强制需要未配置的第二个人；异常人工财务覆盖可选配置更严格复核。

grant按来源订单＋grant_type唯一；同档续购start=max(now,current_end)，end=start+duration_days，初期不隐式跨档折算，不自动续费。权限实时由grants计算，定时任务仅通知／缓存。已接受任务使用entitlementSnapshot，排队期间到期不取消。

4. 点数与平台预算

available=未过期lot余额－有效reservation；lot按最近到期优先，金额与点数整数。接受quote时同事务锁定用户可用lot和稳定accountBillingId预算账户，检查并发、报价版本／有效期，建立reservation和job／outbox；缺一回滚全部。

结算operationId唯一且reservation状态条件变更，成功一次settle；失败一次release到原lot，赠送点数到期不因失败重置有效期。供应商状态未知保持预占并查询，建议30分钟终局后释放；迟到结果不重新发布／补扣，可能供应商迟到成本归平台并对账。

平台预算按采购账户／环境／北京时间日归属，同账户跨连接和模型共享；版本变更和午夜都不清掉未终局预占。停接新任务保留查询／结算；每日对账区分估计费用、实际费用和用户点数。

5. 存储、目录与删除

上传reserved→uploaded→pending_scan→ready／rejected，预占在失败释放，上传验证前不排版。folder移动在事务内查租户／角色／循环／深度／同名与revision。批量失败不隐含部分成功；引用中的素材阻断删除并显示影响。

对象locator不可从当前basePrefix反推。迁移copy→hash_verified→locator_switched→retained→cleaned；活动snapshot pin的旧位置保留，不能变成只读新位置的可变别名。取消迁移不会自动删除已经被新作品使用的副本。

逻辑trash立即限制新使用，但既有合法引用／活动任务按pin合同处理；物理purge必须再次查pin、保留期限和删除标记。账号注销立即撤销会话、分享和新任务入口；财务／审计按核验规则保留，用户原片依公开周期删除。备份恢复重放tombstone，防止删除数据复活。

6. 状态边界测试

每条状态边同时测试正常、重放、乱序、并发和崩溃恢复；不存在的边返回冲突，不允许管理员任意改成功状态代替正常账本／审核。数据库实际SQL保证项及服务事务保证项见database/transaction-safety.md，测试目录的cases.csv提供对应ID。

全模块实施与交接索引

用户已明确全程AI开发。以下均为待执行计划，实施前读取统一合同并核对当前代码，不先生成空模块冒充完成。C1是本地可运行闭环，C3/C4再接云存储与持久队列。

单元	计划	任务数	主依赖
C1	2026-10-21-c1-editor-render	10	文档与渲染技术合同
C2	2026-10-21-c2-layout	6	C1文档与模板
C3	2026-10-21-c3-cloud-identity-mini	8	C1端口；真实微信/云接入另验
C4	2026-10-21-c4-jobs-commerce	8	C3身份/权限＋C1渲染
C5	2026-10-21-c5-ai-admin	7	C3资产＋C4账本/任务/配置
C6	2026-10-21-c6-release-operations	5	贯穿C1 – C5，最终全链路

合计44项任务、220个执行步骤；每项都有文件职责、接口、失败断言、失败／通过命令和限定提交。全局追踪见requirements.csv，逐项任务表见execution-tasks.csv。

执行约定

推荐本会话按依赖顺序实施，单元完成后独立复核；如用户选择逐任务多AI实施复核，则按对应流程处理。当前全程AI偏好已确认，具体代码执行方式和最终基线评审在启动编码时落实；不再询问人工团队人数。

每项先确认有效失败，再实现该功能，跑针对性验证并提交；证据写docs/implementation/<unit>，服务模式区分mock/sandbox/live。必要的真实服务账号在接入时集中确认。技术选择造成接口变化时同步修改OpenAPI/schema/SQL/计划/页面/用例。

本轮统一修订

- API/SQL任务采用八个状态，确认上游用running＋providerSubmissionState=unknown；不另存unknown_upstream。

- 订单收款与履约分维度，refund使用明确查询状态；支付渠道统一wechat_pay/wechat_virtual。

- 图像wire format为jpeg，文件扩展名可.jpg；ZIP仅为多面／隐私打包产物，非单面格式。

- API模板场景包含ink；模板槽位、占位资源与实例绑定显式定义，草稿种子不是可直接发布的请求。

- 工作区采购／兑换／退款首期仅owner，admin管理设置／作品素材；页面中的workspace_admin是该工作区admin的界面名称，不能当平台管理角色。

- API采用NestJS＋FastifyAdapter，不在不同单元混用两套服务端框架；精确依赖和兼容性在首项技术验证锁定。

- 外部textSuggestion为可选预留关闭，图像GenerationInput不能当文本计费接口；手工标题编辑与规则排版独立可用。

全模块发布准出与交接责任

本表为未来执行清单；未勾表示未执行，不是缺失文档。文档可交接与产品可上线分别判断。

☐ 产品范围／默认假设／角色经研发评审；49需求都有有效实现、页面/API和验收证据，范围变更有记录。

☐ C1真实可编辑相册、保存恢复、33滤镜、模板、文字／图层和PNG／JPEG／PDF样片通过；字体、模板、滤镜、原片与renderer hash齐全。

☐ C2全量／重复／锁定／无解／局部应用／manifest重算与候选过期通过，不因得分高忽略硬约束。

☐ C3微信／网页身份、绑定／注销、工作区权限、私有上传／目录、云revision、mini白名单、分享通知实测；无跨租户访问。

☐ C4持久任务、Outbox恢复、租约fence／取消、配置轮换／回滚、会员／点数／兑换、支付退款对账实测，无重复履约、负余额或双扣。

☐ C5实际模型／商用来源、参考用途同意、参数／报价／预算、unknown查询、审核／标识、结果入册和后台实测；无重复付费提交。

☐ 所有P0通过，P1无未评估高风险；mock/sandbox/live/真机分别标记，blocked_external不当passed。

☐ 上传恶意文件、SSRF／DNS重绑、Cookie＋CSRF、令牌重用、角色撤销、回调篡改／重放、敏感信息扫描通过。

☐ SQL迁移、版本兼容、备份与对象一致恢复、tombstone重放、恢复点后渠道对账、未完任务重建有实际演练记录。

☐ 浏览器／mini真机矩阵、弱网／中断、真实平台支付能力、视觉／像素／DPI／页序和容量预算记录齐全。

☐ 真实经营主体、微信／商户／模型／云资质和价格／退款／隐私／保留政策确认；未开通能力禁用且对外范围准确。

☐ 生产秘密／权限／域名／证书、镜像digest、lockfile／SBOM、schema／migration／config版本、健康、告警目的地和回滚值可核对。

☐ 发布候选附commit、环境、真实报告索引、已知限制／风险处理、灰度监控与可审查部署步骤；生产发布按实际授权执行。

交接记录至少包括：本包版本与SHA、代码分支／commit（实施后）、每单元完成任务与失败项、外部接入能力、实际环境配置引用、未决业务项、最近可恢复版本、恢复／回滚证据。全程AI执行仍要产生这些证据，不能用“ AI已经写完 ”代替验收。

决策、假设与真实接入清单

本表完整记录尚未由经营者确定的输入，不将外部账号／价格伪造为技术缺失。所有模块先有默认禁用、契约模拟和可验收设计；真实能力启用前逐项关闭阻断。

ID	项目	当前基线／默认	何时确认／影响
DEC-01	自定义目录	暂定后台根前缀＋用户素材逻辑目录，两类均设计	C3接口评审前；可减范围，不阻断文档编制
DEC-02	云存储	OSS／COS／S3分别适配，平台统一私有存储	C3真实联调；选择供应商、区域、桶、域名、凭据权限、预算
DEC-03	AI服务	平台统一模型配置，无用户BYOK；三种原创候选风格	C5真实联调；模型／商业条款／数据用途／成本／标识审核
DEC-04	商品价格与额度	SKU版本化，无真实售价；模拟环境用明显TEST商品	C4上线前；价格／有效期／赠送／退款和毛利确认
DEC-05	微信能力	web／mini分开AppID，UnionID仅可信范围关联	C3联调；主体、类目、回调域、平台权限
DEC-06	支付渠道	网页与mini虚拟权益各自adapter，能力不足不开放购买	C4联调；商户与应用、凭据、证书、公钥、准入与iOS等限制
DEC-07	部署成本与可用性	独立API／worker、托管PG／Redis、私有对象；不用K8s作为首期前提	预发；云区域、实例、备份、告警接收、域名／证书、成本上限
DEC-08	保留与隐私	30天回收、7天导出为建议值，原片不默认训练	发布前；经营者确认公开政策与适用规则，数据库tombstone同步
DEC-09	AI开发执行	全程AI已确认；推荐本会话顺序执行，单元后独立复核	编码启动时确认本包基线和执行方式；不重新询问团队人数
DEC-10	模板与视觉	C1五类原创种子模板、33配方数值合同；视觉需真实评审	C1验收；字体许可、授权照片、中文、肤色／色卡确认

接入秘密只通过环境秘密配置或密钥服务注入；聊天／文档／Git中不收集明文key。无需用户逐字段设计技术参数，开发者依据schema、权限和测试记录落实。跨模块变更填change-log，标明影响的需求、API、数据库、计划、原型和验收，不能只更新一页。

产品代码实施按已启用Superpowers架构流程评审本包和单元计划后开始；用户本轮明确授权完整文档编制，因此不因等待增量文档批准而留下文档空白。真实付费调用／发布仅在其具体实现和费用可审查时按授权范围处理。

API v1 研发合同与接入说明

状态：设计交付，2026-10-21。权威机器合同为 openapi.yaml，共享约定为 CONTRACT-RULES.md。不代表接口、支付、微信、AI 或云账户已经实现、采购或上线。根地址 /v1，OpenAPI paths 不重复 /v1。接口环境由部署固定，业务请求不得切换环境。所有示例 ID、摘要及证明均为演示值，不含真实价格、密钥或采购结论。

合同组成与类型边界

OpenAPI 3.1 使用 JSON Schema draft2020-12。业务对象仅引用五个 schema 根：

API 字段	schema 根	用法
ProjectDocument.document、模板文档、候选文档	schemas/document.schema.json#/$defs/root	自有 DocumentModel 是唯一业务文档模型；不接受 Fabric 私有 JSON 代替
LayoutJobCreate.layoutSpec	schemas/layout.schema.json#/$defs/root	完整 LayoutRequest；包含 projectId/baseRevision/photos/constraints/templateRef/algorithmVersion/seed
PreflightRequest.exportSpec、normalizedSpec	schemas/export.schema.json#/$defs/root	ExportOptions：png/jpeg/pdf、DPI72/150/300、singlePage/spread、pageIds、封面、出血、JPEG底色
ConfigDraftCreate.value、ConfigDraft.value	schemas/config.schema.json#/$defs/root	按 kind 区分的类型化配置值；不是任意 JSON 编辑器
ProviderAdapterContract	schemas/provider.schema.json#/$defs/root	经过契约验证的适配器版本、能力与输入/结果限制

API 本地 schema 只定义传输信封、请求、列表、权限证明和业务摘要。FilterOperation.parameters 是唯一有意开放的键值配方字段：仅允许有限数字/短字符串、最多10项，仍需通过指定 recipeVersion 的不可变参数白名单；禁止脚本、CSS 和模板表达式。配置、项目保存、任务输入均无任意对象请求兜底。

所有公开 ID 是1 – 128字符不透明字符串，不能当成可信权限凭证；按具体共享 schema 的进一步限制校验。SHA-256 为64位小写十六进制。JSON 数值上限9007199254740991，金额为人民币分整数，点数为整数。时间传 UTC RFC3339；预算日、经营日期及交付展示使用 Asia/Shanghai。服务端校验引用存在、权限、跨字段不变量、唯一性和状态转换，不能只靠 JSON Schema。

响应与分页

业务成功：{data,requestId}；业务失败：{error:{code,message,retryable,details},requestId}，同时返回 X-Request-Id。禁止使用 HTTP200 包装失败。details 是已声明的诊断数组，不能回传秘密、上游原始错误、对象 locator 或其他工作区信息。500只返回通用信息和关联 ID。

游标列表 data:{items,nextCursor,hasMore}；limit默认20、最大100。末页 hasMore:false,nextCursor:null，有后页时必须有非空 nextCursor。游标绑定成员作用域、筛选、排序与稳定的 (sortKey,id) 边界；不得接收可修改 SQL offset。权限变化重新校验，不信任旧游标。首次请求可指定筛选，继续请求必须相同；过期或不匹配游标返回400 INVALID_CURSOR。列表在并发修改时不承诺数据库快照；不可变账本/版本列表可按 createdAt/id 稳定遍历。

支付/退款/供应商/通知回调是外部协议例外：成功 ACK 按登记适配器的确定格式发送，不能包装业务信封。健康 readiness503 也是明确例外：返回 ReadyStatus 的成功式诊断信封，表示探针状态而非业务成功。

三种身份边界

网页安全写操作采用 OpenAPI security 的 WebCookie AND CsrfToken，或独立的 MiniBearer 分支；不是 Cookie 或 CSRF 二选一。网页会话 Cookie 必须 HttpOnly、Secure、SameSite=Lax，登录后轮换；所有 Cookie 写操作校验 session-bound CSRF token 和批准的 Origin，CORS精确来源。GET只读取，不产生账本或资源变更。GET /auth/session 在已登录同源会话中返回 CSRF，不返回 Cookie token。

小程序使用短期 opaque bearer，访问 token建议≤1小时；刷新 token家族轮换、哈希存储、重用检测撤销。刷新接口使用其专用 refreshToken，不接受普通 Cookie 作为替代。客户端传来的 platform、openid、unionid、session_key 不能决定身份；平台及当前授权能力由会话和服务端适配器核验。APP-RESERVE仅保留共享接口模型；capabilities返回 app_reserved 不可用，不承诺 App 登录、支付或安装包已交付。

回调的登录 security 为 []，含义是“ 独立外部协议验签 ”，绝不是不验身份；x-independent-callback-verification.required:true 是硬性接入条件，见下节。后台使用同一安全会话机制并额外要求平台 RBAC、敏感操作目的绑定复验；普通 workspace owner不具备后台权限。

扫码流程是公开登录引导能力，不要求已有网页账号：

1. 浏览器先本地生成高熵browserNonce，再 POST /auth/web/qr-sessions {browserNonce}；首请求幂等范围绑定nonce hash及Origin，服务端另签匿名Cookie。接口校验 Origin、限速并绑定匿名浏览器 Secure HttpOnly nonce；sceneToken、pollToken 是不同随机能力。

2. 浏览器以 X-QR-Poll-Token 轮询自己会话；pollToken+原浏览器 nonce匹配，最短间隔1秒、总有效≤5分钟，不返回内部微信身份。

3. 已登录小程序使用 MiniBearer+sceneToken+显式确认调用 confirm。

4. 原浏览器带 pollToken、browserNonce、单次consumeProof调用 consume，原子消费并轮换网页 Cookie。该路径使用浏览器 nonce与Origin保护，不伪造已登录 CSRF要求；旧 Cookie不能替代能力验证。

5. 过期/消费/撤销不可再次用于登录；只有相同幂等请求可重放已确定结果，不能把证明转给另一浏览器。

短信 challenge限制用途、手机号/IP频率、验证码尝试和有效期；对不存在账户使用相同通用响应防枚举。登录保护还包括 CAPTCHA 和登录 CSRF的浏览器状态绑定。敏感操作先 POST /auth/reauthenticate，返回≤5分钟、绑定 actor/session/purpose的 reauthProof；后台秘密、财务、发布、运维及所有权操作还要求 X-Reauthentication-Proof。接口 body中声明的 proof与header在同时出现时必须相同并用途匹配；禁止跨会话复用。验证手机号或微信身份后才能生成证明，客户端不能自签。

工作区、资源与许可证权限

每个 operation 都有 x-required-permission。这是可实现的权限键，不是可由客户端申报的 scope。每次查询/写入先验证用户、工作区成员状态，再由资源记录推导真实 workspace，再校验角色/权限、资源状态、许可证和能力。只知道另一个工作区的 assetId、orderId、revision、outputId 或 jobId 不产生访问权；返回通用404避免存在性泄露。JWT或 Cookie成功不能替代任何一步。

角色	默认权限范围
viewer	工作区、已授权项目/素材只读及被允许下载；无上传/保存/创建任务/付款
designer	viewer能力＋assets.write、projects.write、layout/generation/exports创建；assets.delete由角色策略显式授权
admin	designer能力＋工作区偏好、成员（不提升到owner）、分享管理；账务购买/退款须显式授权
owner	工作区管理和账务授权、成员/席位/所有权转移；不能绕过平台限制或版权
平台角色	platform.config.*、platform.secrets.*、platform.content.*、platform.finance.*、platform.ops.*、platform.audit.read按职责分开授予

后台发布还必须按配置 kind校验专项权限：payment/sku/aiPricing要求财务，storage/retention/queue/monitor要求运维，content/share及模板/字体要求内容/许可，auth/account要求安全身份管理，aiConnection/aiRoute要求AI配置管理。通用 config.publish不是跨职责万能许可。记录操作人、理由、请求ID和前后hash，审计不可修改。

workspace owner转移只允许现有合格成员，需CAS和复验；不能借 member update删除最后owner。成员删除立刻禁止新资源访问和新下载令牌。个人与影楼权益、作品、素材不隐式迁移；首期不提供跨工作区 move或自动账号合并。付费/许可资源需后端检查，不能只隐藏前端入口。

绑定必须验证当前和目标身份；撞号返回 BINDING_CONFLICT，首期无隐式合并。解绑后必须有剩余登录方式。账户删除先列 owned_studio、活动财务/任务及合法保留阻断项，再安排可取消的删除期。撤会话/分享，按保留/备份策略处理物理对象；法规保留财务记录与用户内容删除分开。不可宣称立即抹除所有备份。个人数据导出需复验，仅本人数据及被许可清单，通过短期私有输出获取。

幂等与CAS

所有可重放创建、财务、任务写入，以及合同标记的普通写入要求 Idempotency-Key。建议安全随机16 – 128 ASCII字符；在工作区范围跨路径唯一，固定保留7天。无workspace的已登录操作按用户，预登录操作按受限浏览器/challenge范围。服务器规范化method、path、body和影响业务的参数，保存请求hash、首个确定状态码及响应。并发同key串行裁决；同key同输入返回原结果（含原resourceId，不重复扣款/扣点/发短信），同key不同输入返回409 IDEMPOTENCY_MISMATCH；处理中返回409 IDEMPOTENCY_IN_PROGRESS，配 Retry-After，不能再执行。超出7天不得假设仍幂等，客户端应查询已知资源后决定新操作。

幂等不绕过当前会话和资源访问检查；读取原响应之前重新鉴权，避免已撤成员取回敏感支付参数。报价过期、下载令牌过期或网页登录证明消费后，旧key仅重放原结果，不续期；新请求用新key并重检。回调不使用客户端 Idempotency-Key，而以已验证 channel/account/eventId唯一约束。

文档、文件夹、元数据、偏好、配置草稿和合同列出的修订写入要求 If-Match: "12"；禁止 *。缺失428 CAS_REQUIRED，陈旧409 REVISION_CONFLICT及当前revision，不能把409改为无条件覆盖。读取返回ETag；写入事务比较当前revision、追加不可变内容、更新指针、写审计/Outbox。body中expectedRevision或baseRevision存在时必须与If-Match、候选基线一致。幂等重放成功在CAS重检前识别，但仍重检当前权限；新key面对旧CAS必须冲突。

revision是资源局部计数，不可互换：项目metadata与文档写都更新项目revision；账号偏好、目录、素材元数据与配置草稿各有独立revision；历史文档revision及原字节不覆写。失败重试或恢复历史都会产生新revision/新job，不改历史。

主要模块调用顺序

素材：签发uploads并原子预占物理配额→仅上传受限对象→complete→HEAD、长度/hash、magic、EXIF方向、解码及恶意内容检查→pending_scan/ready或rejected。原片只接受JPEG/PNG，≤25,000,000字节且≤80MP；不接受任意URL素材。只在本工作区去重；归一化图和预览独立存储。字体上传走单独的后台有许可暂存接口，字体解析隔离。暂存key只能由服务端分配，客户端不能传bucket/key。

目录：元数据组织，与对象locator分离；名称去首尾空白，NFC+casefold唯一，1 – 80 Unicode码点，无斜线/控制字符/./..，深度≤10。move事务检查循环、同名、工作区与CAS。目录树删除/恢复、batch-move全有或全无；任一素材被相册引用则拒绝整树删除，不静默删片或部分成功。活动任务pin、法律保留及回收期后再物理清理；物理确认后才释放占用。basePrefix切换不重命名用户目录或旧locator。

文档：创建项目→读取自有document和manifest→PATCH typed commands→确认server revision才显示已保存。完整网页用replaceDocument保存任意受支持编辑操作；mini仅setText/setCrop/replaceAsset/reorderSpreads，且须在模板授权字段和服务端白名单内，不能通过clientPlatform伪装网页。未知同版本层原样保留，mini不得丢弃不认识字段；未知未来版本只读，阻断保存/导出。跨页1 – 50、照片≤500、每跨页递归层≤100、组深≤5。Schema之外验证重复ID、树循环、裁剪和比例、资源权属、不可变版本与预算。冲突只恢复草稿/用户明确重试，不自动覆盖或自动多人合并。

内容：场景统一wedding/travel/family/portrait/ink（国风），保留other作为扩展；项目创建、模板创建/响应和模板scene筛选使用同一枚举。Template.slots与TemplateVersionCreate.slots均为TemplateSlot数组，slotCount必须等于数组长度。每个slotId和(spreadId,layerId)唯一，递归定位的文档层必须真实存在且kind与image/text类型一致。图像必填槽bindingRequired=true，placeholderAssetId须等于对应image.assetId且已注册为有许可的不可变平台占位素材；ProjectCreate.templateBindings逐槽显式绑定ready且获授权的工作区assetId，slotId唯一且不得指向文字槽/未知槽；无templateVersionId时不能传非空bindings。项目实例化时必须替换为获授权工作区素材，否则TEMPLATE_BINDING_REQUIRED，不能把合成ID或任意URL当占位图。文字槽bindingRequired=false，mini字段只能text且为document.text.editableFields子集；图片mini字段只能crop/assetId，不能修改未授权层。content/template-seeds.json是原创槽位设计草稿，不是直接可提交的发布请求；先实例化完整DocumentModel图层、注册占位/示例素材与授权字体，再构建TemplateVersionCreate并审查发布。templates/fonts返回可见许可标记，不发永久公开对象URL。字体下载需授权和用途许可；不可嵌入不等于可无条件栅格商用。后台 staged upload→template/font version draft→许可证证据→publication，冻结字节/hash。退役限制新引用，历史记录不抹除；实际输出仍须按当前合法使用策略判断。SKU/预设通过类型化配置草稿创建版本并经publication/发布审批，不改历史订单版本。

排版：LayoutRequest.projectId必须等于路径项目；layoutSpec是唯一请求模型，photos中的尺寸/时间校验服务端素材，baseRevision必须存在且可读。规则排版不默认发照片到生图模型、不消耗生图点数；固定seed+算法/模板/素材版本可复现。必选/禁用/锁定、照片数量、重复与页数为硬约束，不能用总分抵消。无解422 LAYOUT_CONSTRAINT_UNSATISFIABLE，details包含冲突constraintIds。候选不修改当前文档；apply-layout需CAS及候选/当前baseRevision一致。

生图：v1收费图像协议仅接受textToImage/imageToImage；provider元数据中的textSuggestion仅为预留可声明能力，capabilities必须保持该独立能力unconfigured/disabled，直到专用文本输入/输出、报价/结算契约通过审查，不能套用imageCount/ratio表单。规则

排版不依赖它。读取已验证预设/能力→提交完整GenerationInput获取5分钟quote→接受quote/inputHash→创建job并在同事务预占用户点数和平台采购账户预算/写outbox。imageToImage要求所选参考素材权利和传输同意；仅选中参考图传输，默认剥离GPS/原文件名，不传整个相册。每个收费child1张，批量逐一报价/结算，不提供含糊的部分成功账本。采购成本只在后台，用户报价只显示点数。quote冻结输入、route/model/preset/config/price/policy，接受后配置切换不改任务。

任务：Job.progress为0 – 100百分比，与SQL相同；所有job绑定workspaceId，平台任务使用服务端创建的platform工作区，个人隐私导出使用本人personal工作区。公开createWorkspace仍仅personal/studio。数据库是唯一事实源，Outbox/BullMQ只调度。queued→running→packaging→succeeded，失败/取消终态互斥；临时故障retry_wait，提交状态未知保持state=running、stage=confirming_upstream、providerSubmissionState=unknown。租约和递增fence保护结果发布，临时对象只有当前fence成功落库才成为Output。取消若先赢则迟到结果不能发布，成功若先赢返回既有成功。供应商submit超时必须按原providerRequestId查询，不能换模型/跨供应商重复收费提交；无查询能力不得启用异步收费功能。明确成功且审核/标识通过结算一次，明确失败释放一次；终局超时按冻结政策释放后，迟到结果不扣点、不发布。jobs/retry不接受需要再次收费的生图，返回NEW_QUOTE_REQUIRED，重新报价。

计费：服务端SKU版本决定价格/渠道/平台，客户端不传价格。渠道规范值统一为wechat_pay / wechat_virtual，配置、API和数据库相同。Order.fulfillmentState使用unfulfilled/fulfilled/reversed/manual_review，paid+unfulfilled表示待履约，不能丢弃退款后的reversed和人工核对manual_review。order→payment-attempt→渠道launch→验签回调或受控上游查询→支付事实/一次履约/追加账本；客户端支付成功永远不直接发权益。微信支付/虚拟支付按当前商户准入、终端规则和已验证SDK合同启用，未知组合422 CHANNEL_UNAVAILABLE，不伪造可用性。Refund.state严格使用requested/submitted/unknown/succeeded/failed/manual_review；审核approve后进入渠道提交，reject记录failed+REFUND_REJECTED和审计，不引入approved/rejected持久状态。退款核验已消费点数/来源批次和剩余权益，原子申请/审核/渠道执行/回收，不能制造负余额或重复退款。兑换码不进日志，哈希存储，事务锁防双花和maxUses竞争；错误统一避免枚举。

导出：revision+ExportOptions预检→明确确认全部acknowledgementRequired issue ID→export-jobs提交绑定token/hash→冻结文档/资源/许可/权益/renderer/config版本与pin→逐面渲染→私有产物。缺原片/字体/授权、文字溢出/缺字、未知schema/渲染层、预算超限阻断；PPI100 – 199警告、<100强警告需明确确认。单面≤50MP；600×300mm、300DPI、四边3mm出血为7157×3614px。JPEG需不透明底色；普通PDF为高分辨率栅格页，首期无PDF/X、CMYK、可搜索矢量文字。每个preflightToken绑定actor/workspace/project/revision/spec hash/manifest/renderer/policy且≤5分钟；有blocker不发token。导出事务再次校验权限、资源与预算；token不能绕过后来被撤销的合法权限。

下载/分享：每次签发前校验成员或有效share scope，短期私有URL≤300秒。分享只包含冻结output集合，不包含原片、项目编辑权、工作区成员权；口令模式必须设置口令hash，公开访问失败不暴露原因。撤销后不发新token，已签发直连URL可用到期，不能承诺即时撤销；需即时撤销时必须另接验证过的网关/边缘鉴权。通知仅本人/实际订阅同意，发送失败独立重试，不回滚任务/支付终态，接口不能指定任意收件人或自由消息内容。

配置发布、秘密与运维

21个schema kind对应配置中心20主题：storage、aiConnection、auth、payment、sku、editor、layout、export、queue、content、share、notification、retention、monitor、workspace、account、site、upload、aiRoute、aiPricing、albumDefaults（归入相册默认值主题）。ConfigDraftCreate.category必须与value.kind相同，更新不允许跨kind。客户端/工作区偏好不能提高额度、跳过审核或覆盖平台停用。无有效发布为unconfigured，不等于通过连接验证。

草稿→CAS编辑→结构/权限/依赖检查→impact→所需分层tests→publish，原子release pointer。测试level统一为schema（结构）、connection（网络/认证）、storage_roundtrip（对象写读删）、capability（模型契约）、sample（真实样图）。ConfigTest.state是明确投影：queued/running来自关联config_test job，passed/failed/cleanup_required来自不可变结果，expiresAt过期或hash/secret改变显示expired，不修改旧结果。测试绑定draft/configHash/secret版本，≤5分钟；任何修改立即失效。connection（网络/认证）不代表对象读写删或模型能推理；storage测试使用随机temporary key PUT→GET→hash→DELETE→确认，仅触及本次测试对象。sample（真实付费样图）需操作者显式预计成本上限确认，来自平台测试预算，不能默认用客户点数/原片。没有实际执行不能登记passed。

发布冻结原有任务/订单/对象locator，新请求使用新版本。回滚是复验秘密与依赖后重新发布旧组合，不能回滚账本/文件/运行任务。紧急停用只停新请求，合法查询、结算、删除按规则继续。运行中的provider query仍需原冻结秘密；正常revoke检查pin，紧急泄露撤销需审计并安全终结，不因此重发收费任务。

SecretCreate/Rotate的value为write-only，日志/APM/审计不采集；返回只有mask、版本、状态、验证时间。配置里的secretBinding仅为ref/version。无show-secret接口。端点主机必须部署批准HTTPS白名单；禁loopback/私网/link-local/metadata、DNS重绑定、重定向绕过、任意headers/脚本。只有已实现并测试的OSS/COS/S3或AI adapter可启用；“ 兼容 ”或“ 认证成功 ”不自动代表所

有能力支持。

storage migration：plan估算范围/流量/成本和pin→显式成本上限confirm→逐对象复制→尺寸/SHA校验→CAS更新locator→保留旧副本/活动snapshot pin→引用核对→按retention清理。逐对象检查点重试；不能全局字符串替换路径；跨地域检查数据位置政策，旧profile仍有对象/pin时不能退役。basePrefix只改变新写入。

/health/live仅探针进程；/health/ready仅接收请求的基础依赖。实际capability就绪独立显示，不把某次检查当全站已验证。admin/operations/status返回脱敏监控；对账只能以渠道/账本/数据库事实和唯一operationId修复，不允许任意set-success。备份恢复演练只面向isolated_ephemeral目标，数据库/对象备份分别验证，重放删除标记；不可通过业务API部署生产或恢复覆盖生产。实际云账号、镜像摘要、商户资质、报警联系人与RPO/RTO由后续接入决策及演练确认。

回调原始字节与独立验证

路由：payment-callbacks/{channel}、refund-callbacks/{channel}、provider-callbacks/{adapterId}、notification-callbacks/{adapterId}。只接受服务端登记适配器和固定环境账户，未知channel/adapter404；不得根据body任意选择验证key。Content-Type application/json，body≤1MiB。OpenAPI给出微信支付加密信封和平台登记适配器标准信封；直接供应商若使用不同wire schema，必须先提交完整类型、验签/ACK/重放契约并审查后启用callback能力，否则采用query。不能把任意payload“兼容”作为已实现支持。

处理顺序是硬约束：

1. 保留原始HTTP实体字节与原始验签header，禁止JSON parse/restringify、更改空白、换行、解压后错误重建、代理header覆盖后再验签；部署层对签名相关header重复/歧义直接拒绝。若供应商签名规定压缩实体层，按其固定适配器合同选定字节层。

2. 按已登记channel/account/key版本解析必要timestamp/nonce/key-id/signature，校验已批准算法及轮换证书；微信支付使用Wechatpay-Timestamp/Nonce/Serial/Signature。时钟偏差默认±300秒，适配器协议更严格则取更严格；重放缓存不能替代数据库event唯一约束。其它适配器使用登记的X-Callback-*或明确供应商原header，禁止用户自定义。

3. 验证精确原始字节签名后，才解析和验证typed envelope；微信外层验签后才AES-GCM解密resource。验证事件类型与payload分支、商户/应用/采购账户、币种、金额、order/refund/providerRequest绑定及当前状态；不信任body自报的workspaceId/完成状态。

4. 事务记录不可变event摘要与UNIQUE(channel,account,eventId)，原子应用合法状态迁移/账本/Outbox。重复同事件返回既有ACK；相同eventId但内容hash不同触发安全告警，不覆盖。body、签名、完整提示词、原片、秘密不进普通日志；如需受限证据保留按隐私期限加密分离。

5. durable提交后才发供应商确定ACK。微信支付JSON {code:"SUCCESS",message:"成功"}；登记适配器可固定文本success。无匹配协议不返回通用成功。验签失败401/403；格式失败400；可恢复持久化故障500并依适配器触发重试；业务终态冲突记录核对，不能凭重复回调任意倒退状态。

6. 上游成功/取消竞争、重复回调、坏URL/大输出均走冻结job/fence和独立结算流程；结果URL仅服务端批准结果域名可抓取，重定向/字节/解码像素/hash再校验，quarantine→审核/标识→ready，客户端永不把上游URL当永久asset引用。

OpenAPI header参数因不同供应商头集合为OR而显示required:false；这不使签名可选。x-independent-callback-verification.required:true及登记适配器的完整必需header集合是必检条件，缺任何必需头即拒绝。IP白名单可加强防护，但不能代替验签。Cookie、MiniBearer、API管理员凭据都不能授权回调。

错误码与客户端决策

HTTP	code	客户端/服务端规则
400	INVALID_REQUEST、INVALID_CURSOR	修正参数/重开列表，不盲重试
401	AUTH_EXPIRED、CALLBACK_SIGNATURE_INVALID	重新登录；回调拒绝且不产生状态变更
403	FORBIDDEN、LICENSE_RESTRICTED	请求授权或更换合法资源，不重复提交
404	NOT_FOUND	不披露跨工作区资源是否存在
402	CREDIT_INSUFFICIENT	充值/更换工作区后重新报价，不预创建收费任务
409	REVISION_CONFLICT、BINDING_CONFLICT、ASSET_IN_USE、FOLDER_NAME_CONFLICT	刷新基线/展示冲突，不隐式覆盖/合并/部分删除
409	IDEMPOTENCY_MISMATCH、IDEMPOTENCY_IN_PROGRESS	前者停止并修正key使用；后者同key稍后查询/重放
409	PAYMENT_PENDING、NEW_QUOTE_REQUIRED、INVALID_STATE	查询权威事实；需要新报价时重新确认
410	QUOTE_EXPIRED、RESOURCE_EXPIRED、QR_EXPIRED	获取新报价/令牌；旧key不续期
413	PAYLOAD_TOO_LARGE、EXPORT_PIXEL_LIMIT	降低图片/页数/DPI后重新预检
422	ASSET_NOT_READY、ASSET_MISSING、FONT_UNAVAILABLE、FONT_LICENSE_REQUIRED、TEXT_OVERFLOW、GLYPH_MISSING	等待验证或替换资源；缺失不自动替图/替字体
422	UNKNOWN_SCHEMA_VERSION、UNKNOWN_RENDER_LAYER、PRECHECK_BLOCKED	升级/迁移或修复文档；不得跳过阻断
422	LAYOUT_CONSTRAINT_UNSATISFIABLE	展示constraintIds；用户调整硬约束
422	CONTENT_REJECTED、CHANNEL_UNAVAILABLE、PARAMETER_UNSUPPORTED	修改合法输入或使用已验证能力；不扣点/假装成功
428	CAS_REQUIRED	读取revision并提供If-Match
429	RATE_LIMITED、PROVIDER_RATE_LIMIT	遵循Retry-After，保留幂等key，禁止重复收费submit
503	CAPABILITY_UNAVAILABLE、PROVIDER_TIMEOUT、BUDGET_EXHAUSTED	展示disabled/unconfigured/degraded原因；未知submit状态先查询
500	INTERNAL_ERROR	展示requestId；先查询已知订单/任务/幂等结果，不无脑重新付款

任务error.code可包含CANCELLED（state=cancelled）、PROVIDER_TIMEOUT、CONTENT_REJECTED及上表业务错误；HTTP查询任务本身可200，任务业务终态通过data.state/error呈现。retryable只说明有安全恢复路径，不授权重新付费；客户端必须遵循该操作和provider状态规则。

研发验证边界

离线检查包括OpenAPI3.1语法、所有本地/外部$ref存在、operationId唯一、path参数匹配、typed request/response、每操作module/requirement/权限、示例schema和共享业务schema兼容。业务联调必须另验证：Cookie/CSRF与mini bearer、公开QR能力/nonce、跨租户404、CAS竞争、同key异payload、回调原字节/验签/重放/金额、账本一次结算、未知上游不重付、任务fence/取消竞争、字体版权/缺字、打印几何/像素、配置测试失效、迁移pin、备份删除标记及渠道准入。离线通过不能替代这些运行证据；本交付未调用任何真实付费服务或生产账户。

Schema语义、版本与示例

5份JSON Schema使用draft2020-12，无网络依赖，文件相对引用；每个均有$defs/root。document定义权威文档，layout定义请求／candidate／assignmentManifest，export定义选项／资源／快照／预检，config定义21类typed value覆盖20配置域，provider定义适配器元数据及提交／查询状态。

结构通过不代表业务通过。必须在API与worker校验：ID全局唯一、有限JSON数值、组递归总量、crop矩形和fit比例、工作区归属、素材ready与不可变hash、字体／模板许可、照片完整性和锁定、view与pageIds、像素乘积、资源内存预算、文字缺字／超框、警告确认、模型真实能力和预算。

config使用kind判别，kind等于路由namespace；API envelope与配置release独立。任何secret只能secretRef＋secretVersion引用；JSON schema禁止额外字段仍不能代替秘密分类和网络目的地校验。storage路径在URL百分号解码后检查前缀越界；元数据名称不拼云路径；endpoint需HTTPS、批准host、DNS／重定向重复验证。

跨字段约束：layout min<=max、必选／排除互斥、所有ID属于输入；config时长／租约／预算与adapter相容，weights和为1；export pageIds按文档顺序筛选、不重复、不把封面算入内页。根schema保留同版本未知图层／字段，导出遇未知层阻断；未知schemaVersion不能自动截断。

SHA对稳定canonical数据计算：对象键Unicode码点排序、数组原序、几何最多6位小数、严格JSON，不含NaN／Infinity。revision数据与引用文件hash不可变。导出snapshotHash绑定文档、manifest、选项、配置和创建时间；manifest不含签名URL或密钥。

examples为自制小型JSON输入／合同样例，不表示实际素材或供应商可用。invalid-cases.json列出应被结构或语义校验拒绝的变体；tools/validate_bundle.py会实际验证。API框架仍须使用生成DTO或等价validator，禁止只在文档包运行后假定服务端安全。

内容种子与许可边界

- filter-catalog.json：完整33项配方、强度、seed与原CSV SHA；kernel定义见filter-kernel.md。配方是设计合同，视觉状态仍required，不能伪造视觉验收。

- template-seeds.json：五类原创槽位设计（婚礼／旅行／亲子／写真／国风），含图框／标题／日期、可编辑字段和源坐标；不是直接提交admin templateVersion API的有效发布请求。实现需将槽位转成DocumentModel，注册有许可占位素材／字体，建立spreadId/layerId的TemplateSlot绑定，生成合法预览、licenseRef后再发布；替换绑定后产出真实文档，不以占位图作为客户成品。

- style-presets.json：山水水墨／人像淡彩／花鸟写意三份项目原创提示词草案，模型与报价未绑定且处于draft；模型商业授权／数据用途／审核标识与真实样例完成后才发布。没有调用受限Zine Skill或复制其样例。

- 字体推荐@fontsource/noto-sans-sc与noto-serif-sc各400/700真实字重、候选5.3.1/OFL-1.1，C1实施复制实际文件和许可证并计算SHA；文档不提供不存在的字体文件hash或把系统fallback当授权字体。

不可变内容版本、源文件hash、许可证据、发布时间、撤回／退役均进入后台登记。许可不明默认不可发布；会员商用权益不能替代字体／图片／模型的具体许可。生成结果来源、原始参考用途与标识随素材／导出保留。

CPU滤镜数值合同 v1

这是对 v0.1 留待固定算法的本次工程建议，需与计划一起评审；所有操作按 CSV 顺序执行。RGB 归一化至 [0,1]，每步 clamp，内部 float64，最后量化 round(255×C)；alpha 原字节不变。亮度 Y=.2126R+.7152G+.0722B。

操作	定义
exposure:e	clamp(C×2^e)
contrast:k	clamp((C-.5)×(1+k)+.5)
saturation:s	clamp(Y+(C-Y)×(1+s))
temperature:t	clamp([R+.08t,G+.02t,B-.08t])
tint:q	clamp([R+.04q,G-.08q,B+.04q])
gamma:g	C^(1/g)，g>0
grayscale:a	C×(1-a)+Y×a
sepia:a	标准矩阵行 [.393,.769,.189]、[.349,.686,.168]、[.272,.534,.131] 后 clamp，再与输入按 a 混合
duotone:dark,light	dark×(1-Y)+light×Y，颜色由十六进制转归一化 sRGB
posterize:L	round(C×(L-1))/(L-1)，L 为2 – 256整数
vignette:v	源图像素中心归一化到 x,y∈[-1,1]；d=min(1,sqrt(x ² +y ²)/sqrt(2))；RGB乘 1-v×clamp((d-.2)/.8)^2
grain:a	源整数坐标 x,y 与 seed 混合：h=seed XOR imul(x+1,73856093) XOR imul(y+1,19349663)；MurmurHash3 fmix32 后转无符号 n=h/4294967296-.5；RGB各通道加 a×n

fmix32：h ^= h>>>16；h=imul(h,0x85ebca6b)；h ^= h>>>13；h=imul(h,0xc2b2ae35)；h ^= h>>>16。grain 在同像素使用同一个噪声值；tile 坐标必须加 offset，因此切块不改变颗粒。

最终强度 s 使用 IEC sRGB 转线性函数（阈值 .04045，低段除12.92，高段 ((C+.055)/1.055)^2.4），混合 linear(original)×(1-s)+linear(effect)×s，再转回 sRGB（阈值 .0031308，低段乘12.92，高段 1.055×L^(1/2.4)-.055）。s=0 必须字节直通。不得把 s 乘进每个配方参数来冒充此混合。

全模块页面与交互合同 v1.0

状态：待实现、待研发评审的设计合同。2026-10-21；所有新原型为原创静态示意，无客户原片、真实交易、模型结果或真实收费报价。页面可见不代表服务已实现。继承 共同规则、原始 PRD、原型说明、v0.3 基线和 v0.4 配置清单。冲突按共同规则及本包统一合同处理，不修改历史版本。

1. 页面登记与角色的阅读规则

pages.csv 是唯一页面 ID 登记表：WEB-01～22、MINI-01～09、ADMIN-01～28、COMMON-01～06，共 65 个页面／复合视图。编号稳定，后续追加不能复用已退役编号。route 是产品导航设计，不是承诺存在的服务地址；mini:// 是小程序路由记法。动态参数只能引用当前授权资源，前端传入工作区不等于有权访问。

每页明确主要动作与 loading、empty、error、forbidden、offline、conflict 六类状态。非可编辑页的 conflict 指服务器状态或资源授权变化，不能为满足表格而提供“ 强制覆盖 ”。所有错误就地保留合法输入；敏感验证码、支付凭据、完整兑换码和秘密不持久化。跨工作区切换清空上一区的敏感表格及未提交财务选择；未保存文档先走保存／本地副本流程。

permissions.csv 是 54 个动作 × 10 个角色的矩阵。DENY 是默认；ALLOW 仍需登录、作用域、资源状态、平台能力、权益和服务端校验；CONDITIONAL 用于分享令牌；SCOPED 表示额外命名空间／字段／专项权限约束，绝非全后台管理员。guest 是访客；owner/workspace_admin/designer/viewer 属于工作区；support/content/finance/ops/auditor 是平台角色。一个人可以兼任，但每次操作须明确作用域，不把平台角色叠加成读取所有客户照片的能力。个人区只有本人；影楼首期单使用者，邀请与多席位管理仅预留，界面不得出现可用的邀请按钮。

界面矩阵的workspace_admin对应API／SQL的工作区role=admin，是同一角色的显示别名；平台ops等角色独立，接口不能接收显示别名来提升权限。

采购、订单详情、退款和兑换首期仅 owner；workspace_admin 管理区默认值与作品素材，不自动有财务权限。designer 可编辑和回收无引用素材；任一相册／活动任务引用都会整批阻断回收。viewer 只读获授权作品、历史、任务与产物，不提交导出、生图、取消、分享或恢复。页面登记的角色白名单与动作矩阵取交集；同一矩阵行涉及多页时，不扩大任一页的角色范围。未列出的操作一律禁止。平台人员仅看各域摘要；为审核必须临时查看隔离内容时，另需 purpose 权限、范围、期限和审计，不能添加通用客户原片列表。

2. 全端反馈、保存与版本冲突

保存指示固定使用“ 保存中／已保存〔时间〕／保存失败／离线，变更尚未同步 ”。仅收到服务端保存确认才显示已保存；不可将 localStorage 或画布重绘当作云保存。最新保存作品、未同步本地变更、排队任务各有独立状态。发生错误时标明最近已保存时间和恢复入口。

1. 每次编辑以加载的 revision 为基线。连续拖动或强度滑动先本地预览，操作结束合并一个历史记录，再自动保存。先完成历史记录再发送保存，失败不会清除撤销记录。

2. 路由离开或刷新且有待同步变更，应用内对话框标题“ 还有未同步的修改 ”，按钮“ 保存并离开 ”“ 保留本地副本后离开 ”“ 继续编辑 ”。保存失败停留并可选择副本；浏览器关闭仅使用原生 beforeunload 提醒，不承诺自定义文案或强行阻止关闭。

3. 断网保留最近基线和本地操作副本，禁止新的付费任务和云保存成功提示。小程序可能被系统杀死，不承诺长时间后台上传；重新启动展示实际已持久化的恢复副本。敏感提示词默认只在当前会话暂存，长期保存须有清晰偏好。

4. 恢复网络先读取远端。基线一致可同步；409 或远端已变进入 COMMON-04。两栏显示本地副本与远端的时间、跨页变化摘要，不把任意结构自动合并。提供“ 另存恢复草稿 ”“ 打开远端并保留本地副本 ”“ 返回比较 ”；有权限且资源完整才允许在最新基线上重新应用已检查的操作。

5. 历史恢复生成新 revision，保留被恢复前的版本；确认框明确“ 将以所选快照建立新版本，当前内容仍可在历史查看 ”。默认至少最近 20 个云恢复快照为建议配置；尚未生成不能伪造。撤销／重做至少 100 次有效操作，在内存预算内，重载后的历史深度如有变化须明确说明，不承诺浏览器永久内存历史。

6. 用户被撤权时立即停止云写入和新链接签发，遮蔽远端内容；允许保留已在本机的未同步副本并说明安全处理，不为其创建未经授权的新云项目。未知同版本字段原样保留；未来 schema 或未知渲染层可读摘要，阻断编辑性的破坏保存及导出，转兼容性说明。

配置草稿、目录和恢复同样使用版本比较。管理员发布冲突展示当前 release 与自己草稿的差异，不能覆盖当前发布指针。恢复草稿不含秘密明文。网络超时不等于提交失败：有幂等键的操作先查询已存在结果；不重新创建第二个订单／任务／兑换／退款。

3. 登录、绑定、注销与工作区

WEB-01 / MINI-01：先读已配置登录 capability；网页展示短期一次性微信二维码、剩余有效时间和刷新动作。二维码过期后禁扫码成功推断；扫描与服务端确认分别为“已扫描，请在微信确认”和“正在确认登录”。短信只在已配置并可用时显示，验证码倒计时不代替服务端限流。小程序由平台登录 code 换取会话，不读客户端 OpenID 作为登录凭证，不强索头像／昵称／手机号。

WEB-02：用户先验证当前身份，再验证拟绑定身份；只显示昵称／掩码及渠道，不显示 OpenID。目标已属于另一内部账号时，提示“此身份已关联其他账号，需要验证双方并检查作品与订单归属”，进入受控绑定处理，不按同手机号／昵称静默合并。首期没有安全的跨账号资产合并实现时明确“需支持处理”，不能声称已合并。解绑确认展示将失去的登录方式；不允许移除最后一种有效登录／恢复方式。绑定和解绑成功撤销有关旧会话，要求重登的范围需显示。

WEB-03 / MINI-01：注销先读取本人账号、所有关联区、活动任务、待付／待履约／待退款事项和分享影响。显示“注销后立即退出全部设备，不能再创建任务；照片及派生数据按公示期限删除，依法保留的交易与审计记录限制访问”。具体保留天数取已发布政策；没有政策则禁提交并显示未就绪。存在未结款项、未知生成任务、区归属未处理等阻断时逐项提供处理入口；不能把“等待确认”伪装已取消。通过后重新验证身份，要求用户明确勾选理解影响，再点“确认注销”；二次重复点击复用原申请。成功撤销会话、分享和新任务入口，并展示公开申请号与处理状态访问方式；不得承诺系统无法做到的即时物理删除。不是让 owner 用此入口注销他人。

WEB-04：区切换器总显示当前区名称，交易与生成确认框再次显示归属区；跨区素材移动首期拒绝。创建个人区不会把原来区的订单转移过去。到期或超配额保留已存作品、原片和合法生成结果；限制新上传与新付费任务，提供下载清理，不能悄悄删数据。

4. 素材、文件夹与上传

WEB-06 / WEB-07 / MINI-02：目录是逻辑元数据。面包屑、搜索、按名／时间排序、回收站和容量条独立于相册跨页排序。目录名去前后空白，1 – 80 个 Unicode 码点、NFC＋大小写折叠判同名；禁止斜杠、反斜杠、控制字符、. 和 ..，最多 10 级。名称校验失败就地提示；移动不允许成为自身子孙或跨工作区。

批量移动先显示“将 N 项移至〔目标目录〕”，确认后单次事务；任一项权限／目标／版本失败，显示整批未提交，保留选中项。目录／素材回收先预检子目录数、素材数、引用相册及活动任务数量。确认框“移入回收站”，说明不立即删除云对象；任一引用会破坏完整性时整批禁提交，提供查看引用及单独整理无引用项。恢复同名必须用户改名或选其他父目录，绝不覆盖。

上传选片阶段逐项检查 JPEG／PNG，单图 ≤25,000,000 字节、≤80MP，项目最多 500 照片；不能根据扩展名推断可用。HEIC／RAW／PSD 提示转换；WebP 若未通过兼容验证不接受。状态：待上传→传输中→验证中→可用／已拒绝。传输百分比只代表字节，不代表解码通过；ready 前不进入排版。取消未完成项不删除已就绪原图。配额不足保留已就绪项，剩余项说明原因；批量素材导入不是文件夹移动的原子事务，各项结果须如实可查。

重复检测仅当前工作区；提示“发现相同文件，使用已有素材／仍保留新记录”，不能泄露他区存在与否。不默认发送到 AI；EXIF 时间用于分组先说明，公开预览和成品默认去 GPS。上传凭据过期重新鉴权后续传或重传；仅 adapter 真正支持分片时显示续传，不能假承诺。网页上传若依赖浏览器文件读取，关闭页面会暂停未完成项；服务端已接受的生成／导出任务不受此限制。

5. 编辑器、模板与 33 个滤镜

WEB-08：桌面 ≥1280px 为完整工作区，1024 – 1279px 可收侧栏，<1024px 转只读预览和手机上传入口。顶部固定保存、撤销重做、排版与导出；左素材工具、中画布、右图层属性、底部跨页。页数始终分别显示“内页跨页”和“内页”，如“10 跨页＝20 内页”；封面正背独立、不计内页跨页。

图片、文字、矩形、圆和线支持选择、多选、组／解组、锁定、显隐、复制删除、顺序、对齐吸附、旋转缩放与透明度；100 图层／跨页（含递归）、组深度≤5。锁定图层不能拖动或参与局部重排变更。隐藏图层在图层面板保留并有文字状态。删除跨页确认显示页数和照片引用影响；复制跨页产生独立可变图层，不能改一页连带改另一页。

裁剪双击图片进入，亦有菜单／键盘“ 编辑裁剪 ”。只对当前图片显示图框、原图范围和主体／低清警告；cover 默认，contain 可选。图片平移缩放独立于图框。Enter／完成提交为一个操作，Esc／取消还原进入前裁剪。替换照片默认保留图框，重新计算合适裁剪并提示检查；绝不覆盖原片。

文字面板包括字体版本、字号、颜色、粗细、对齐、字距和行距；IME 组合输入期间不抢 Enter 或触发快捷键。溢出、缺字、缺授权字体以图标＋说明标记，可点击定位；换成建议字体必须确认并作为可撤销操作，不自动替换。

滤镜面板把“ 原图 ”单列，和稳定 ID F01 – F33 分开；名称和配方见 filters.csv。不得把原图算作第 34 个滤镜，也不得用 AI 生图代替配方。控件有 0 – 100% 整数强度、可键盘输入的数值框及可读名称；0% 视觉等于原图，但可保留已选配方。滑动仅预览，释放或数值确认记录一次操作；多照片“ 应用到所选照片 ”确认 N 张，将全部配方与强度作为一个撤销操作。没有图片选中、选到文字／形状时禁应用并说明“ 仅用于图片 ”；混合选择先标明将只改 N 个图片层。预览使用相同配方版本的低清结果，导出重用不可变版本；配方未发布／资源缺失不假显示可用。

WEB-10：按婚礼／旅行／亲子／写真／国风、尺寸、槽位、权限筛选模板。预览是隔离草稿，展示照片映射、文字替换、未分配项及将改变的跨页范围。用户可来回比较，关闭不会写文档。点击“ 应用到 N 个跨页 ”先确认覆盖范围、锁定冲突与授权；不兼容则禁用并提供改变范围／模板。应用成功为一个可撤销事务，旧文档在历史可恢复。预览期间本作品已改变则原预览失效，重新映射；不拿过期映射覆盖用户修改。

6. 照片排版、完整性与局部重排

WEB-05 / WEB-11 / MINI-03～04：入口“ 整理已有照片 ”与“ AI 艺术素材 ”分开，前者不创建生图任务、不扣生图点数。默认全部使用、每张一次；用户主动更改后才允许筛选或指定重复。自定义顺序不被 EXIF 顺序覆盖。可设置密度、页数、模板版本、必选禁用与锁定；封面和锁定跨页单独说明。

运行前检查可行性：50 张全用、10 跨页最多 2 张应显示“ 至少需要 25 跨页；增加跨页／提高每跨页上限／明确允许选片 ”，不自动删掉 30 张。结果返回实际有效候选数，推荐 3 个；不足 3 个如实显示。每个候选展示已用、未用、重复清单、硬约束结果及主体裁剪警告，可逐片定位。漏片、非法重复、越界、锁定改变或清单与真实文档不符的候选禁止应用，评分再高也不放行。

局部重排先选择明确跨页，面板显示锁定照片／图框／文字和保持不变的范围。“ 应用此方案 ”仅改选定范围，作为一次撤销；应用前再次核对基线、选片集合和真实候选内容。候选计算不是可靠任务中心的付费生图；若实现仅为当前请求，关闭面板可终止计算且可重新算。只有返回已创建 durable job 后才可承诺切端恢复，不把浏览器内 Promise 说成持久队列。

7. AI 创作、同意、报价与结果

WEB-12～13 / MINI-09：仅展示已发布且许可、模型契约、预算和审核标识就绪的风格。初始风格为山水水墨、人像淡彩、花鸟写意的设计目标，未经真实授权样例制作不展示为真实效果。用户只看公开名称、允许模型（如开放）、比例、强度等白名单参数；采购成本、endpoint、bucket、secretRef、供应商诊断只在授权后台。

参考图可选，选入后解释“ 仅将所选图片用于本次生成，可能改变人物和细节，不承诺人脸一致；请确认你有权处理图中人物 ”。明确勾选用途同意，记录政策版本与所选素材；拒绝可继续文生图或普通排版。不能把同意默认为全相册／后续训练用途；换参考图、用途或政策需重评有效同意。无权、回收或未就绪图不能选。比例、强度及可选 seed 等仅按模型支持显示，不支持参数不静默忽略。

点击“ 获取报价 ”只报价不预占。报价卡显示当前工作区、预设／输出规格、1 张图、本次点数、余额、5 分钟有效倒计时、成功结算／失败释放／未知保留预占的规则和条款；原型不预填真实点数。用户确认“ 使用 N 点开始生成 ”才提交；输入、参考、模型、比例或强度变化即清掉旧报价，重新获取。过期、模型停新、预算不足或授权变化时拒绝接受并解释，不能以新价格自动扣点。已接受任务有服务端任务号，生成中可离开、刷新、切端，在任务中心继续查看。

用户态顺序：已接受／排队→生成中→结果检查中（解码、审核、标识）→成功，或失败／取消；提交超时且上游未知显示“ 确认处理中，点数仍被预占，请勿重复提交 ”。按服务端冻结规则查询原任务；不是再次扣费重试。运行中取消提示“ 正在请求取消，最终结果以确认状态为准；若已完成将按成功结算 ”，不先显示点数已退。终局释放后迟到结果不能重新发布和扣点。用户只看点数预占／结算／释放，不看模型采购成本。

合格结果以私有素材入库，保留生成来源和标识。历史可看创建时间、状态、点数记录、来源、失败原因类别和申诉入口。审核拒绝、坏图或隔离结果不提供插入和公共回显。网页“ 加入素材 ”或“ 插入当前跨页 ”创建新图片层，不覆盖原片；标题、姓名、日期为独立文字层，改字不重新生图。小程序只能入素材或替换模板允许的照片槽，不新增任意高级图层。已用模板或图层变更导致冲突时先重选位置。换风格是新报价新任务，旧结果不自动删除。

8. 导出预检、队列关闭与下载

WEB-14～16 / MINI-06～07：先保存，选择 JPG／PNG／普通 sRGB PDF、页范围、singlePage／spread、72／150／300DPI、尺寸与出血。封面范围独立。展示实际输出像素，不以 DPI 标签假冒清晰度；600×300mm、300DPI、四边3mm出血为 7157×3614px。每输出面≤50MP，超过给降低规格／拆分的明确建议，不静默降质。PDF/X、CMYK、印厂拼版不在首期。

预检绑定保存版本、资源版本和全部输出参数。缺／无权／未就绪原片、字体缺失／无授权、文字超框／缺字、未知 schema／渲染层、非法几何、生成标识缺失、超资源预算均阻断；缩略图不可替代原图。可定位修复后重检。PPI100 – 199为警告，<100强警告且须明确逐类确认；近安全区／跨中缝警告亦可确认。确认框列出受影响照片和跨页，并说明低清不能保证印刷质量；阻断项无“ 仍然导出 ”。

文档／输出参数／资源或配置变化使原预检失效；提交前重检，成功冻结快照与提交时权益。排队期间到期不影响已接受任务。继续编辑产生新 revision，不影响原产物；详情使用“ 导出的是提交时已保存版本 ”而非混入最新编辑。

提交成功后给任务号和“ 可关闭此页，稍后在任务中心查看 ”。排队／处理／打包／成功／失败／已取消分开；没有进度数据时只显示阶段和等待提示，不伪造百分比与 ETA。取消不是删除任务记录；运行中到取消确认后才终态，成功与取消只能有一个生效。失败按原因提供“ 回编辑修复 ”或“ 按当前设置重新提交 ”，后者是显式新任务，新预检／权益，原失败记录保留；不能把无限重放作为免费补偿。

下载按当前权限签发短期链接，过期重新鉴权签发；产物本身过期（建议7天，取正式政策）时说明已清理，可在项目资源尚在时新建导出。小程序的图片保存需要相册权限，PDF／ZIP按实际平台文件能力提供预览／转发，系统不支持则诚实说明，不显示虚假“ 已保存到相册 ”。导出成功和订阅消息发送成功不同；用户拒绝通知不影响任务，可随时在任务中心查。

9. 会员、真实渠道限制、支付与售后

WEB-17～20 / MINI-08：先选当前工作区，显示有效会员天数、权益、点数批次可用／预占／将过期和用量。商品价格与数量只来自已发布 SKU；未确定价格显示“ 尚未开放购买 ”，不能用线框示意值冒充生产报价。期限按明确天数展示，续购同档展示起止；跨档折算未确定则不开放。首期没有自动续费，不默认勾选续费或营销同意。素材商用范围看具体许可证，去品牌水印不移除应有 AI 标识。

支付前确认区、商品版本、人民币价格、有效天数、生效方式、退款政策和渠道。WEB-18 显示“ 支付中／等待服务端确认／正在开通／已开通 ”；客户端 success 仅查单，不立刻给权益。用户取消支付可返回订单，关闭订单前先查单；若已扣款但履约未完成，保留原订单并提供查单及支持入口，不诱导再次付款。同一事件重复通知不重复开通，页面以订单和履约事实刷新，不靠“ 支付成功动画 ”判定。

小程序会员／点数属于数字权益，按服务端 capability 检查主体类目、虚拟支付、iOS／Android、客户端版本、应用商品映射与退款能力。未开通、停用、未配置、降级分别说明；不调用实物 wx.requestPayment 冒充准入，不自行放站外购买引导规避规则。网页渠道独立核验。实际平台资质未接入前，线框购买按钮仅静态示意，不能测试真实扣款。

订单保留收款事实，退款为独立进度。申请先读本单可退款权益／点数、已消费／预占／冻结，展示撤销哪笔 grant、不会影响哪些独立购买。未结任务先取消确认或结算后再计算；不从别的订单点数批次强扣。确认“ 申请退款 ”后显示申请中／渠道处理中／已退款／未成功；不得把已提交申请说成到账。正式政策未定时不承诺自动全退；超出自动条件进入支持和人工流程，并遵守适用消费者规则。强制退款遇已消费，记录财务差异不造负余额；前台只展示可理解状态。

兑换输入默认遮挡中段，不持久缓存原始码；提交前确认工作区和将获得的明确权益。无效／过期／已用通用提示，防枚举；网络未知先查幂等结果。成功只发一次并跳权益详情。后台停用仅阻止新兑换，不能无声撤回已合法所得。人工补点、延长、返还和撤销必须专项权限、原因、工单与操作前后摘要。正常渠道退款可由单个有权财务人员完成；对异常人工调整可另启更严的双人复核策略，启用后由另一finance.reviewer复核且禁止自审，不作为首期默认阻断。

10. 小程序范围与跨端接续

MINI-01→02→03→04→05→06→07 是完整快速成册路径；一屏一任务，触控目标至少44px。选片后先等待可用，设置明确内页跨页，候选支持滑动比较且有文字清单。轻编辑每次一个文字字段或图片裁剪，完成／取消明确，切跨页前同步或保留草稿。

高级图层、组合、形状、任意版式与网页独有字段以只读预览保留，提供“ 可在电脑网页继续编辑 ”的事实说明；不能把桌面提示变成购买绕行引导。服务端白名单 patch 不得用小程序可见字段重建整份文档。与网页同时修改按 revision 冲突处理，必要时另存恢复草稿。跨端使用相同内部用户、工作区、作品、任务和在线权益；App 仅保留数据接口，不新增已可用 App 页面或独立账本。

11. 分享、隐私与能力缺失

项目默认私有；WEB-21 可授权确定范围的成品预览与期限，不能把目录移动当作改变 ACL。确认框显示谁可看、哪些跨页、是否可下载和到期时间。复制的是分享链接而非永久原片 URL。撤销成功立即阻止新的鉴权；已签发直链在最长300秒等正式有效期内可能仍可用，不能承诺追回已下载文件；如业务需即时撤销须启用真正支持的下载网关并在策略中说明。

COMMON-06 先校验分享再取图片，未授权／过期／撤销统一不可用，避免泄露作品名或客户身份。公开成品默认去GPS，生成来源与标识继续保留。COMMON-05 展示协议隐私版本、参考图用途和删除请求入口；不同意生成用途仍可普通排版，通知订阅可拒绝退订，不把多个非必要目的合并为登录条件。

capability 的 available／disabled／unconfigured／degraded 控制每项新操作。未配置云存储可看已有缓存／恢复，但不显示上传成功；未配置 AI 可看旧合法结果但不报价；未配置支付不出现可用购买按钮；字体许可缺失只阻断相关模板／导出。前台只显示通用原因及可行动作，不显示供应商地址、根前缀、密钥引用或采购预算。服务端仍做同样校验，隐藏按钮不能代替鉴权。

12. 后台20域、发布与测试

ADMIN-01 概览仅展示授权域的负责人角色、当前有效版本、草稿、未配置项、最近测试范围与时间；不使用一个笼统“ 全站已验证 ”标签。下面20域一一对应 v0.4，无隐含设置页缺口。部署域数据库／Redis／secret服务根地址在部署配置，不在依赖同一数据库的动态表单。

配置域／页面	可写者与关键动作	失败、缺失和影响必须可见
系统／环境 ADMIN-02	ops：站点信息、白名单域名、协议、web/mini能力、维护	环境固定dev/staging/prod；请求不可传env切生产；协议缺失关闭相关处理
云存储 ADMIN-03	ops：adapter、region、bucket、批准HTTPS endpoint、秘密引用、根前缀、私有下载、测试	路径预览仅给后台；只影响新写入；无适配器实现不能启用；公共桶拒绝
用户目录 ADMIN-04	ops：10级上限、名称、引用保护、回收策略	策略不授予管理员看原片；不能关闭工作区隔离或静默删除引用
上传素材 ADMIN-05	ops：格式／字节／像素／配额、预览与未完成清理	上限不得越过合同资源预算；未验证不能ready
AI供应商 ADMIN-06	ops：批准端点秘密版本、超时限流并发、用途、预算	网络／认证、契约、付费样图三层报告；单认证成功不是可推理
AI模型路由 ADMIN-07	ops登记能力和参数白名单、不可变模型版本、默认路由；content只读	不支持图生图的聊天接口拒绝；未知提交仍查旧供应商，不能切路由重付
风格报价 ADMIN-08	content编辑风格许可与示例；finance编辑点数成本规则；ops只看依赖	缺许可／模型／审核禁止发布；跨域组合相容且各域均授权；旧报价和任务冻结
登录账号 ADMIN-09	ops：AppID、回调、秘密、会话、绑定注销、短信备用	微信／短信未接通不伪可用；不展示任何OpenID／秘密全文
工作区授权 ADMIN-10	ops角色结构；finance权益参数	不能让工作区自己提高平台配额；席位字段不表示邀请UI已交付
支付 ADMIN-11	ops技术接入证书应用映射；finance准入结算摘要	按端／系统版本列真实渠道能力；模拟验签不表示真实准入
商品会员 ADMIN-12	finance价格分、天数、权益、赠送、有效期、退款、停售	价格待经营者确认时不发布；停售不改旧订单／grant
相册默认 ADMIN-13	content尺寸跨页出血安全区默认模板；ops只读	仅影响新项目，不能重排旧相册；无合法模板可空白
编辑器排版 ADMIN-14	content模板字体33配方许可；ops硬约束评分候选预算	不允许任意脚本；硬约束失败不能以美观评分放行；版本不可变
导出 ADMIN-15	ops格式DPI视图像素内存保留；content只读许可关联	不越部署硬上限；不静默降质；旧已接受任务按快照
队列worker ADMIN-16	ops并发租约退避公平取消与OOM策略	只调度不改财务终态；不把Redis丢失说成任务丢失；硬上限不可覆盖

配置域／页面	可写者与关键动作	失败、缺失和影响必须可见
审核标识 ADMIN-17	content政策来源显隐标识申诉；ops连接秘密	缺审核／标识阻断结果发布；品牌水印与法定标识分开
分享下载 ADMIN-18	ops默认私有、期限、短链、真正网关能力	无即时鉴权网关不能承诺即时撤销已签直链；content只读
通知 ADMIN-19	ops渠道秘密；content事件模板	平台未开通／用户拒绝不发送；任务仍能查询；退订生效
备份删除 ADMIN-20	ops备份引用、恢复演练、对象版本、回收日志期限	备份文件存在不等于恢复成功；物理删除需无引用、保留期和活动pin检查
监控运营 ADMIN-21	ops失败率延迟资源与告警；finance成本预算；auditor只读	80%／100%预算阈值；当日已发生+预占不因换key归零；告警目的地受控

配置编辑流程固定 draft→validated→published→retired。编辑已发布版会新建草稿，不原地改历史；提交草稿含baseVersion，冲突比较。保存时结构校验，发布前校验权限、许可、依赖及测试。测试报告显示 testId、被测配置摘要、秘密版本、实际验证层级、耗时、错误类别和时间，5分钟有效；改配置或秘密立即作废。真实样图测试单独弹窗显示平台预计人民币成本和预算归属，明确“不会扣客户点数”，用户确认后才真实调用；尚无价格不允许发起有费用测试。

存储测试为独立随机temporary对象写→读→hash核对→删→确认清理；只列桶不算写读通过，删除失败展示残留清理告警，不删除既有对象。AI测试覆盖真实模型参数，隔离输出和标识检查；URL／DNS／重定向等安全拒绝只显示脱敏错误，不能发送秘密到用户任意地址。

ADMIN-22 发布确认展示命名空间、将用版本、依赖、测试范围、影响的新请求、费用参数变化、存储新写位置及保持的旧任务／订单。需专项config.publish、重新验证、原因；跨域组合检查操作者对各域的授权后原子发布；可由兼具所需权限的单个操作者完成，不默认要求多人批准。worker刷新失败仍使用旧完整release并告警，不拼新旧模型价格。回滚是复验旧组合后发新release，不能复活已撤销凭据，不回滚账本、不删新文件、不改历史订单任务。紧急停用是独立有原因审计的“停止新请求”，旧查询、结算、安全删除仍按规则继续；不是把成功任务改失败。

秘密只能输入新版本或选择已有引用，返回掩码、版本、最近测试、到期和权限摘要；无任何“显示完整密钥”。轮换确认展示新请求切换、旧任务固定版本及后续撤销计划。BYOS／BYOK首期关闭，不提供“填任意地址／key就兼容”入口。

13. 后台任务、对账、许可、迁移、审计与恢复

ADMIN-23 任务详情：support仅工单关联错误摘要；ops查阶段、尝试、冻结配置和供应商查询摘要；finance只看预占／结算／释放及成本差异；content按权限看审核。记录任务号和请求号，不展示客户完整提示词／原片／支付token。操作“查询上游”“请求取消”“修复后恢复处理”必须专项权限、工单、原因和影响预览。供应商状态未知先查询，不出现“直接再生成”按钮；不能手工设成功绕过可用结果和账本。用户需新任务时先新报价与明确确认。

ADMIN-24 对账：按日期、渠道、订单、差异类型筛选，支付／退款／履约／点数来源分别并排。差异不是重复发权益的许可。动作“查单”“提交复核”“按渠道退款”分别显示约束；退款请求显示本单金额、可退范围、冻结额度和进行中任务，禁止从无关批次扣回。正常查单对账和渠道退款由一个有权finance确认影响后执行，再等渠道确认／失败，不要求未配备的第二人员。异常人工补点、延长、撤销、返还按已发布策略执行；如额外启用双人控制，则提出→另一finance.reviewer复核→执行，禁止自审并保留拒绝意见。ops不代替finance发钱；support不能补点。回调重放与主动查单显示同一已确认事实，不制造新授予。

ADMIN-25 迁移：先选同环境来源／目标不可变profile及范围，估算对象数量、字节、请求费用和双份存储，确认“仅迁移已核验对象，活动任务可能继续保留旧副本”。运行显示扫描、复制、校验、条件更新、待pin释放、清理及检查点。hash不一致标失败且旧locator保留；断点续跑不会重复改已完成定位；暂停／取消只停止后续项，不回删合法目标或活动源。新prefix不是迁移按钮，不在用户目录改名时搬云对象。任何物理清理先证实无引用、无pin和到保留期，展示可清范围并确认。

ADMIN-26 许可登记：来源作者、条款／版本、商业范围、修改再分发、署名、地域、到期、证据、模型风格版本分别填写。非商业／未知／过期阻断新商业发布；停用不抹掉旧结果的来源和当时证据。许可过期对历史成品的影响依实际条款单独评估，不能自动宣称旧商业权永久存在。前台只展示用户所需的用途和署名提示。

ADMIN-27 审计不可编辑。按角色和目的限定检索时间、操作者、命名空间、请求号、原因及前后版本摘要；不存秘密／原片／完整提示词。导出另需audit.export、范围和原因，并审计此次导出。ADMIN-28 兑换批次原始码只创建时一次性受控下载；之后只掩码和不可逆记录，不提供“找回原码”。停用影响预览明确已兑换次数和未来受阻范围，追回应另走财务复核。

ADMIN-20 备份与恢复：展示数据库／对象／秘密引用覆盖、最近备份、恢复演练时间、实际观测RPO/RTO和失败项；无演练用“ 未验证 ”，不能将目标写成实测。恢复先在隔离环境选备份、检查依赖、校验对象和业务一致性、记录报告；生产切换必须具体范围、维护窗口、回退方案和专项权限，属于另一次受控运维动作。本设计稿不执行部署或恢复，不伪造结果。账号删除与回收清理使用同一引用／保留策略，必要审计财务保留不提供全员查看。

14. 确认弹窗的规范

所有破坏性／计费确认均含动作标题、当前区或平台环境、影响范围、可撤销性、按钮动词。默认焦点放标题或安全操作，取消不消耗资源。以下内容不能被一个泛用“ 确定？ ”替代：

触发	必须显示	确认后可观察结果
删除跨页	N跨页／N内页、照片仍在素材库、可撤销	单一历史操作，焦点移到最近剩余跨页
回收文件夹	子目录／素材数、引用影响、不立即物理删	整批成功或明确未提交；引用项阻断
模板／候选应用	替换范围、锁定、遗漏重复、基线状态	一次撤销，原文档可恢复
生图提交	工作区、规格、点数、有效报价、同意、失败规则	服务端任务号与预占，未知保持待查
低清导出	跨页照片、实际PPI、低清后果	绑定当前预检的明确警告确认
任务取消	任务、可能已完成、最终状态及结算规则	取消请求中，确认终态前不宣称退款
购买／兑换	工作区、商品权益或码归属、价格期限条款	服务端确认权益，仅发一次
退款	本单金额、源批次冻结、权益撤销范围、渠道	独立退款进度，未确认不说到账
注销	会话分享立即撤销、数据政策、未结项	申请确认与退出，真实删除异步可追踪
发布／回滚	环境、版本组合、测试、影响、原因	新release，可见审计，旧事实保持
迁移／清理	范围费用、校验、pin、保留与可回退性	可追踪持久任务，清理结果可审计

15. 无障碍、键盘与移动边界

页面用真正的button、link、input及heading；图标按钮有具体aria-label（如“ 将照片旋转90度 ”，不只“ 更多 ”）。保存、上传、任务阶段用aria-live=polite；阻断错误用alert并有文字，颜色不作为唯一编码。正文对比目标≥4.5:1，非文本关键边界≥3:1，支持200%缩放与reduced-motion，不闪烁进度。触控最小44×44px，避免依赖悬停。

弹窗role=dialog、aria-modal、标题关联，打开后焦点进入且Tab圈定，Esc取消（不可中断的已提交动作只关闭展示不撤销服务端事实），关闭回触发按钮；异步完成不突然抢走正在输入的焦点。页面加载不反复读全表；增量状态仅读变化。错误摘要可定位到字段，输入保留，屏幕阅读器可知道未通过的限制。

桌面Ctrl/Cmd+Z撤销、Ctrl/Cmd+Shift+Z重做；Ctrl/Cmd+S请求保存；Delete删除当前可删除选择；方向键1px、Shift方向键10px；Esc退出裁剪／取消当前选择；Tab沿工具、画布代理、属性、跨页导航按视觉顺序走。输入框、IME与原生浏览器快捷键冲突时优先编辑文字；屏幕阅读器使用等价工具按钮。所有拖放操作（排序、移动、裁剪、层级）提供菜单／数字输入／“ 前移 后移 ”替代。画布不可只靠一个canvas可访问：有可聚焦图层树、可读层类型与状态、属性表单和跨页列表。

小程序用平台可访问组件和明确标签，键盘不假定存在；底部操作避安全区，裁剪双指缩放有缩放滑条替代。高级图层只读说明不打断每次操作。二维码有同等可读登录方式说明，短信备用是否存在由真实能力决定，不能为了无障碍显示未实现服务。导出文件的内容可访问性另需实现验证，本静态稿不声称普通PDF自动具备完整标签结构。

16. 设计自检与可评审证据

本次离线自检检查65页面ID唯一、路线唯一、六状态非空、模块与需求ID合法；54权限行引用有效页面且每角色值均为允许枚举；本目录原型及历史引用文件存在；4幅新SVG可解析，无外部图像。结果记录见 validation.json。这是文档一致性证据，未执行支付、微信、AI、上传、队列、无障碍或业务端到端联调。

后续验收必须实际验证：viewer无写入口且服务端拒绝；双端冲突不丢高级层；关闭页面后已接受任务可找回；未知AI提交不二次扣点；支付前端success不提前开通；退款冻结来源正确；33配方逐一视觉验收；键盘完整编辑路径；20域未配置状态逐一

可见。详细业务验收与API以本包其他合同为准。

环境、部署、发布与恢复操作

1. 环境分离与秘密

dev、staging、prod分开数据库、对象桶／前缀、Redis、微信应用／渠道、AI采购账户和预算。正式应用镜像尚未构建，compose.review.yaml为后续集成拓扑模板；语法可校验，不能当作当前可启动产品。生产使用已构建并签名／锁定digest的镜像，静态站CDN＋TLS网关，API建议两个实例，独立worker，托管PG／Redis和私有对象。

environment.schema.json定义变量合同，.env.example只含非秘密样例与文件引用。应用须实现_FILE载入并验证文件权限，缺项启动失败。不会把真实URL密码或供应商key塞进示例。DB／Redis秘密和会话签名放挂载文件；供应商秘密通过受控secretRef版本加载，不在前端构建时注入。

变量	说明／生产约束
APP_ENV／SERVICE_ROLE	dev/staging/prod；api/export-worker/ai-worker等角色，客户端不能选择环境
PUBLIC_ORIGIN／PUBLIC_API_ORIGIN	精确HTTPS来源，CORS／CSRF匹配；本地dev可localhost
HTTP_PORT／TRUST_PROXY_HOPS	默认3000／1；只信任真实网关层数，防伪造来源
DATABASE_URL_FILE／REDIS_URL_FILE	挂载秘密文件，不输出内容；生产传输加密和最小数据库角色
SESSION_SIGNING_SECRET_FILE	生产强随机秘密且可轮换，禁止默认常量
SECRET_STORE_PROVIDER／SECRET_STORE_REGION	localFiles仅dev；生产按实际云秘密服务适配和workload身份
SECRET_STORE_CATALOG_FILE	dev秘密引用到本地文件的受控映射，不接受请求指定路径
CONFIG_REFRESH_SECONDS	默认30秒；取得完整release后原子替换，不混合配置
CONFIG_MAX_STALENESS_SECONDS	默认300秒；超时停止新写／claim并告警，保留在途终结和安全只读
WORKER_MEMORY_MIB／WORKER_CONCURRENCY	导出初始2048MiB／1，实测加容器余量后调整，最大值由部署硬限制控制
LOG_LEVEL／LOG_REDACTION	info／strict；不得记录token、照片、完整提示词或签名下载地址
RENDER_BROWSER_REVISION	实际Playwright修订；不得写latest，构建证据含版本和字体／kernelhash
RELEASE_ID／COMMIT_SHA	发布批次与真实commit，写入任务manifest及观测标签
API_IMAGE／WEB_IMAGE／ADMIN_IMAGE／WORKER_IMAGE	构建后填写；生产必须含sha256摘要，文档不伪造真实镜像
POSTGRES_IMAGE／REDIS_IMAGE	开发参考postgres17／redis7，实际测试记录确切版本；生产托管服务或锁定digest

2. 构建与预发

1. 对照需求追踪，锁定本次代码、OpenAPI、schema、SQL migration和内容／配置版本；执行单元、集成、浏览器和契约测试，保留失败详情。

2. 从lockfile构建web/admin/api/worker；固定Chromium和合法字体，生成SBOM／依赖许可／镜像摘要；运行无秘密扫描。没有实际产物不得填假digest。

3. 本地集成先准备秘密文件和非生产配置；用docker compose config检查拓扑后启动应用构建产物。compose模板不创建真实云桶／商户号／AI采购账户。

4. 预发加载空库执行up，验证schema；有历史数据时先备份并执行expand兼容迁移，再部署兼容双版本的API，最后做数据回填和contract。不要把删除字段与首次读取新字段放一版。

5. 运行healthLive／healthReady，后者检查DB、必要Redis、配置版本、secret绑定和自身角色所需依赖；AI缺配置只关闭生成能力，不阻断普通作品读取。

6. 在后台建立存储profile并测试临时对象写读删，建立内容／字体／滤镜版本，配置微信／支付与模型能力。真实样图先展示费用并取得操作者确认，使用测试预算。

7. 执行全链路E2E：mini选片／成册→web编辑→预检／队列→下载；支付小额真实下单／履约／退款；AI状态未知／迟到／成本对账。缺资质的能力保持unconfigured并在发布范围表说明，不标记通过。

3. 发布与回滚

发布前必须有测试报告、数据库／对象恢复记录、前后兼容说明、迁移校验、版本化配置、渠道准入、AI标识／隐私／许可、告警目的地和回滚值。内部工作区→5%→20%→100%为推荐灰度，每段观察保存成功率、任务错误与财务差异；实际时长依据流量记录，不用无请求的窗口证明稳定。

阻断级：作品丢失、跨租户访问、重复扣点／履约立即停相关写；未知供应商提交不重新发起。关闭新生成／购买／导出开关不删除已保存作品。API回滚必须仍能读新schema；数据库优先前向修复，不在真实业务期间执行down删表。回滚配置复验旧密钥有效，不回滚财务历史。

记录发布ID、commit／镜像摘要、数据库版本、配置release、执行时间（UTC存储、北京时间展示）、观测值和结果。没有值班人时先确定告警接收方式；文档完成不代替经营者的生产发布授权。

4. 恢复演练

数据库推荐连续归档＋每日备份，目标RPO≤15分钟、RTO≤4小时，必须实测；对象存储版本／备份覆盖原片、文档、字体和模板。Redis非事实源，可由jobs＋outbox重建；对象迁移不算备份。

恢复流程：停止支付／生成新写和outbox→复制备份到隔离目标→恢复PG至选定时间点→匹配对象版本并抽查manifest SHA→重放tombstones→对账恢复点后的支付／退款／点数／上游生成状态→重建未终局队列并使旧fence失效→只读验收作品和下载权限→小流量恢复新写。未与渠道对账前不重新履约或重新提交付费模型。

每月预发演练、每季度完整一致性演练；记录备份点、恢复耗时、账本汇总、孤立对象、未完任务与删除重放。超过保留期的原片不能因备份恢复重新对用户可见。

5. 故障操作

故障	保护动作	恢复检查
Redis丢失	暂停重复投递风暴，保留PG任务／账本	jobs/outbox重建，租约fence递增，终态／输出唯一
worker OOM	降并发、隔离输入，不盲试同预算	单面像素／字体／缓存峰值检查，明确要求用户改规格
存储不可读	缺原片阻断，不回退缩略图	权限／profile版本／locator／hash恢复后重新预检
支付到账未履约	查单与原事件共用幂等事务	原订单唯一grant／lot，账单一致；客服不直接再送
AI超时／限流	停新提交，查询原providerRequestId	终态／预占／迟到费用核对，不跨模型重复付费
配置刷新失败	使用上一完整快照、标记陈旧、限新写	新测试hash和依赖一致后原子切换；不恢复已撤销key
跨租户访问	停入口、撤销短期访问、保留脱敏审计	查所属／分享／签名边界，修复后负面回归

6. 监控与成本

核心仪表：API保存失败、CAS冲突、上传pending、导出排队／内存、任务失败和fence冲突、支付履约延迟、负点数／重复operation、AI估计与实际费用、存储活跃／回收／版本占用、备份失败和恢复耗时。初始阈值沿用v0.1：5xx>2%持续5分且有最小请求量、任务最老等待>10分、预算80%预警／100%停新付费、存储80%告警。

告警只发到经营者配置并验证的目的地；本次文档不发送任何外部消息。记录每有效生成成本、每本相册渲染和下载成本、用户原片／版本存储，价格决策不拿虚构账单推导。

全模块测试策略与证据规则

1. 测试资产

需求追踪49项统一需求；业务验收73项（49个主验收＋24个竞态／故障／边界）；API操作合同测试逐项覆盖全部155个操作。它们是待执行测试设计，不把文档编制当作产品通过。

contracts/examples和invalid-cases提供结构／语义样本；tools/validate_bundle.py实际运行文档合同校验。database/contract-checks.sql在隔离PG执行39个预期拒绝断言与合法样本，证据范围见transaction-safety.md。SQL检查不能代替API授权、并发账本、供应商或真机联调。

2. 测试分层与模式

层次	输入／执行	证据要求
合同／静态	OpenAPI3.1、schema meta、全部$ref、有效与无效JSON、CSV追踪、计划／页面ID、SQL解析	校验器版本／命令／结果及SHA；只证明对应合同属性
模型／规则	C1文档、方向／crop／滤镜／文字／几何、C2容量／manifest	确定fixture、seed、输入／预期、失败与修复；非法输入必须拒绝
数据集成	空库up／down、唯一约束、事务回滚、CAS／fence／预占、Outbox、去重	真实隔离PG／Redis／对象模拟，记录版本与负面断言；不能对生产库运行
API／安全	逐155操作的有效请求、缺字段／类型错误、匿名／错角色／跨租户、CSRF、CAS、幂等	预期HTTP／error.code、请求号，禁止在证据存token／原片
端到端	mini→web→导出，会员→订单→回调→权益，AI报价→结果→入册	明确mock/sandbox/live，基线快照及产物hash；mock不能满足真实准入关口
视觉／设备	中文禁则／肤色／色卡／透明／裁剪、浏览器／微信真机	固定字体／renderer，截图、人工结果、实际系统／基础库／设备型号
性能／恢复	阶梯会话、耐久、OOM／限流、备份、队列重建、删除重放	实际硬件、样本、负载、耗时／峰值、失败、费用与RPO／RTO

3. API操作级测试生成规则

api-contract-cases.csv为每operation标出method/path、权限分支、必要headers、request schema、成功／失败HTTP及需求。实现时依OpenAPI生成或手写用例，必须包括：最小合法请求、边界合法值、缺必填／错误类型／未知额外字段、401与403、存在但跨工作区资源、过期proof/token。带CAS写入测缺失428／陈旧409；幂等创建测同输入回放和不同输入409。callbacks用原始字节有效签名／篡改／重放／商户金额不匹配，不套Cookie鉴权；公开QR轮询仍需browserNonce／scene绑定。

只有适用才测该分支；不能给公开health构造虚假的必须401，不能给无CAS只读GET强制要求If-Match。业务状态／照片完整性／许可／预算等错误必须在结构校验后再测，不能以JSON Schema成功代替事务通过。

4. 渲染黄金样本

至少含横竖混合、EXIF6/8、暗部高光、肤色、灰阶／色卡、透明PNG、细线、长中文、标点、中英混排、组合字符、已知缺字、两种字体400/700。33滤镜强度0／.5／1，固定seed和tile边界；强度0字节直通，tile拼接等于整图。照片先归一方向、滤镜再crop与缩放，预览不采用另一套CSS效果。

PNG／JPEG重新解码验证尺寸；PNG pHYs、JPEG JFIF密度、PDF页数／MediaBox／TrimBox／BleedBox、singlePage／spread和独立封面分别检查。600×300mm／300DPI／3mm出血要求7157×3614。图像相同环境像素hash应重复一致；JPEG有损与跨浏览器边缘不做错误字节承诺。SSIM建议稳定区域≥.99，但漏字／漏片／页序错误无论分数都拒绝。

5. 浏览器与mini真机

桌面Chrome／Edge最近两大版本，Safari最近两大版本列兼容验证；正式版本填入真实矩阵。小程序当前稳定与确定最低基础库，Android低／中／高内存档、较旧／当前iPhone；覆盖弱网、微信权限拒绝、后台切换、上传中断、支付能力差异、切端任务找回。模拟器和API mock不等于真机结果。

6. 容量与发布关口

初始测试阶梯1→10→50→100活跃会话，每级15分钟稳定段、混合负载2小时耐久；200照片规则排版、50／200／500照片输入、最多100内页逐面导出。记录p50/p95/p99、错误率、队列等待、CPU／内存峰值、存储与供应商费用；用契约模拟做供应商限流压力，不轰击付费API。

沿用目标：普通API p95<500ms、保存<1s、拖动≥30FPS、小程序已有预览首屏<3s（20Mbps下行）、200照片规则候选<30s、20跨页300DPI处理<180s（8vCPU/16GB目标worker，排队另计）。这些是待验证预算，实际达不到就调整规格／资源和产品承诺，不能在报告里改成功。

准出：P0全通过；P1无未评估高风险；无作品丢失／跨租户访问／重复扣点／履约／负余额；真实微信／支付／AI链路和许可／标识核验；备份恢复／删除重放；平台能力和隐私政策；灰度／监控／回滚可执行。未准入的可选能力须明确disabled且不进入对外发布范围。

7. 报告格式

每条结果包括testId、requirementIds、commit、environment、serviceMode、inputFixtureHash、versions、startedAt/finishedAt（UTC）、status、actual、expected、evidencePaths、request/job/order IDs脱敏、knownLimitations。status只能passed/failed/blocked_external/skipped，后两者不能当通过。报告不存在时用planned_not_executed清单，不生成虚构运行数据。

事务、一致性与迁移边界（v1.0 设计合同）

本目录交付 PostgreSQL 16/17 的设计与可执行初始迁移，不交付 API、worker、支付/微信/存储/AI 适配器实现。共92表，覆盖身份、工作区、素材/目录、版本化内容、配置、商业/权益、点数、采购预算、任务、分享、通知、删除与备份。交易尝试、回调事实、履约、点数、采购成本和输出分别建表，使引用、金额与去重可单独约束。设计覆盖面达到全部模块；生产上线仍需服务事务、角色授权、真实适配器契约、并发/恢复/性能测试。

迁移与权限

- 001_initial.up.sql 在空数据库执行，显式 BEGIN/COMMIT，创建专属 ai_album schema；没有扩展、超级用户函数、外部连接或自动种子。所有表按依赖顺序创建，无循环 FK。PK、FK、CHECK、唯一键均命名且不与其他表同名。应用连接显式 search_path=ai_album,pg_catalog、TimeZone=UTC；不接受业务请求切换 schema、环境或采购账户。

- 生产需受控部署账号拥有 schema，业务账号不能是 owner、superuser 或 BYPASSRLS，不能 ALTER、TRUNCATE、DISABLE TRIGGER。按模块授予最小表权限；不可变/账本表只授 SELECT/INSERT，mutable 表才授所需 UPDATE。PUBLIC 已无本 schema 对象权限。迁移不自行创建真实运行角色或假定已有用户。

- 没有依赖 RLS 的隐含隔离。资源查询必须带 workspace_id，按当前成员/角色检查，复合 FK 拦截跨租户写引用。id 全局唯一不等于可绕过工作区鉴权。成员移除后不应继续发签名下载 URL；已有 URL 在 TTL 内仍可能有效。

- 001_initial.down.sql 是不可恢复的本地临时库清空，受会话 ai_album.allow_destructive_local_reset=yes 保护；该标志仅防误操作，不是生产识别或权限隔离。不得对生产执行。生产数据库回退用兼容的 forward migration，灾难恢复用已验证备份与墓碑重放，不能用 DROP SCHEMA 回滚版本。

- 不可变 SQL trigger 拦截 UPDATE/DELETE，不声称拦截表所有者或 TRUNCATE。数据到期的依法清理需单独维护身份、停服窗口/锁、审计及专门清理迁移；业务角色不能临时关闭 trigger。上迁移失败整个事务回滚；重复上迁移主动失败，迁移版本与 SQL hash 由部署工具记录。

SQL 强制与服务强制清单

不变量	SQL 直接保证	事务服务必须实现
ID/金额/时间	text domain 长度1 – 128；bigint 0 – 9007199254740991；SHA-256格式；timestamptz	安全随机生成，不从微信ID、序号或姓名构造；UTC序列化
所属工作区	复合 FK、workspace 不可修改 trigger	每个读取/写入鉴权；跨空间复制只能新资源和显式授权，不改 workspace
文件夹	同空间父级 FK、自父级拒绝、有效名称、活跃同父级规范名唯一（含 NULL 根）	NFC+casefold、去空白、祖先无环、≤10级、整子树深度、expectedRevision；所有目录结构写在同一空间锁下串行
不可变版本	revision、模板/字体/滤镜/模型/配置/快照等34表禁止 UPDATE/DELETE；存储对象 locator/content 不可修改	JSON Schema、未知同版本层保留、未来版本/未知渲染层阻断、完整引用/授权/配额检查
素材/上传	字节≤25MB、尺寸≤80MP、JPEG/PNG；ready/trashed必须有解码事实；初始化后字节元数据不可改	25MB为25_000_000；HEAD/magic/真实解码与hash；同一字节才可切换 copied object；不信客户端尺寸/metadata
项目当前版本	唯一(project,number)/(project,commandId)，新版本number=base+1，不可变	锁项目、CAS current_revision、插revision和引用、更新指针同事务；current_revision>0必须存在匹配版本
配置环境	profile/secret、provider/secret、active-release复合环境 FK；不可变发布版本	完整 schema、发布清单所有依赖同环境/可用/许可/测试hash匹配，CAS生效指针；不把SQL object CHECK当发布验证
金额履约	非负与界限；渠道事件唯一；一订单一grant_type、一兑换码一空间；provider交易号唯一	验签/商户/金额/币种/order匹配、合法订单状态、累积退款≤已收金额、不得重复履约
点数	lot各桶总和=granted；流水 operation_id 全局唯一；有效桶迁移；追加不可改	账户投影等于各lot/流水；按最早到期分配、预占总额=报价、分配总额=预占、一次终局、退款/到期补偿
采购预算	稳定账户/连接/当天 FK；非负桶；独立采购成本流水operation_id唯一	上海业务日、账户+连接双上限原子预占、预算0停止新任务、旧日未知预占继续保留、实际超估记账而非丢事实
任务	状态集合与转换；初始queued/fence0；revision+1，只有queued→running领取能使fence/attempt+1；终态不可改；输出写入锁住job并查当前未过期fence/状态	领取/续租/结果/取消条件写；fence仅领取新执行增加；snapshot/quote/job一致；调用上游前持久化提交意图；不重付
pin/输出	任务/对象/版本引用；output(job,index)唯一；私有对象locator不可改	全资源manifest逐项pin、全对象hash/权限；同事务输出+成功+账本+outbox；仅成功任务可下载；失败孤儿清理
幂等/分享/删除	作用域+key唯一（NULL平台作用域也唯一）、≥7天；token只存hash、墓碑追加	请求canonical hash比较与409；分享密码/到期/撤销检查；注销撤会话/分享/停止新任务、合法保留/备份删除重放

统一锁和事务重试

采用 READ COMMITTED 加显式行锁与条件更新；需要多行不变量的事务不能只读后写。统一锁序：幂等占位 → workspace（若目录/成员/容量/工作区余额涉及）→ project（若保存）→ 已存在job（若终局）→ order（若履约/退款）→ credit_account → credit_lots（expires_at,id排序）→ billing_account → budget_day → provider connection → provider_budget_day → reservation（id排序）→ 子记录。单事务若跨流程，跳过不涉及的锁但不倒序。新建 job 在接受事务内尚未对外可见，不等同于“ 先锁一个别的既有 job ”。维护任务遵同锁序；先读候选ID，锁后重新验证。目录和成员写锁 workspace；worker金融终局不需要反向锁workspace。

平台运维任务 privacy_export/config_test/reconciliation/backup_restore_drill 与其他任务同样需要非空 workspace_id；由服务端分配 kind=platform 的专用工作区，公开创建工作区接口只能创建 personal/studio，不能让客户端自选平台空间。SQL包含这些job kinds，不代表这些worker已实现。

所有锁内检查用数据库 clock_timestamp()（不依赖客户端时钟；now()在长事务内不前进）。业务事务有短 statement_timeout/lock_timeout，不在持锁时调用第三方网络。40001/40P01 可有限退避重跑整个事务；23505 仅在能识别命名唯一键时转成幂等重读或冲突，不能吞掉全部数据库异常。外部调用与未知 COMMIT 均通过operation_id/provider_request_id查询已存在事实，不凭一次连接错误再次收费。

工作区余额账号、当日预算行的首次创建：INSERT ... ON CONFLICT DO NOTHING，再 SELECT ... FOR UPDATE。不要以“没有行可锁”视为已排他。行内 CHECK 防负数，但跨表余额/预算总量无触发器自动维护：仅财务事务服务可执行相关写入，每次同事务更新lot、account、reservation、append-only ledger、outbox。

登录、绑定与注销

1. 微信 code 交换和回调证据在适配器验证；不存 session_key。生成一次性 auth_challenge，token仅存hash、expires_at。消费在行锁下校验 state/时效并条件改为 consumed；扫码确认不等于已发放会话。

2. 绑定需当前会话再认证证明和目标身份新证明。锁用户与挑战，尝试写 (provider,app_id,subject_digest)；唯一冲突返回身份已占用，不按昵称/手机号/无范围UnionID自动合并。UnionID只在经过验证的union_scope内使用。手机号/subject摘要应为服务端带pepper不可逆稳定摘要；pepper版本与可撤销加密映射在秘密服务托管，不进SQL/日志。

3. 会话刷新锁旧 session，校验 refresh_hash/过期/撤销，撤旧发新同事务，轮换重放触发会话链撤销；csrf_hash 用于cookie写请求验证。绑定/登录挑战state与bound身份/会话必须同事务，过期挑战定期清理。

4. 删除请求需再认证、延迟/确认与影响预览。锁用户/所属空间，处理影楼owner移交和未结订单、未知AI成本、法定财务保留；未解决时blocked。确认后撤销全部会话、绑定访问、分享和新作业权限；通知不发送敏感详情。

5. 资产先撤访问，再在引用/活动pin/保留检查后删除对象；写 deletion_tombstones。同一原子业务操作使用稳定operation_id。用户/工作区行保留去标识 anchor 以维持账本 FK，删除个人资料与外部秘密映射；不能物理级联删财务事实或把deleted状态当已清理对象。删除完成必须有对象清理结果和备份保留说明；备份到期前可恢复数据时不得宣称“所有副本即时删除”。

上传、目录、回收与存储迁移

- 创建上传先锁 workspace/usage_account，检查 used+reserved+bytes≤当前有效限制，插 quota_reservation 和 pending_upload asset、upload_session，记录服务端生成的profileVersion/bucket/key。asset在初始化阶段允许hash、object/dimensions为NULL；ready前必须完成真实验证。上传凭据本身不存库。外部上传无长数据库事务。

- complete使用幂等事务进入verifying，真实HEAD/解码/hash在事务外；最终按 expectedRevision/状态条件写已验证 storage_object，初始化asset不可变元数据并ready，扣reserved加used，同事务插必要outbox。重复complete只返回已完成结果。失败/超时释放容量一次；若对象已上传但未绑定，进入受保留策略保护的孤儿清理。原文件名仅私有展示，绝不参与key。对象locator创建后不可改。

- 所有创建/重命名/移动/目录删除/恢复先 SELECT id FROM workspaces WHERE id=:wid FOR UPDATE。在此锁下递归CTE查祖先和子树：拒绝目标是自己/后代、总深度>10、跨空间、已删目标、规范名冲突；expectedRevision条件写。所有目录结构写都遵循该锁，避免两个并发互换父级形成环；仅 CHECK parent<>id 不能防多节点循环。

- 删除目录先算整个子树和全部资产引用（current/retained project revision、活动job pin、法律留存）；任何会破坏引用的项目即整批409，不隐含部分成功。写 trash_operation/items 和原parent/name；事务内标记deleted_at与state。恢复校验原父级/目标、冲突、深度，冲突由用户改名/选目标，不覆盖；物理用量不因进回收站减少。

- 清理按复核revision、保留期、引用、pin、备份策略决定；锁顺序同工作区，禁止仅按创建时间在云桶生命周期直接删除仍引用原片。实际对象DELETE确认后才能减少物理used。未完成上传24h、回收30d、输出7d仅默认策略，发布版本冻结并由清理服务执行；活动作业pin优先。

- 默认profile/basePrefix切换只影响新写入。旧对象按历史row读取；basePrefix不得在读时重算。SQL只做字面前缀安全检查，应用还须拒绝百分号编码/重复解码逃逸、绝对路径、空段、控制字符，规范化后展开固定v1 token；endpoint/域名/CORS精确白名单由发布验证器验证。

- 迁移复制到新storage_object，校验字节/hash和供应商objectVersion，再在锁下按 asset.expectedRevision、source object/hash 比较并更新 assets.original_object_id（内容hash不变）。旧storage_objects row从不retarget；asset_variants不可变，重做variant需新pipeline版本。resource_pins存job所属空间和独立storage_workspace_id，允许引用已授权的平台字体/模板对象，不能据此读取其他工作区素材。跨空间pin必须是全局授权内容且版本对象匹配，由服务检查。

- 迁移切换只修改当前asset指针；冻结manifest和pin仍持有旧objectId/locator。旧副本保留到最后pin释放+保留期，逐对象检查点可续传。失败/回滚只改新请求路由，不删除副本；retired profile如仍有对象或pin不可撤读取凭据。

保存、候选与导出提交

- 保存以 If-Match字符串解析安全正整数。锁project并查current_revision，校验document.schema.json与引用：1 – 50跨页、≤500照片、每跨页≤100递归层、group深度≤5、坐标有限、无外部URL、未知同版本层原样保留。写文档对象后在事务内追加project_revision/asset引用并条件更新 current_revision；孤儿对象由回收清理。commandId重复同body返回原revision，异body冲突。schema迁移新建版本，不改旧记录；恢复草稿不移动current_revision。

- layout_jobs冻结base revision/约束/seed；layout_candidates为不可变候选，不更新文档。apply以当前revision CAS，验证candidate来自此项目、未过期、约束/资源/许可有效，再产生新revision；冲突保存恢复草稿或409。建议文字不被当代码执行。

- preflight固定revision与normalizedSpecHash，检查字体存在/授权/缺字、文字超框、未知schema/render层、原图缺失、PPI警告、目标像素。SQL仅可保证已存max_face_pixels≤50MP/DPI=72/150/300；全部业务检查和真实预估由服务负责。机器格式枚举固定jpeg/png/pdf（JPEG文件扩展名可为.jpg）；ZIP只作为多输出面归档/隐私导出包的job_output格式，不能作为exportOptions或preflight格式。job_outputs另外允许json供结构化结果。PPI100 – 199警告，<100强警告需明确确认；禁止把这些警告当可跳过缺资源阻断。

- 接受导出在同事务校验preflight token hash/时效/revision/specHash、无blocker、必要warning ack、当前权益和资源预算；插job_snapshot、job、精确resource_pins、必要quota预占、outbox、幂等响应。快照固定config release、font/template/filter versions、asset对象locator/hash、renderer版本和镜像digest；manifest必须完全闭合且受JSON Schema约束。job.kind/quote/snapshot/preflight/当前授权等交叉一致性由事务服务保证，不声称所有这些由FK覆盖。

订单、支付、兑换与退款

1. 创建订单在服务端读取已启用sku的不可变version，冻结CNY金额和渠道/平台，写幂等记录。支付尝试有独立attempt和稳定provider_request_id（商户号作用域）；客户端“支付成功”只触发查询，不能发权益。

2. 回调先独立验签/证书/时间/商户/app/币种/金额和provider transaction匹配。验签通过的事件按(channel,merchant_account_ref,provider_event_id)插payment_event，同事务锁order/attempt，检查状态和金额，改paid并插唯一entitlement_grant、credit_lot、credit_ledger、outbox。若同事件已存在，验证body_hash后读结果；若前事务未知，查询订单/渠道事实。一次provider transaction只能绑定一个attempt。

3. 拆开pay event和履约状态是为重放恢复；重放同事实只补缺少的唯一grant/lot/ledger。不可让管理员直接改订单为paid绕过事件与账本；人工调整必须明确审批、operation_id、补偿流水和审计原因。

4. 兑换锁code与batch，检查启用/expiry/used_count<max_uses、用户/空间权限；插redemption（同code同空间唯一），used_count+1，追加来源grant/lot/ledger/outbox同事务。批次停用与兑换须统一锁序；原始兑换码仅初次安全交付，数据库/日志只hash，接口限速且错误不暴露枚举信息。

5. 退款锁order、对应attempt和相关credit_account/lots；累计成功+待定退款预留不能超过已收金额。建立refund稳定operation_id，调用渠道在事务外，unknown先查询。渠道终局成功同事务反转grant/可撤销credit余额、追加revoke/补偿流水；已消费点数/已交付结果按公开退款规则转manual_review，不能改历史流水为0。活跃预占先查询/取消终局，不能在worker可结算时又释放同一批点数。需要取消时先挂起退款，在独立任务事务完成取消/终局后再重试退款，不持有order锁反向请求job锁。

6. 会员到期阻止新收费/超额操作，不自动删已有作品/素材；权益来自时间窗与策略版本，不能仅信subscriptions.state缓存。续费生成新订单/时间窗，不改旧SKU价格。源order/grant、引用版本及财务事实按法定期限留存并去标识。

点数、报价与稳定采购预算

报价阶段不扣款/不预占。generation_quote冻结configRelease、route/model/provider/preset/costRule、inputHash、consent/reference、点数/预估与上限成本、5分钟有效期，每子任务1张图。用户合法授权和明确传输用途单独generation_consent；不把全部相册上传。公开响应不暴露采购成本/平台endpoint/秘密。

接受事务：

1. 原子插幂等占位，重复key先比较canonical request_hash；锁所需工作区/余额。锁quote或利用 jobs(workspace_id,quote_id)唯一键保证仅接受一次。按clock_timestamp检验expires_at、输入hash、模型能力/路由/预设支持白名单、当前许可、参考图所属及可

用、config和secret允许新提交。已接受的同key重复返回既有job，即使quote后来过期，不重新预占。

2. 锁credit_account，按 (expires_at,id) 锁未到期可用lots。余额不足402；按FEFO分配 reservation_allocations，available→reserved，更新account，追加每lot reserve流水。quote点数为0允许零amount reservation、零allocation，不写零金额credit_ledger。

3. 采购日以 (:acceptedAt AT TIME ZONE 'Asia/Shanghai')::date 计算，稳定归属ai_billing_accounts.account_billing_id。锁 billing_account → budget_day → connection → provider_budget_day；检查账户/连接两者有效上限，当前 reserved+spent+quote.maxCost不得超限。0表示停新付费；密钥、connection version、模型/路由切换不创建新的账户预算桶。相同采购账户多connection共享budget_day，同时计各自connection上限。

4. 预算row的limit_fen是当日接受策略快照，运营降额不能抹去既有reserved/spent；锁同row更新limit后若已超限，只阻止新增，不拒绝记录实际费用。budget_days不以CHECK强制spent+reserved≤limit，避免实际超估费用无法入账；服务在接受新任务时必须检查两层上限。spent+reserved不得超JSON安全整数，极端溢出需财务异常队列和停新任务，不能丢账。

5. 同事务插snapshot/job、credit_reservation、budget_reservation和采购reserve流水、pins、outbox、幂等响应。预算预占归属接受日不因午夜/租约重试迁移到今日。提交后才可调度调用供应商；任何一步失败全回滚。

终局事务：先锁job并检查expectedFence/状态/lease，再依统一顺序锁余额、lots、采购账户及两个预算桶、两种reservation。成功且审核/标识通过：reserved→consumed，release采购保守预占、记actual成本，发布私有结果/来源，job=succeeded并terminal_at，同事务append ledgers、outbox。明确拒绝/失败：reserved→available（若lot未过期且grant有效），否则reserved→expired/revoked；采购成本按已发生事实记账，其余释放；job失败或取消。每个reservation仅从held条件转一次，operation_id唯一作为第二道防重，抢占失败回滚整个结果事务。

- lot到期与预占竞争：过期sweeper锁credit_account、lot，仅available→expired；reserved保持，不能偷走运行任务已占额度。若接受先锁并检查未到期，后到期仍可用既有预占成功结算；若过期先锁，接受看不到可用额。终局失败释放时重新检验lot时效，已到期部分进expired而非恢复available。账户available/reserved与lot投影同事务变更，不能先释放再扣。

- 未知供应商状态：提交前持久化stable provider_request_id与submission intent，网络调用不持金融锁。超时/进程崩溃/响应丢失把状态保持unknown/accepted，查询原request/job；不跨供应商、不凭429/超时盲目再次付费提交。只有确认从未接受且备用满足原报价能力/质量/价格，才可用固定备用，否则释放旧报价并让用户新确认。

- 终局时限默认30分钟可配置，expiry sweeper先按job锁与供应商查询结果/合同判定。仍未知到合同上限时，停止发布资格、释放用户点数与未发生的采购预占，job进入终态；迟到结果不再次扣点、不发布。已产生/迟到确认的采购成本进 provider_cost_ledger late_charge/reconcile并归原budget_day，可能超限后停新任务；不得为保持预算表面不过限隐去真实费用。

- 费用与点数分离：非收费query若供应商按次收费，也写采购成本。用户点数不随供应商实际价格变动重新扣。实际费用>max预估属于运营告警/策略校正，仍记录；采购对账不能修改已结用户ledger。取消仅表示请求，需上游明确终局或时限规则后释放，防止取消同时上游已收费的双花。

- 账本operation_id每个物理流水行全局唯一、opaque≤128；多lot同一业务操作用共同operation_group_id、递增operation_line与各自独立operation_id，不能多行复用一个ID。服务重试使用已持久化同ID，禁止临时重生成规避唯一键。余额/lot校正只写有原因的补偿流水，禁止UPDATE历史。

worker、租约、输出、Outbox 与幂等

- dispatcher按outbox未sent且available、claim过期选行 FOR UPDATE SKIP LOCKED，写有到期claim后提交；向BullMQ投递以 jobId dedupe，成功后条件sent_at。队列断线重投；投递成功但DB确认丢失可重复，worker按job事实去重。sent不等于job完成；定期用jobs/outbox修复队列，不用Redis重建余额。

- worker领取必须条件 state=queued（或先retry_wait→queued）、revision匹配，fence=fence+1, attempt_count+1, revision+1, lease_owner, lease_expires_at。续租只在id/workspace/current fence/owner/允许状态且 lease仍有效时更新revision+1；过期worker不能自行续回。后台回收器先按租约过期与provider_submission_state判断query/retry_wait，不能让unknown上游任务重新submit。

- 每次条件结果写 WHERE id/workspace/fence/revision/state/leaseOwner/leaseExpiresAt；0行表示失去资格，停止发布。SQL trigger验证fence单调和transition，但不能替代expectedFence WHERE，也不声称知道调用者worker身份。

- 结果先写独有临时object，真实下载校验、MIME、尺寸、SHA、审核/标识通过再进入结果事务。INSERT job_output触发器锁job并验证可发布state、当前fence和未过期lease；服务须在同事务完成账本与job=succeeded，下载只查succeeded任务。若只插输出不成功，此row不可被下载，后续错误清理/补偿由维护流程处理。旧worker/fence、已cancel_requested或终态均不能插新output。

- cancellation先条件queued/running/packaging/retry_wait→cancel_requested；运行worker在安全点按fence终局cancelled。若成功先锁并提交，终态trigger阻止改cancelled，取消API返回既有成功；若cancel_requested先提交，success/output都被拒绝。两个顺序的SQL检查已验证，但没有声称运行了真实多连接竞态压测。

- 最大重试默认3次（SQL可配置1 – 10）；invalid schema/font/license/budget等永久失败不重试。重试仅已确认无上游重复付费风险；OOM不得无变化重跑相同内存，重试parentJob产生新quote/权限/预占，旧终态不可复活。取消/失败/成功仅一个终态。

- output过期/分享撤销仅停止新签名；已发300秒URL不是即时可撤销。生成结果上游URL仅后端临时取回，允许域名、重定向次数/大小/时限/解码像素都由adapter验证，禁止任意URL/SSRF/原图GPS外泄。生成provenance冻结具体model/provider/preset/consent/许可/审核/标识，不伪造可重复像素保证。

- Idempotency-Key：工作区请求principal_scope固定workspace，同workspace跨operation仍唯一；登录/绑定/平台操作workspace=NULL且principal_scope为经过认证用户/平台或匿名流程的稳定服务端作用域。不能接受客户端指定scope。首次INSERT唯一冲突后等待/重读已有记录；相同hash已完成返回原状态/响应，异hash409，处理中返回可轮询状态不执行第二次。

- 幂等占位、业务写、最终响应同事务；涉及外部调用的流程先提交可恢复业务意图，未知结果由稳定requestId查询。至少7天保留，自created_at算，过期删除前确认无非终态业务；到期允许重用key不免除订单/grant/ledger/provider_request_id的永久财务去重。canonical hash覆盖method/path/主体/workspace与规范请求body，排除随机签名和传输header。

通知、分享、审计与恢复

站内通知以recipient+event_key去重，投递按channel+attempt记录；只从已提交outbox消费，偏好禁止时suppressed。第三方通知响应丢失先按provider_message_id查询，不能声称各渠道恰好一次。小程序订阅授权单独服务校验；保留App适配不表示首期已发送App通知。分享固定项目revision、用途scope和expiry，token使用高熵随机值只存hash，可选强密码hash；每次访问鉴权、口令、限速与撤销校验，分享不授权任意assetId下载。审计、通知payload只存脱敏业务摘要。

备份同时涵盖PostgreSQL、对象与配置版本引用；秘密由秘密服务独立备份，SQL只引用。恢复先隔离外发/付费调用，恢复数据库+对象至一致水位、重放 deletion_tombstones 到最新已知水位，恢复法律保留与撤销状态，再对账订单/供应商unknown任务/点数/预算/对象pins，用jobs+outbox重建调度，完成恢复演练后开放访问。恢复不得把已撤销分享/已删对象重新公开；RPO/RTO与备份保留期须在生产环境实测，本目录不编造达标值。

本次实际验证与后续门槛

2026-10-21，在本会话的隔离Docker临时库实际运行 PostgreSQL 17.11 与 16.15，未连接生产、未调用支付/微信/AI/存储服务。实际结果见本文末尾记录。contract-checks.sql 是可重复的SQL负面/正面样本，每次BEGIN/ROLLBACK，不是产品测试套件。两个版本都执行了完整SQL迁移与约束样本；最终jpeg/zip枚举对齐后仅重新执行PostgreSQL17，PG16记录适用于枚举修订前的结构与36项样本，不将其表述为最终文件已再次跑过PG16。

复现（仅有授权的临时空数据库）：

```
psql -X -v ON_ERROR_STOP=1 "$TEMP_DATABASE_URL" -f 001_initial.up.sql
psql -X -v ON_ERROR_STOP=1 "$TEMP_DATABASE_URL" -f contract-checks.sql
# 未设置本地标志时 down 应失败且 schema 保留
psql -X -v ON_ERROR_STOP=1 "$TEMP_DATABASE_URL" -f 001_initial.down.sql
# 仅本地销毁：在同一个连接设置标志再执行 down
psql -X -v ON_ERROR_STOP=1 "$TEMP_DATABASE_URL" \
-c "SET ai_album.allow_destructive_local_reset='yes';" -f 001_initial.down.sql
psql -X -v ON_ERROR_STOP=1 "$TEMP_DATABASE_URL" -f 001_initial.up.sql
```

待实施后必须增加真实多连接测试：双扣点/两连接预算穿透、上海午夜旧预占、lot到期竞争、退款与运行中job、folder并发互换父级、租约过期旧worker提交、成功/取消同时提交、outbox重复投递、DB COMMIT丢响应、provider已接受但submit超时、迟到收费对账、secret紧急撤销、迁移与pin/删除竞态、恢复后墓碑重放，以及全量schema/许可与导出像素样本。SQL检查不能证明这些服务协议已实现。

实际SQL验证记录：

检查	PostgreSQL 17.11（最终jpeg/zip修订）	PostgreSQL 16.15（枚举修订前）
空库执行初始迁移，ON_ERROR_STOP=1	通过	通过

检查	PostgreSQL 17.11（最终jpeg/zip修订）	PostgreSQL 16.15（枚举修订前）
contract-checks.sql 预期失败断言（逐个SQLSTATE匹配）	最终39项通过	修订前36项通过
有效上传元数据初始化、live-fence输出、终态与lot/ledger夹具	通过；新增jpeg preflight和jpeg/zip输出通过（最终ROLLBACK）	修订前夹具通过（最终ROLLBACK）
未设置local-reset标志时down拒绝，原schema保留	通过，psql退出3	通过，psql退出3
设置local-reset后down，再重新up	通过	通过
目录自检：92基础表/746约束/203外键/重复约束名0	通过	通过

容器使用 postgres:17-alpine 与 postgres:16-alpine，本次镜像内容摘要分别 sha256:b0f9560a2de083e2cc7382e75f808c7381a32852a7ec49117deedb300e552b24、sha256:721873c34ceb9f8d8fc265984940dc982404c105f19ad51be9fdc5970a6080ea；仅为本次临时验证环境证据，不是产品生产镜像锁定。容器无宿主端口映射，执行使用 docker exec -i ... psql -X -U postgres -d contract_test -v ON_ERROR_STOP=1。下迁移仅在这些临时容器运行。SQLSTATE负面断言分别覆盖23514、23503、23505；没有把顺序化状态断言称为真实并发竞态测试。

最终枚举补充验证：export_preflights.format=jpeg成功、jpg/zip分别以23514拒绝；job_outputs.format=jpeg/zip成功、jpg以23514拒绝。最终SQL在PostgreSQL17重新空库up、39项负面断言/正面夹具、down保护拒绝、guarded down/up及目录自检均重新执行。provider submit未知状态查询、双预算锁及终局释放仍是待实现服务事务要求；本SQL检查不证明这些跨供应商协议已经实现。

数据字典（v1.0 设计合同）

2026-10-21；继承 v0.1 架构实体与 v0.4 配置、目录、AI 增补。本文与迁移共同定义设计；不是已部署服务。

共 92 张表。跨身份、租户、不可变内容、配置版本、财务、供应商成本、任务、分享与运维归一化拆分；数量大于60是为避免将订单、尝试、回调、点数、采购成本和任务合并为无法约束的 JSON。首期可按模块实施，但首个生产数据库不得只落部分财务约束。

类型、所有权和引用约定

- opaque_id 是带 1 – 128 Unicode 字符约束的 PostgreSQL text domain；不是 UUID，不解析 ID 前缀。全部主键均为此类型，无自增。safe_uint 是 bigint，范围 0 – 9007199254740991；对外 JSON 永远不超过此界限。sha256_hex 为64位小写十六进制。

- 所有 created_at 默认 now()，时间列均 timestamptz；会话设置 UTC，驱动按 UTC 序列化。PostgreSQL 存储瞬时值，预算日 date 是按 Asia/Shanghai 计算后的业务日。

- 每表 id 主键和 created_at 创建时间不可改。含 workspace_id 的业务表同时 UNIQUE(workspace_id,id)，所有其下租户资源采用复合 FK，禁止单改 workspace 进行迁移。审计/墓碑/幂等等可含可空 workspace，表示平台或个人作用域，不作为资源越权入口。

- workspace_members.role 只授工作区角色，admin_role_assignments 为独立平台 RBAC。owner 成员与 workspaces.owner_user_id 的一致性在持有工作区锁的事务内保证。平台字体/模板的字节放入专用 kind=platform 工作区；用户引用全局已授权版本，不获得该工作区访问权。

- body/manifest/features/recipe/layout_json 等 JSON 仅承载有独立 JSON Schema 的可变结构；金额、状态、版本、归属、引用与关键索引均为关系列。JSON Schema、授权、大小限制和 manifest 引用一致性由服务验证，SQL 的 object/array CHECK 不替代它们。

- revision 为 bigint 乐观锁字段；应用接受十进制字符串 If-Match，按 expectedRevision 条件更新。job 的状态 trigger 额外要求每次 revision+1；其他表需服务执行 CAS。项目 current_revision 是版本序号而非公开 ID。

- 所有 FK 默认 NO ACTION、不级联删财务/作品。不可变表在 UPDATE/DELETE 时拒绝；TRUNCATE 和所有者禁用 trigger 不在业务权限内。不可变记录的到期脱敏/清除需单独维护权限和审计迁移。

- secret_references 仅存密钥服务 locator、版本、用途、状态；不存密码、token、微信 session_key、支付凭据。会话和分享 token、兑换码只存摘要；外部身份 subject 用带服务端 pepper 的稳定摘要匹配，pepper 的版本及可回收加密映射由秘密服务托管，账户删除时同步销毁。

- 状态 CHECK 限制可用值；除 jobs 明确状态 trigger 外，跨表状态机、金额相等、支付验签、时效和授权均由事务服务实施，详见 transaction-safety.md。config_tests、backup_runs 是将来运行证据表，不是本次测试记录。

表和字段

以下逐列列出精确类型、NOT NULL 和默认值；id/created_at 在每表显式列出。唯一键、CHECK、FK 的权威表达式在相邻 SQL，ERD 展示所有 FK。

users

内部用户与去标识状态。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
status	text NOT NULL DEFAULT 'active'
display_name	text NOT NULL
revision	bigint NOT NULL DEFAULT 1
last_login_at	timestamptz

字段	SQL 类型与约束
deleted_at	timestamptz

关系与唯一性：CHECK (status IN ('active','suspended','deleting','deleted')); CHECK (char_length(display_name) BETWEEN 1 AND 120); CHECK (revision 0)。

auth_identities

外部身份；加密映射保管于秘密服务，匹配仅用摘要。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
user_id	opaque_id NOT NULL
provider	text NOT NULL
app_id	text NOT NULL
subject_digest	sha256_hex NOT NULL
union_scope	text
union_digest	sha256_hex
verified_at	timestamptz NOT NULL
revoked_at	timestamptz

关系与唯一性：CHECK (provider IN ('wechat_mini','wechat_web','phone','app_reserved')); CHECK ((union_scope IS NULL) = (union_digest IS NULL)); UNIQUE (provider, app_id, subject_digest); FOREIGN KEY (user_id) REFERENCES users(id)。

auth_sessions

网页 cookie／小程序 bearer 的服务器会话。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
user_id	opaque_id NOT NULL
client_kind	text NOT NULL
refresh_hash	sha256_hex NOT NULL
csrf_hash	sha256_hex
expires_at	timestamptz NOT NULL
revoked_at	timestamptz
rotated_from_id	opaque_id

关系与唯一性：CHECK (client_kind IN ('web','mini','app_reserved')); CHECK (expires_at created_at); UNIQUE (refresh_hash); FOREIGN KEY (user_id) REFERENCES users(id); FOREIGN KEY (rotated_from_id) REFERENCES auth_sessions(id)。

auth_challenges

二维码、登录和再认证一次性挑战。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
user_id	opaque_id
kind	text NOT NULL
token_hash	sha256_hex NOT NULL
state	text NOT NULL DEFAULT 'pending'
expires_at	timestamptz NOT NULL
consumed_at	timestamptz
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (kind IN ('web_qr','login','binding','deletion')); CHECK (state IN ('pending','confirmed','consumed','expired','cancelled')); CHECK (expires_at created_at); CHECK (revision 0); UNIQUE (token_hash); FOREIGN KEY (user_id) REFERENCES users(id)。

binding_requests

绑定需现有身份再认证；身份冲突不自动合并。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
user_id	opaque_id NOT NULL
challenge_id	opaque_id NOT NULL
provider	text NOT NULL
app_id	text NOT NULL
subject_digest	sha256_hex NOT NULL
state	text NOT NULL DEFAULT 'pending'
expires_at	timestamptz NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (state IN ('pending','bound','conflict','expired','cancelled')); CHECK (expires_at created_at); CHECK (revision 0); FOREIGN KEY (user_id) REFERENCES users(id); FOREIGN KEY (challenge_id) REFERENCES auth_challenges(id)。

account_deletion_requests

账号删除工作流与法定留存检查。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL

字段	SQL 类型与约束
created_at	timestamptz NOT NULL DEFAULT now()
user_id	opaque_id NOT NULL
state	text NOT NULL DEFAULT 'requested'
not_before	timestamptz NOT NULL
confirmed_at	timestamptz
completed_at	timestamptz
reason_code	text NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (state IN ('requested','confirmed','blocked','executing','completed','cancelled')); CHECK (not_before >= created_at); CHECK (revision 0); FOREIGN KEY (user_id) REFERENCES users(id)。

workspaces

稳定资源归属；成员变动不改变资产 workspace。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
owner_user_id	opaque_id NOT NULL
kind	text NOT NULL
name	text NOT NULL
state	text NOT NULL DEFAULT 'active'
revision	bigint NOT NULL DEFAULT 1
deleted_at	timestamptz

关系与唯一性：CHECK (kind IN ('personal','studio','platform')); CHECK (state IN ('active','suspended','deleting','deleted')); CHECK (char_length(name) BETWEEN 1 AND 120); CHECK (revision 0); FOREIGN KEY (owner_user_id) REFERENCES users(id)。

workspace_members

成员角色；唯一 owner 与席位由事务服务保证。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
user_id	opaque_id NOT NULL
role	text NOT NULL
state	text NOT NULL DEFAULT 'active'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (role IN ('owner','admin','designer','viewer')); CHECK (state IN ('invited','active','removed')); CHECK (revision 0); UNIQUE (workspace_id, user_id); FOREIGN KEY (user_id) REFERENCES users(id)。

workspace_settings

空间偏好不能提升平台配额。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
revision	bigint NOT NULL DEFAULT 1
default_width_mm	numeric(10,4) NOT NULL DEFAULT 300
default_height_mm	numeric(10,4) NOT NULL DEFAULT 300
share_enabled	boolean NOT NULL DEFAULT false
notification_enabled	boolean NOT NULL DEFAULT true
settings_json	jsonb NOT NULL DEFAULT '{}'

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (revision 0); CHECK (default_width_mm 0 AND default_width_mm <= 2000); CHECK (default_height_mm 0 AND default_height_mm <= 2000); CHECK (jsonb_typeof(settings_json) = 'object'); UNIQUE (workspace_id)。

user_preferences

个人界面偏好。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
user_id	opaque_id NOT NULL
revision	bigint NOT NULL DEFAULT 1
locale	text NOT NULL DEFAULT 'zh-CN'
display_timezone	text NOT NULL DEFAULT 'Asia/Shanghai'
preferences_json	jsonb NOT NULL DEFAULT '{}'

关系与唯一性：CHECK (revision 0); CHECK (jsonb_typeof(preferences_json) = 'object'); UNIQUE (user_id); FOREIGN KEY (user_id) REFERENCES users(id)。

secret_references

秘密版本引用；从不存原始密钥。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL

字段	SQL 类型与约束
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
secret_ref	text NOT NULL
secret_version	text NOT NULL
purpose	text NOT NULL
state	text NOT NULL DEFAULT 'available'

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (state IN ('available','revoked','retired')); CHECK (char_length(secret_ref) BETWEEN 1 AND 512); CHECK (char_length(secret_version) BETWEEN 1 AND 128); UNIQUE (environment, secret_ref, secret_version); UNIQUE (id, environment)。

admin_role_assignments

平台最小权限管理员。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
user_id	opaque_id NOT NULL
role	text NOT NULL
expires_at	timestamptz
revoked_at	timestamptz

关系与唯一性：CHECK (role IN ('platform_ops','finance','content','support','auditor')); UNIQUE (user_id, role); FOREIGN KEY (user_id) REFERENCES users(id)。

config_drafts

动态配置草稿 CAS；发布前 schema 与连接测试。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
namespace	text NOT NULL
revision	bigint NOT NULL DEFAULT 1
schema_version	integer NOT NULL DEFAULT 1
body	jsonb NOT NULL
body_hash	sha256_hex NOT NULL
updated_by	opaque_id NOT NULL

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (revision 0); CHECK (schema_version 0); CHECK (jsonb_typeof(body) = 'object'); UNIQUE (environment, namespace); FOREIGN KEY (updated_by) REFERENCES users(id); UNIQUE (id, environment)。

config_releases

不可变配置发布；回滚是发布新指针。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
release_number	bigint NOT NULL
schema_version	integer NOT NULL
manifest_hash	sha256_hex NOT NULL
manifest	jsonb NOT NULL
published_by	opaque_id NOT NULL
previous_release_id	opaque_id

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (release_number 0); CHECK (schema_version 0); CHECK (jsonb_typeof(manifest) = 'object'); UNIQUE (environment, release_number); FOREIGN KEY (published_by) REFERENCES users(id); UNIQUE (id, environment); FOREIGN KEY (previous_release_id, environment) REFERENCES config_releases(id, environment)。

config_active_releases

每环境原子生效指针。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
release_id	opaque_id NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (revision 0); UNIQUE (environment); FOREIGN KEY (release_id, environment) REFERENCES config_releases(id, environment)。

config_release_items

发布中命名空间不可变快照。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
release_id	opaque_id NOT NULL
namespace	text NOT NULL

字段	SQL 类型与约束
schema_version	integer NOT NULL
body_hash	sha256_hex NOT NULL
body	jsonb NOT NULL

关系与唯一性：CHECK (schema_version 0); CHECK (jsonb_typeof(body) = 'object'); UNIQUE (release_id, namespace); FOREIGN KEY (release_id) REFERENCES config_releases(id)。

config_tests

明确测试目标 hash；报告不伪称已执行样图。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
draft_id	opaque_id
release_id	opaque_id
subject_kind	text NOT NULL
subject_id	opaque_id NOT NULL
subject_hash	sha256_hex NOT NULL
test_kind	text NOT NULL
state	text NOT NULL
report	jsonb NOT NULL
estimated_cost_fen	safe_uint NOT NULL DEFAULT 0
actual_cost_fen	safe_uint NOT NULL DEFAULT 0
executed_by	opaque_id NOT NULL

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (test_kind IN ('schema','connection','capability','sample','storage_roundtrip')); CHECK (state IN ('passed','failed','cleanup_required')); CHECK (jsonb_typeof(report) = 'object'); FOREIGN KEY (executed_by) REFERENCES users(id); FOREIGN KEY (draft_id, environment) REFERENCES config_drafts(id, environment); FOREIGN KEY (release_id, environment) REFERENCES config_releases(id, environment)。

audit_events

追加审计；禁止原图、完整提示词、密钥及令牌。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id
actor_user_id	opaque_id

字段	SQL 类型与约束
action	text NOT NULL
target_kind	text NOT NULL
target_id	opaque_id
request_id	opaque_id NOT NULL
reason	text NOT NULL
redacted_details	jsonb NOT NULL DEFAULT '{}'

关系与唯一性：CHECK (jsonb_typeof(redacted_details) = 'object'); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); FOREIGN KEY (actor_user_id) REFERENCES users(id)。

license_versions

字体、模板、模型与示例的不可变授权证据。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
license_key	text NOT NULL
version	integer NOT NULL
source_uri	text NOT NULL
terms_hash	sha256_hex NOT NULL
commercial_allowed	boolean NOT NULL
embedding_allowed	boolean NOT NULL DEFAULT false
reference_input_allowed	boolean NOT NULL DEFAULT false
valid_from	timestamptz NOT NULL
valid_until	timestamptz
territories	text[] NOT NULL
restrictions	jsonb NOT NULL DEFAULT '{}'

关系与唯一性：CHECK (version 0); CHECK (valid_until IS NULL OR valid_until valid_from); CHECK (jsonb_typeof(restrictions) = ' object'); UNIQUE (license_key, version)。

storage_profiles

稳定存储连接身份；旧版本只读保留。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
name	text NOT NULL

字段	SQL 类型与约束
state	text NOT NULL DEFAULT 'draft'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (state IN ('draft','active','readonly','retired')); CHECK (revision 0); UNIQUE (environment, name); UNIQUE (id, environment)。

storage_profile_versions

不可变存储定位配置，不随新根前缀改写旧对象。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
profile_id	opaque_id NOT NULL
version	integer NOT NULL
environment	text NOT NULL
provider	text NOT NULL
adapter_version	text NOT NULL
region	text NOT NULL
endpoint	text NOT NULL
bucket	text NOT NULL
base_prefix	text NOT NULL
key_pattern_version	text NOT NULL DEFAULT 'v1'
secret_reference_id	opaque_id NOT NULL
public_access	boolean NOT NULL DEFAULT false
download_ttl_seconds	integer NOT NULL DEFAULT 300
upload_ttl_seconds	integer NOT NULL DEFAULT 900
encryption_mode	text NOT NULL
allowed_origins	text[] NOT NULL DEFAULT '{}'
capabilities	jsonb NOT NULL DEFAULT '{}'

关系与唯一性：CHECK (version 0); CHECK (provider IN ('oss','cos','s3')); CHECK (environment IN ('dev','staging','prod')); CHECK (endpoint ~ '^https://'); CHECK (char_length(base_prefix) BETWEEN 1 AND 256 AND base_prefix !~ '(^/|\.\.|[[:cntrl:]])' AND position(chr(92) in base_prefix) = 0); CHECK (key_pattern_version = 'v1'); CHECK (NOT public_access); CHECK (download_ttl_seconds BETWEEN 1 AND 3600); CHECK (upload_ttl_seconds BETWEEN 1 AND 3600); CHECK (encryption_mode IN ('providerManaged','kms')); CHECK (jsonb_typeof(capabilities) = 'object'); UNIQUE (profile_id, version); FOREIGN KEY (profile_id, environment) REFERENCES storage_profiles(id, environment); FOREIGN KEY (secret_reference_id, environment) REFERENCES secret_references(id, environment)。

storage_objects

物理对象与定位；迁移创建新对象，资产 locator 条件切换。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
profile_version_id	opaque_id NOT NULL
bucket	text NOT NULL
object_key	text NOT NULL
object_version_id	text
kind	text NOT NULL
sha256	sha256_hex NOT NULL
byte_size	safe_uint NOT NULL
mime_type	text NOT NULL
state	text NOT NULL DEFAULT 'pending'
locator_revision	bigint NOT NULL DEFAULT 1
verified_at	timestamptz
deleted_at	timestamptz

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (kind IN ('original','normalized','preview','document','font','template','generated','export','temporary')); CHECK (state IN ('pending','verified','quarantine','deleting','deleted','missing')); CHECK (locator_revision 0); CHECK (char_length(object_key) BETWEEN 1 AND 1024 AND object_key !~ '(^/|[[:cntrl:]])'); UNIQUE (profile_version_id, bucket, object_key); FOREIGN KEY (profile_version_id) REFERENCES storage_profile_versions(id)。

asset_folders

逻辑目录；根为 NULL，NFC casefold 由事务服务计算。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
parent_id	opaque_id
name	text NOT NULL
normalized_name	text NOT NULL
revision	bigint NOT NULL DEFAULT 1
created_by	opaque_id NOT NULL
deleted_at	timestamptz

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (parent_id IS NULL OR parent_id <id); CHECK (char_length(name) BETWEEN 1 AND 80 AND name = btrim(name) AND name NOT IN ('.','..') AND name !~ '[/[:cntrl:]]' AND position(chr(92) in name) = 0); CHECK (char_length(normalized_name) BETWEEN 1 AND 240);

CHECK (revision 0); FOREIGN KEY (workspace_id, parent_id) REFERENCES asset_folders(workspace_id, id); FOREIGN KEY (created_by) REFERENCES users(id)。

assets

稳定 assetId 与不可变字节内容；文件夹/定位可事务更新。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
folder_id	opaque_id
original_object_id	opaque_id
original_filename	text NOT NULL
sha256	sha256_hex
byte_size	safe_uint NOT NULL
width	integer
height	integer
mime_type	text NOT NULL
origin	text NOT NULL
state	text NOT NULL DEFAULT 'pending_upload'
revision	bigint NOT NULL DEFAULT 1
deleted_at	timestamptz

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (byte_size BETWEEN 1 AND 25000000); CHECK ((width IS NULL) = (height IS NULL)); CHECK (state NOT IN ('ready','trashed') OR (original_object_id IS NOT NULL AND sha256 IS NOT NULL AND width IS NOT NULL AND height IS NOT NULL)); CHECK (width BETWEEN 1 AND 80000000 AND height BETWEEN 1 AND 80000000 AND width::bigint * height::bigint <= 80000000); CHECK (mime_type IN ('image/jpeg','image/png')); CHECK (origin IN ('upload','generated','import')); CHECK (state IN ('pending_upload','pending_scan','quarantine','ready','rejected','trashed','missing','deleted')); CHECK (revision 0); CHECK (char_length(original_filename) BETWEEN 1 AND 255); FOREIGN KEY (workspace_id, folder_id) REFERENCES asset_folders(workspace_id, id); FOREIGN KEY (workspace_id, original_object_id) REFERENCES storage_objects(workspace_id, id)。

asset_variants

归一化和预览不可变版本。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
asset_id	opaque_id NOT NULL
variant	text NOT NULL

字段	SQL 类型与约束
storage_object_id	opaque_id NOT NULL
pipeline_version	text NOT NULL
width	integer NOT NULL
height	integer NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (variant IN ('normalized','preview','thumbnail')); CHECK (width 0 AND height 0); UNIQUE (workspace_id, asset_id, variant, pipeline_version); FOREIGN KEY (workspace_id, asset_id) REFERENCES assets(workspace_id, id); FOREIGN KEY (workspace_id, storage_object_id) REFERENCES storage_objects(workspace_id, id)。

storage_usage_accounts

物理字节占用投影；回收站仍计量。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
category	text NOT NULL
used_bytes	safe_uint NOT NULL DEFAULT 0
reserved_bytes	safe_uint NOT NULL DEFAULT 0
limit_bytes	safe_uint NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (category IN ('original','derived','document','generated','export')); CHECK (revision 0); UNIQUE (workspace_id, category)。

storage_quota_reservations

上传/产物并发容量预占。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
usage_account_id	opaque_id NOT NULL
operation_id	opaque_id NOT NULL
bytes	safe_uint NOT NULL
state	text NOT NULL DEFAULT 'held'
expires_at	timestamptz NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (bytes 0); CHECK (state IN ('held','consumed','released','expired')); CHECK (expires_at created_at); UNIQUE (workspace_id, operation_id);

FOREIGN KEY (workspace_id, usage_account_id) REFERENCES storage_usage_accounts(workspace_id, id)。

upload_sessions

受限上传凭据不入库；完成后 HEAD/magic/解码。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
asset_id	opaque_id NOT NULL
quota_reservation_id	opaque_id NOT NULL
profile_version_id	opaque_id NOT NULL
bucket	text NOT NULL
object_key	text NOT NULL
created_by	opaque_id NOT NULL
expected_bytes	safe_uint NOT NULL
expected_mime	text NOT NULL
state	text NOT NULL DEFAULT 'issued'
expires_at	timestamptz NOT NULL
completed_at	timestamptz
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (expected_bytes BETWEEN 1 AND 25000000); CHECK (expected_mime IN ('image/jpeg','image/png')); CHECK (state IN ('issued','uploaded','verifying','ready','rejected','expired')); CHECK (expires_at created_at); CHECK (revision 0); UNIQUE (workspace_id, asset_id); FOREIGN KEY (workspace_id, asset_id) REFERENCES assets(workspace_id, id); FOREIGN KEY (workspace_id, quota_reservation_id) REFERENCES storage_quota_reservations(workspace_id, id); FOREIGN KEY (profile_version_id) REFERENCES storage_profile_versions(id); FOREIGN KEY (created_by) REFERENCES users(id)。

trash_operations

整批删除/恢复操作日志；有引用则整批拒绝。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
operation_id	opaque_id NOT NULL
created_by	opaque_id NOT NULL
state	text NOT NULL DEFAULT 'trashed'
restore_until	timestamptz NOT NULL

字段	SQL 类型与约束
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (state IN ('trashed','restored','purging','purged')); CHECK (restore_until created_at); CHECK (revision 0); UNIQUE (workspace_id, operation_id); FOREIGN KEY (created_by) REFERENCES users(id)。

trash_items

恢复原父级与名称；资产和目录二选一。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
trash_operation_id	opaque_id NOT NULL
asset_id	opaque_id
folder_id	opaque_id
original_parent_id	opaque_id
original_name	text NOT NULL
expected_revision	bigint NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (num_nonnulls(asset_id, folder_id) = 1); CHECK (expected_revision 0); FOREIGN KEY (workspace_id, trash_operation_id) REFERENCES trash_operations(workspace_id, id); FOREIGN KEY (workspace_id, asset_id) REFERENCES assets(workspace_id, id); FOREIGN KEY (workspace_id, folder_id) REFERENCES asset_folders(workspace_id, id); FOREIGN KEY (workspace_id, original_parent_id) REFERENCES asset_folders(workspace_id, id)。

storage_migrations

人工确认后复制/校验/切换，旧副本单独清理。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
source_profile_version_id	opaque_id NOT NULL
target_profile_version_id	opaque_id NOT NULL
state	text NOT NULL DEFAULT 'planned'
confirmed_by	opaque_id NOT NULL
estimated_bytes	safe_uint NOT NULL
estimated_cost_fen	safe_uint NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (source_profile_version_id <target_profile_version_id); CHECK (state IN ('planned','copying','verifying','switching','retaining','completed','failed','cancelled')); CHECK (revision 0); FOREIGN KEY (source_profile_version_id) REFERENCES storage_profile_versions(id); FOREIGN KEY (target_profile_version_id) REFERENCES storage_profile_versions(id); FOREIGN KEY (confirmed_by) REFERENCES users(id)。

storage_migration_items

逐对象可恢复检查点；pin 仍引用原对象。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
migration_id	opaque_id NOT NULL
source_object_id	opaque_id NOT NULL
target_object_id	opaque_id
expected_locator_revision	bigint NOT NULL
state	text NOT NULL DEFAULT 'pending'
verified_hash	sha256_hex
retain_source_until	timestamptz
error_code	text

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (state IN ('pending','copied','verified','switched','retained','cleaned','failed')); CHECK (expected_locator_revision 0); CHECK (target_object_id IS NULL OR target_object_id <source_object_id); UNIQUE (workspace_id, migration_id, source_object_id); FOREIGN KEY (workspace_id, migration_id) REFERENCES storage_migrations(workspace_id, id); FOREIGN KEY (workspace_id, source_object_id) REFERENCES storage_objects(workspace_id, id); FOREIGN KEY (workspace_id, target_object_id) REFERENCES storage_objects(workspace_id, id)。

projects

项目当前版本 CAS；创建事务必须插入 revision 1。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
name	text NOT NULL
current_revision	bigint NOT NULL DEFAULT 0
state	text NOT NULL DEFAULT 'active'
created_by	opaque_id NOT NULL
deleted_at	timestamptz

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (char_length(name) BETWEEN 1 AND 120); CHECK (current_revision >= 0); CHECK (state IN ('active','archived','trashed','deleted')); FOREIGN KEY (created_by) REFERENCES users(id)。

project_revisions

不可变文档版本；对象承载 canonical document schema。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
project_id	opaque_id NOT NULL
number	bigint NOT NULL
schema_version	integer NOT NULL
storage_object_id	opaque_id NOT NULL
document_hash	sha256_hex NOT NULL
spread_count	integer NOT NULL
photo_count	integer NOT NULL
created_by	opaque_id NOT NULL
command_id	opaque_id NOT NULL
base_revision	bigint NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (number 0); CHECK (schema_version 0); CHECK (spread_count BETWEEN 1 AND 50); CHECK (photo_count BETWEEN 0 AND 500); CHECK (base_revision >= 0 AND number = base_revision + 1); UNIQUE (workspace_id, project_id, number); UNIQUE (workspace_id, project_id, command_id); UNIQUE (workspace_id, project_id, id); FOREIGN KEY (workspace_id, project_id) REFERENCES projects(workspace_id, id); FOREIGN KEY (workspace_id, storage_object_id) REFERENCES storage_objects(workspace_id, id); FOREIGN KEY (created_by) REFERENCES users(id)。

project_revision_assets

文档资产引用索引支持引用阻断与回收。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
revision_id	opaque_id NOT NULL
asset_id	opaque_id NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); UNIQUE (workspace_id, revision_id, asset_id); FOREIGN KEY (workspace_id, revision_id) REFERENCES project_revisions(workspace_id, id); FOREIGN KEY (workspace_id, asset_id) REFERENCES assets(workspace_id, id)。

recovery_drafts

冲突恢复草稿；不冒充已保存 revision。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
project_id	opaque_id NOT NULL
user_id	opaque_id NOT NULL
base_revision	bigint NOT NULL
schema_version	integer NOT NULL
storage_object_id	opaque_id NOT NULL
body_hash	sha256_hex NOT NULL
expires_at	timestamptz NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (base_revision 0); CHECK (schema_version 0); CHECK (expires_at created_at); CHECK (revision 0); FOREIGN KEY (workspace_id, project_id) REFERENCES projects(workspace_id, id); FOREIGN KEY (workspace_id, storage_object_id) REFERENCES storage_objects(workspace_id, id); FOREIGN KEY (user_id) REFERENCES users(id)。

templates

template 稳定目录项。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
name	text NOT NULL
state	text NOT NULL DEFAULT 'draft'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (char_length(name) BETWEEN 1 AND 120); CHECK (state IN ('draft','review','published','retired')); CHECK (revision 0)。

fonts

font 稳定目录项。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
name	text NOT NULL

字段	SQL 类型与约束
state	text NOT NULL DEFAULT 'draft'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (char_length(name) BETWEEN 1 AND 120); CHECK (state IN ('draft','review','published','retired')); CHECK (revision 0)。

filters

filter 稳定目录项。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
name	text NOT NULL
state	text NOT NULL DEFAULT 'draft'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (char_length(name) BETWEEN 1 AND 120); CHECK (state IN ('draft','review','published','retired')); CHECK (revision 0)。

template_versions

不可变模板文档/槽位/小程序可改字段。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
template_id	opaque_id NOT NULL
version	integer NOT NULL
schema_version	integer NOT NULL
license_version_id	opaque_id NOT NULL
storage_workspace_id	opaque_id NOT NULL
storage_object_id	opaque_id NOT NULL
body_hash	sha256_hex NOT NULL
slot_count	integer NOT NULL
editable_fields	jsonb NOT NULL
constraints_json	jsonb NOT NULL

关系与唯一性：CHECK (version 0 AND schema_version 0); CHECK (slot_count BETWEEN 0 AND 500); CHECK (jsonb_typeof(editable_fields) = 'array'); CHECK (jsonb_typeof(constraints_json) = 'object'); UNIQUE (template_id, version); FOREIGN KEY (template_id) REFERENCES templates(id); FOREIGN KEY (license_version_id) REFERENCES license_versions(id); FOREIGN KEY (storage_workspace_id, storage_object_id) REFERENCES storage_objects(workspace_id, id)。

font_versions

不可变授权字体与字形覆盖清单。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
font_id	opaque_id NOT NULL
version	integer NOT NULL
license_version_id	opaque_id NOT NULL
storage_workspace_id	opaque_id NOT NULL
storage_object_id	opaque_id NOT NULL
sha256	sha256_hex NOT NULL
family_name	text NOT NULL
postscript_name	text NOT NULL
weight	integer NOT NULL
embedding_allowed	boolean NOT NULL
coverage_hash	sha256_hex NOT NULL

关系与唯一性：CHECK (version 0); CHECK (weight BETWEEN 1 AND 1000); UNIQUE (font_id, version); FOREIGN KEY (font_id) REFERENCES fonts(id); FOREIGN KEY (license_version_id) REFERENCES license_versions(id); FOREIGN KEY (storage_workspace_id, storage_object_id) REFERENCES storage_objects(workspace_id, id)。

filter_versions

33 滤镜配方不可变；CPU/GPU 共用版本。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
filter_id	opaque_id NOT NULL
version	integer NOT NULL
recipe_hash	sha256_hex NOT NULL
recipe	jsonb NOT NULL
renderer_version	text NOT NULL
seed_required	boolean NOT NULL
license_version_id	opaque_id NOT NULL

关系与唯一性：CHECK (version 0); CHECK (jsonb_typeof(recipe) = 'object'); UNIQUE (filter_id, version); FOREIGN KEY (filter_id) REFERENCES filters(id); FOREIGN KEY (license_version_id) REFERENCES license_versions(id)。

billing_skus

商品目录与不可变定价版本分离。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
code	text NOT NULL
name	text NOT NULL
state	text NOT NULL DEFAULT 'draft'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (state IN ('draft','active','retired')); CHECK (revision 0); UNIQUE (code)。

sku_versions

订单冻结实际 SKU 版本。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
sku_id	opaque_id NOT NULL
version	integer NOT NULL
currency	text NOT NULL DEFAULT 'CNY'
price_fen	safe_uint NOT NULL
duration_days	integer NOT NULL
credit_grant	safe_uint NOT NULL
features	jsonb NOT NULL
license_version_id	opaque_id

关系与唯一性：CHECK (version 0); CHECK (currency = 'CNY'); CHECK (price_fen <= 1000000000000); CHECK (duration_days BETWEEN 0 AND 36500); CHECK (jsonb_typeof(features) = 'object'); UNIQUE (sku_id, version); FOREIGN KEY (sku_id) REFERENCES billing_skus(id); FOREIGN KEY (license_version_id) REFERENCES license_versions(id)。

orders

支付事实与权益履约状态分别记录。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
sku_version_id	opaque_id NOT NULL
buyer_user_id	opaque_id NOT NULL

字段	SQL 类型与约束
currency	text NOT NULL DEFAULT 'CNY'
amount_fen	safe_uint NOT NULL
state	text NOT NULL DEFAULT 'pending'
fulfillment_state	text NOT NULL DEFAULT 'unfulfilled'
channel	text NOT NULL
client_platform	text NOT NULL
expires_at	timestamptz NOT NULL
paid_at	timestamptz
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (currency = 'CNY'); CHECK (amount_fen <= 1000000000000); CHECK (state IN ('pending','paid','closed','refund_pending','partially_refunded','refunded')); CHECK (fulfillment_state IN ('unfulfilled','fulfilled','reversed','manual_review')); CHECK (channel IN ('wechat_pay','wechat_virtual')); CHECK (client_platform IN ('web','mini','app_reserved')); CHECK (expires_at created_at); CHECK (revision 0); FOREIGN KEY (sku_version_id) REFERENCES sku_versions(id); FOREIGN KEY (buyer_user_id) REFERENCES users(id)。

payment_attempts

独立支付尝试与未知状态查询标识。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
order_id	opaque_id NOT NULL
attempt	integer NOT NULL
channel	text NOT NULL
merchant_account_ref	text NOT NULL
provider_request_id	text NOT NULL
provider_transaction_id	text
state	text NOT NULL DEFAULT 'created'
amount_fen	safe_uint NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (attempt BETWEEN 1 AND 100); CHECK (state IN ('created','submitted','unknown','paid','closed','failed')); CHECK (revision 0); UNIQUE (workspace_id, order_id, attempt); UNIQUE (channel, merchant_account_ref, provider_request_id); UNIQUE (workspace_id, order_id, id); FOREIGN KEY (workspace_id, order_id) REFERENCES orders(workspace_id, id)。

payment_events

验签后的追加事实；事件去重不等于完成履约。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
payment_attempt_id	opaque_id NOT NULL
channel	text NOT NULL
merchant_account_ref	text NOT NULL
provider_event_id	text NOT NULL
provider_transaction_id	text NOT NULL
body_hash	sha256_hex NOT NULL
verified_at	timestamptz NOT NULL
amount_fen	safe_uint NOT NULL
currency	text NOT NULL DEFAULT 'CNY'
event_kind	text NOT NULL
redacted_event	jsonb NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (currency = 'CNY'); CHECK (event_kind IN ('paid','closed','refund_succeeded','refund_failed')); CHECK (jsonb_typeof(redacted_event) = 'object'); UNIQUE (channel, merchant_account_ref, provider_event_id); FOREIGN KEY (workspace_id, payment_attempt_id) REFERENCES payment_attempts(workspace_id, id)。

refunds

退款在订单锁下限制累计金额并反转权益。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
order_id	opaque_id NOT NULL
payment_attempt_id	opaque_id NOT NULL
operation_id	opaque_id NOT NULL
provider_refund_id	text
amount_fen	safe_uint NOT NULL
state	text NOT NULL DEFAULT 'requested'
reason_code	text NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (amount_fen 0); CHECK (state IN ('requested','submitted','unknown','succeeded','failed','manual_review')); CHECK (revision 0); UNIQUE (workspace_id,

operation_id); FOREIGN KEY (workspace_id, order_id) REFERENCES orders(workspace_id, id); FOREIGN KEY (workspace_id, order_id, payment_attempt_id) REFERENCES payment_attempts(workspace_id, order_id, id)。

redemption_batches

兑换批次；原始码仅生成时一次交付。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
sku_version_id	opaque_id NOT NULL
name	text NOT NULL
expires_at	timestamptz NOT NULL
state	text NOT NULL DEFAULT 'active'
created_by	opaque_id NOT NULL

关系与唯一性：CHECK (expires_at created_at); CHECK (state IN ('active','disabled','expired')); FOREIGN KEY (sku_version_id) REFERENCES sku_versions(id); FOREIGN KEY (created_by) REFERENCES users(id)。

redemption_codes

摘要码与行锁防双花。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
batch_id	opaque_id NOT NULL
code_hash	sha256_hex NOT NULL
max_uses	integer NOT NULL DEFAULT 1
used_count	integer NOT NULL DEFAULT 0
state	text NOT NULL DEFAULT 'active'

关系与唯一性：CHECK (max_uses BETWEEN 1 AND 1000000); CHECK (used_count BETWEEN 0 AND max_uses); CHECK (state IN ('active','disabled','exhausted')); UNIQUE (code_hash); FOREIGN KEY (batch_id) REFERENCES redemption_batches(id)。

redemptions

兑换成功追加事实。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
code_id	opaque_id NOT NULL
user_id	opaque_id NOT NULL

字段	SQL 类型与约束
operation_id	opaque_id NOT NULL
sku_version_id	opaque_id NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); UNIQUE (code_id, workspace_id); UNIQUE (workspace_id, operation_id); FOREIGN KEY (code_id) REFERENCES redemption_codes(id); FOREIGN KEY (user_id) REFERENCES users(id); FOREIGN KEY (sku_version_id) REFERENCES sku_versions(id)。

entitlement_grants

明确来源的权益；到期不删作品。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
source_order_id	opaque_id
source_redemption_id	opaque_id
grant_type	text NOT NULL
starts_at	timestamptz NOT NULL
ends_at	timestamptz NOT NULL
features	jsonb NOT NULL
state	text NOT NULL DEFAULT 'active'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (num_nonnulls(source_order_id, source_redemption_id) = 1); CHECK (ends_at starts_at); CHECK (jsonb_typeof(features) = 'object'); CHECK (state IN ('active','expired','revoked')); CHECK (revision 0); UNIQUE (workspace_id, source_order_id, grant_type); UNIQUE (workspace_id, source_redemption_id, grant_type); FOREIGN KEY (workspace_id, source_order_id) REFERENCES orders(workspace_id, id); FOREIGN KEY (workspace_id, source_redemption_id) REFERENCES redemptions(workspace_id, id)。

subscriptions

首期固定周期会员；不隐含自动扣费续订。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
entitlement_grant_id	opaque_id NOT NULL
source_order_id	opaque_id
starts_at	timestamptz NOT NULL
ends_at	timestamptz NOT NULL

字段	SQL 类型与约束
state	text NOT NULL DEFAULT 'active'

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (ends_at starts_at); CHECK (state IN ('active','expired','revoked')); UNIQUE (workspace_id, entitlement_grant_id); FOREIGN KEY (workspace_id, entitlement_grant_id) REFERENCES entitlement_grants(workspace_id, id); FOREIGN KEY (workspace_id, source_order_id) REFERENCES orders(workspace_id, id)。

credit_accounts

工作区余额投影，必须与 lot/ledger 同事务更新。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
available	safe_uint NOT NULL DEFAULT 0
reserved	safe_uint NOT NULL DEFAULT 0
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (revision 0); CHECK (available::numeric + reserved::numeric <= 9007199254740991); UNIQUE (workspace_id)。

credit_lots

按来源与到期日批次；预占可跨到期完成。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
source_grant_id	opaque_id NOT NULL
granted	safe_uint NOT NULL
available	safe_uint NOT NULL
reserved	safe_uint NOT NULL DEFAULT 0
consumed	safe_uint NOT NULL DEFAULT 0
expired	safe_uint NOT NULL DEFAULT 0
revoked	safe_uint NOT NULL DEFAULT 0
expires_at	timestamptz NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (granted 0); CHECK (available::numeric + reserved::numeric + consumed::numeric + expired::numeric + revoked::numeric = granted::numeric); CHECK (expires_at created_at); CHECK (revision 0); UNIQUE (workspace_id, source_grant_id); FOREIGN KEY (workspace_id,

source_grant_id) REFERENCES entitlement_grants(workspace_id, id)。

ai_billing_accounts

稳定采购账户；连接/密钥版本变更不重置预算。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
account_billing_id	opaque_id NOT NULL
provider_name	text NOT NULL
daily_budget_fen	safe_uint NOT NULL
state	text NOT NULL DEFAULT 'active'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (state IN ('active','stopped','retired')); CHECK (revision 0); UNIQUE (environment, account_billing_id); UNIQUE (id, environment)。

ai_provider_connections

供应商逻辑连接；禁用只停新任务。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
billing_account_id	opaque_id NOT NULL
name	text NOT NULL
state	text NOT NULL DEFAULT 'unconfigured'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (state IN ('disabled','unconfigured','ready','degraded')); CHECK (revision 0); UNIQUE (environment, name); UNIQUE (id, billing_account_id, environment); FOREIGN KEY (billing_account_id, environment) REFERENCES ai_billing_accounts(id, environment); UNIQUE (id, billing_account_id)。

ai_provider_versions

不可变适配器/秘密/隐私能力版本。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
connection_id	opaque_id NOT NULL
billing_account_id	opaque_id NOT NULL

字段	SQL 类型与约束
environment	text NOT NULL
version	integer NOT NULL
adapter_id	text NOT NULL
adapter_version	text NOT NULL
base_url	text NOT NULL
secret_reference_id	opaque_id NOT NULL
auth_mode	text NOT NULL
connect_timeout_ms	integer NOT NULL DEFAULT 5000
request_timeout_ms	integer NOT NULL DEFAULT 60000
max_concurrent	integer NOT NULL
requests_per_minute	integer NOT NULL
daily_budget_fen	safe_uint NOT NULL
allowed_data_use	jsonb NOT NULL
capabilities	jsonb NOT NULL

关系与唯一性：CHECK (version 0); CHECK (base_url ~ '^https://'); CHECK (connect_timeout_ms BETWEEN 100 AND 60000); CHECK (request_timeout_ms BETWEEN 100 AND 600000); CHECK (max_concurrent BETWEEN 1 AND 10000); CHECK (requests_per_minute BETWEEN 1 AND 1000000); CHECK (auth_mode IN ('bearer','signature','adapter_defined')); CHECK (jsonb_typeof(allowed_data_use) = 'object'); CHECK (jsonb_typeof(capabilities) = 'object'); UNIQUE (connection_id, version); UNIQUE (id, connection_id, billing_account_id); FOREIGN KEY (connection_id, billing_account_id, environment) REFERENCES ai_provider_connections(id, billing_account_id, environment); FOREIGN KEY (secret_reference_id, environment) REFERENCES secret_references(id, environment)。

ai_models

模型目录状态依实际契约测试启用。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
name	text NOT NULL
state	text NOT NULL DEFAULT 'unverified'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (state IN ('unverified','ready','disabled','retired')); CHECK (revision 0)。

ai_model_versions

模型能力/限制/授权与契约测试冻结。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL

字段	SQL 类型与约束
created_at	timestamptz NOT NULL DEFAULT now()
model_id	opaque_id NOT NULL
version	integer NOT NULL
provider_version_id	opaque_id NOT NULL
provider_model_id	text NOT NULL
provider_model_version	text NOT NULL
license_version_id	opaque_id NOT NULL
input_schema_version	integer NOT NULL
output_schema_version	integer NOT NULL
capabilities	jsonb NOT NULL
parameter_schema	jsonb NOT NULL
limits_json	jsonb NOT NULL
moderation_policy_ref	text NOT NULL
label_policy_ref	text NOT NULL
test_report_id	opaque_id NOT NULL

关系与唯一性：CHECK (version 0 AND input_schema_version 0 AND output_schema_version 0); CHECK (jsonb_typeof(capabilities) = ' object'); CHECK (jsonb_typeof(parameter_schema) = 'object'); CHECK (jsonb_typeof(limits_json) = 'object'); UNIQUE (model_id, version); UNIQUE (id, provider_version_id); FOREIGN KEY (model_id) REFERENCES ai_models(id); FOREIGN KEY (provider_version_id) REFERENCES ai_provider_versions(id); FOREIGN KEY (license_version_id) REFERENCES license_versions(id); FOREIGN KEY (test_report_id) REFERENCES config_tests(id)。

ai_routes

排版建议/生图/参考图/审核能力独立路由。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
capability	text NOT NULL
name	text NOT NULL
state	text NOT NULL DEFAULT 'draft'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (capability IN ('textSuggestion','textToImage','imageToImage','moderation')); CHECK (state IN ('draft','active','retired')); CHECK (revision 0)。

ai_route_versions

备用仅在确定未接受且满足报价合同前允许。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
route_id	opaque_id NOT NULL
version	integer NOT NULL
primary_model_version_id	opaque_id NOT NULL
fallback_model_version_id	opaque_id
fallback_policy	text NOT NULL DEFAULT 'none'

关系与唯一性：CHECK (version 0); CHECK (fallback_policy IN ('none','definitely_not_accepted')); CHECK ((fallback_policy = 'none') = (fallback_model_version_id IS NULL)); CHECK (fallback_model_version_id IS NULL OR fallback_model_version_id <primary_model_version_id); UNIQUE (route_id, version); FOREIGN KEY (route_id) REFERENCES ai_routes(id); FOREIGN KEY (primary_model_version_id) REFERENCES ai_model_versions(id); FOREIGN KEY (fallback_model_version_id) REFERENCES ai_model_versions(id)。

ai_presets

风格目录；许可不明不得上架。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
name	text NOT NULL
category	text NOT NULL
state	text NOT NULL DEFAULT 'draft'
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (state IN ('draft','review','published','retired')); CHECK (revision 0)。

cost_rule_versions

用户点数与供应商采购价分开版本定价。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
rule_key	text NOT NULL
version	integer NOT NULL
credit_cost	safe_uint NOT NULL
estimated_cost_fen	safe_uint NOT NULL
max_cost_fen	safe_uint NOT NULL
parameters	jsonb NOT NULL

关系与唯一性：CHECK (version 0); CHECK (estimated_cost_fen <= max_cost_fen); CHECK (jsonb_typeof(parameters) = 'object'); UNIQUE (rule_key, version)。

ai_preset_versions

不可变预设模板，仅白名单插值。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
preset_id	opaque_id NOT NULL
version	integer NOT NULL
license_version_id	opaque_id NOT NULL
cost_rule_version_id	opaque_id NOT NULL
parameter_template	jsonb NOT NULL
editable_parameters	text[] NOT NULL
moderation_policy_ref	text NOT NULL
label_policy_ref	text NOT NULL
example_license_version_id	opaque_id

关系与唯一性：CHECK (version 0); CHECK (jsonb_typeof(parameter_template) = 'object'); UNIQUE (preset_id, version); FOREIGN KEY (preset_id) REFERENCES ai_presets(id); FOREIGN KEY (license_version_id) REFERENCES license_versions(id); FOREIGN KEY (cost_rule_version_id) REFERENCES cost_rule_versions(id); FOREIGN KEY (example_license_version_id) REFERENCES license_versions(id)。

ai_preset_models

预设所支持模型的显式白名单。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
preset_version_id	opaque_id NOT NULL
model_version_id	opaque_id NOT NULL

关系与唯一性：UNIQUE (preset_version_id, model_version_id); FOREIGN KEY (preset_version_id) REFERENCES ai_preset_versions(id); FOREIGN KEY (model_version_id) REFERENCES ai_model_versions(id)。

budget_days

Asia/Shanghai 采购账户日预算，旧日未知预占不清零。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
billing_account_id	opaque_id NOT NULL

字段	SQL 类型与约束
budget_date	date NOT NULL
limit_fen	safe_uint NOT NULL
reserved_fen	safe_uint NOT NULL DEFAULT 0
spent_fen	safe_uint NOT NULL DEFAULT 0
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (revision 0); CHECK (reserved_fen::numeric + spent_fen::numeric <= 9007199254740991); UNIQUE (billing_account_id, budget_date); UNIQUE (id, billing_account_id); FOREIGN KEY (billing_account_id) REFERENCES ai_billing_accounts(id)。

provider_budget_days

同账户各连接叠加独立上限，密钥轮换不归零。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
connection_id	opaque_id NOT NULL
billing_account_id	opaque_id NOT NULL
budget_day_id	opaque_id NOT NULL
limit_fen	safe_uint NOT NULL
reserved_fen	safe_uint NOT NULL DEFAULT 0
spent_fen	safe_uint NOT NULL DEFAULT 0
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：CHECK (revision 0); CHECK (reserved_fen::numeric + spent_fen::numeric <= 9007199254740991); UNIQUE (connection_id, budget_day_id); UNIQUE (id, budget_day_id, billing_account_id, connection_id); FOREIGN KEY (budget_day_id, billing_account_id) REFERENCES budget_days(id, billing_account_id); FOREIGN KEY (connection_id, billing_account_id) REFERENCES ai_provider_connections(id, billing_account_id)。

generation_consents

明确选中参考图用途与授权，不默认全相册上传。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
user_id	opaque_id NOT NULL
reference_asset_id	opaque_id
policy_version	text NOT NULL
purpose	text NOT NULL

字段	SQL 类型与约束
consented_at	timestamptz NOT NULL
reference_authorization_ref	text
input_hash	sha256_hex NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (char_length(purpose) BETWEEN 1 AND 240); CHECK (reference_asset_id IS NULL OR reference_authorization_ref IS NOT NULL); FOREIGN KEY (user_id) REFERENCES users(id); FOREIGN KEY (workspace_id, reference_asset_id) REFERENCES assets(workspace_id, id)。

generation_quotes

5分钟报价不预占；接受任务唯一消费此 quote。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
config_release_id	opaque_id NOT NULL
route_version_id	opaque_id NOT NULL
model_version_id	opaque_id NOT NULL
provider_version_id	opaque_id NOT NULL
preset_version_id	opaque_id
cost_rule_version_id	opaque_id NOT NULL
consent_id	opaque_id NOT NULL
reference_asset_id	opaque_id
input_hash	sha256_hex NOT NULL
credit_cost	safe_uint NOT NULL
estimated_cost_fen	safe_uint NOT NULL
max_cost_fen	safe_uint NOT NULL
image_count	integer NOT NULL DEFAULT 1
expires_at	timestamptz NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (estimated_cost_fen <= max_cost_fen); CHECK (image_count = 1); CHECK (expires_at created_at AND expires_at <= created_at + interval '5 minutes'); FOREIGN KEY (config_release_id) REFERENCES config_releases(id); FOREIGN KEY (route_version_id) REFERENCES ai_route_versions(id); FOREIGN KEY (model_version_id, provider_version_id) REFERENCES ai_model_versions(id, provider_version_id); FOREIGN KEY (preset_version_id) REFERENCES ai_preset_versions(id); FOREIGN KEY (cost_rule_version_id) REFERENCES cost_rule_versions(id); FOREIGN KEY (workspace_id, consent_id) REFERENCES generation_consents(workspace_id, id); FOREIGN KEY (workspace_id, reference_asset_id) REFERENCES assets(workspace_id, id)。

export_preflights

导出阻断/警告与参数 hash 绑定，令牌仅摘要。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
project_id	opaque_id NOT NULL
revision_id	opaque_id NOT NULL
normalized_spec_hash	sha256_hex NOT NULL
schema_version	integer NOT NULL
format	text NOT NULL
dpi	integer NOT NULL
max_face_pixels	safe_uint NOT NULL
warnings	jsonb NOT NULL
blockers	jsonb NOT NULL
token_hash	sha256_hex NOT NULL
expires_at	timestamptz NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (schema_version = 1); CHECK (format IN ('jpeg','png','pdf')); CHECK (dpi IN (72,150,300)); CHECK (max_face_pixels BETWEEN 1 AND 50000000); CHECK (jsonb_typeof(warnings) = 'array'); CHECK (jsonb_typeof(blockers) = 'array'); CHECK (expires_at created_at); UNIQUE (token_hash); UNIQUE (workspace_id, project_id, revision_id, id); FOREIGN KEY (workspace_id, project_id, revision_id) REFERENCES project_revisions(workspace_id, project_id, id)。

job_snapshots

任务冻结所有参数、授权与版本；不是客户端任意 JSON。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
kind	text NOT NULL
project_id	opaque_id
revision_id	opaque_id
quote_id	opaque_id
preflight_id	opaque_id
config_release_id	opaque_id NOT NULL
schema_version	integer NOT NULL

字段	SQL 类型与约束
normalized_spec_hash	sha256_hex NOT NULL
manifest_hash	sha256_hex NOT NULL
manifest	jsonb NOT NULL
entitlement_snapshot	jsonb NOT NULL
warnings_acknowledged	jsonb NOT NULL
renderer_version	text
renderer_image_digest	text

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (kind IN ('layout','generation','export','asset_analysis','storage_migration','deletion','privacy_export','config_test','reconciliation','backup_restore_drill')); CHECK (schema_version 0); CHECK ((project_id IS NULL) = (revision_id IS NULL)); CHECK (kind <'generation' OR quote_id IS NOT NULL); CHECK (kind <'export' OR (preflight_id IS NOT NULL AND revision_id IS NOT NULL AND renderer_version IS NOT NULL AND renderer_image_digest ~ '^sha256:[0-9a-f]{64}$')); CHECK (jsonb_typeof(manifest) = 'object'); CHECK (jsonb_typeof(entitlement_snapshot) = 'object'); CHECK (jsonb_typeof(warnings_acknowledged) = 'array'); FOREIGN KEY (workspace_id, project_id, revision_id) REFERENCES project_revisions(workspace_id, project_id, id); FOREIGN KEY (workspace_id, quote_id) REFERENCES generation_quotes(workspace_id, id); FOREIGN KEY (workspace_id, project_id, revision_id, preflight_id) REFERENCES export_preflights(workspace_id, project_id, revision_id, id); FOREIGN KEY (config_release_id) REFERENCES config_releases(id)。

jobs

PostgreSQL 任务事实；条件 fence/lease 写入，终态只一次。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
kind	text NOT NULL
snapshot_id	opaque_id NOT NULL
quote_id	opaque_id
parent_job_id	opaque_id
created_by	opaque_id NOT NULL
state	text NOT NULL DEFAULT 'queued'
progress	integer NOT NULL DEFAULT 0
stage	text NOT NULL DEFAULT 'queued'
fence	bigint NOT NULL DEFAULT 0
revision	bigint NOT NULL DEFAULT 1
lease_owner	opaque_id
lease_expires_at	timestamptz
attempt_count	integer NOT NULL DEFAULT 0

字段	SQL 类型与约束
max_attempts	integer NOT NULL DEFAULT 3
provider_submission_state	text NOT NULL DEFAULT 'not_submitted'
provider_request_id	text
next_attempt_at	timestamptz
cancel_requested_at	timestamptz
terminal_at	timestamptz
error_code	text

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (kind IN ('layout','generation','export','asset_analysis','storage_migration','deletion','privacy_export','config_test','reconciliation','backup_restore_drill')); CHECK (state IN ('queued','running','packaging','retry_wait','cancel_requested','succeeded','failed','cancelled')); CHECK (progress BETWEEN 0 AND 100); CHECK (fence >= 0 AND revision 0); CHECK (attempt_count BETWEEN 0 AND max_attempts AND max_attempts BETWEEN 1 AND 10); CHECK (provider_submission_state IN ('not_submitted','submitting','accepted','unknown','terminal')); CHECK ((lease_owner IS NULL) = (lease_expires_at IS NULL)); CHECK ((state IN ('succeeded','failed','cancelled')) = (terminal_at IS NOT NULL)); CHECK (kind <'generation' OR quote_id IS NOT NULL); UNIQUE (workspace_id, snapshot_id); UNIQUE (workspace_id, quote_id); UNIQUE (workspace_id, id, fence); FOREIGN KEY (workspace_id, snapshot_id) REFERENCES job_snapshots(workspace_id, id); FOREIGN KEY (workspace_id, quote_id) REFERENCES generation_quotes(workspace_id, id); FOREIGN KEY (workspace_id, parent_job_id) REFERENCES jobs(workspace_id, id); FOREIGN KEY (created_by) REFERENCES users(id); UNIQUE (workspace_id, id, quote_id)。

job_attempts

逐次执行与未知上游查询；不把网络超时当失败。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
job_id	opaque_id NOT NULL
attempt	integer NOT NULL
fence	bigint NOT NULL
worker_id	opaque_id NOT NULL
state	text NOT NULL
started_at	timestamptz NOT NULL
ended_at	timestamptz
provider_request_id	text
provider_job_id	text
submission_state	text NOT NULL
error_code	text
next_query_at	timestamptz

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (attempt BETWEEN 1 AND 10); CHECK (fence 0); CHECK (state IN ('running','unknown','succeeded','failed','cancelled','superseded')); CHECK (submission_state IN ('not_submitted','submitting','accepted','unknown','terminal')); CHECK (ended_at IS NULL OR ended_at >= started_at); UNIQUE (workspace_id, job_id, attempt); UNIQUE (workspace_id, job_id, fence); FOREIGN KEY (workspace_id, job_id) REFERENCES jobs(workspace_id, id)。

resource_pins

pin 精确旧 locator 与字体/模板版本；解 pin 后才能清理。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
job_id	opaque_id NOT NULL
storage_workspace_id	opaque_id NOT NULL
storage_object_id	opaque_id NOT NULL
asset_id	opaque_id
font_version_id	opaque_id
template_version_id	opaque_id
filter_version_id	opaque_id
released_at	timestamptz

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); UNIQUE (workspace_id, job_id, storage_object_id); FOREIGN KEY (workspace_id, job_id) REFERENCES jobs(workspace_id, id); FOREIGN KEY (storage_workspace_id, storage_object_id) REFERENCES storage_objects(workspace_id, id); FOREIGN KEY (workspace_id, asset_id) REFERENCES assets(workspace_id, id); FOREIGN KEY (font_version_id) REFERENCES font_versions(id); FOREIGN KEY (template_version_id) REFERENCES template_versions(id); FOREIGN KEY (filter_version_id) REFERENCES filter_versions(id)。

job_outputs

通过 fence 竞态后发布私有产物；迟到临时对象不绑定。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
job_id	opaque_id NOT NULL
artifact_index	integer NOT NULL
storage_object_id	opaque_id NOT NULL
committed_fence	bigint NOT NULL
format	text NOT NULL
sha256	sha256_hex NOT NULL

字段	SQL 类型与约束
byte_size	safe_uint NOT NULL
width	integer
height	integer
expires_at	timestamptz NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (artifact_index BETWEEN 0 AND 999); CHECK (committed_fence 0); CHECK (format IN ('jpeg','png','pdf','json','zip')); CHECK (byte_size 0); CHECK ((width IS NULL) = (height IS NULL)); CHECK (width IS NULL OR (width 0 AND height 0 AND width::bigint * height::bigint <= 50000000)); CHECK (expires_at created_at); UNIQUE (workspace_id, job_id, artifact_index); FOREIGN KEY (workspace_id, job_id) REFERENCES jobs(workspace_id, id); FOREIGN KEY (workspace_id, storage_object_id) REFERENCES storage_objects(workspace_id, id)。

layout_candidates

规则排版候选不修改项目；应用候选需 revision CAS。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
job_id	opaque_id NOT NULL
project_id	opaque_id NOT NULL
base_revision_id	opaque_id NOT NULL
candidate_index	integer NOT NULL
seed	bigint NOT NULL
score	numeric(8,6) NOT NULL
layout_hash	sha256_hex NOT NULL
layout_json	jsonb NOT NULL
expires_at	timestamptz NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (candidate_index BETWEEN 0 AND 19); CHECK (score BETWEEN 0 AND 1); CHECK (jsonb_typeof(layout_json) = 'object'); CHECK (expires_at created_at); UNIQUE (workspace_id, job_id, candidate_index); FOREIGN KEY (workspace_id, job_id) REFERENCES jobs(workspace_id, id); FOREIGN KEY (workspace_id, project_id, base_revision_id) REFERENCES project_revisions(workspace_id, project_id, id)。

credit_reservations

任务一次预占与一次终局；未知状态保留直到终局上限。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL

字段	SQL 类型与约束
job_id	opaque_id NOT NULL
quote_id	opaque_id NOT NULL
amount	safe_uint NOT NULL
state	text NOT NULL DEFAULT 'held'
expires_at	timestamptz NOT NULL
settled_at	timestamptz
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (state IN ('held','settled','released','expired')); CHECK (expires_at created_at); CHECK ((state = 'held') = (settled_at IS NULL)); CHECK (revision 0); UNIQUE (workspace_id, job_id); UNIQUE (workspace_id, quote_id); UNIQUE (workspace_id, id, job_id, quote_id); FOREIGN KEY (workspace_id, quote_id) REFERENCES generation_quotes(workspace_id, id); FOREIGN KEY (workspace_id, job_id, quote_id) REFERENCES jobs(workspace_id, id, quote_id)。

credit_reservation_allocations

按最早到期分配；各批次金额总和由事务服务保证。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
reservation_id	opaque_id NOT NULL
lot_id	opaque_id NOT NULL
amount	safe_uint NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (amount 0); UNIQUE (workspace_id, reservation_id, lot_id); FOREIGN KEY (workspace_id, reservation_id) REFERENCES credit_reservations(workspace_id, id); FOREIGN KEY (workspace_id, lot_id) REFERENCES credit_lots(workspace_id, id)。

credit_ledger

追加非负金额流水；全局 operation_id 唯一，每批次一行。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
operation_id	opaque_id NOT NULL
operation_line	integer NOT NULL DEFAULT 1
operation_group_id	opaque_id NOT NULL
lot_id	opaque_id NOT NULL

字段	SQL 类型与约束
reservation_id	opaque_id
entry_type	text NOT NULL
amount	safe_uint NOT NULL
from_bucket	text NOT NULL
to_bucket	text NOT NULL
balance_after	safe_uint NOT NULL
reason_code	text NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (operation_line 0); CHECK (amount 0); CHECK (entry_type IN ('grant','reserve','settle','release','expire','revoke','adjustment')); CHECK (from_bucket IN ('external','available','reserved','consumed','expired','revoked')); CHECK (to_bucket IN ('external','available','reserved','consumed','expired','revoked')); CHECK (from_bucket <to_bucket); CHECK ((entry_type = 'grant' AND from_bucket = 'external' AND to_bucket = 'available') OR (entry_type = 'reserve' AND from_bucket = 'available' AND to_bucket = ' reserved') OR (entry_type = 'settle' AND from_bucket = 'reserved' AND to_bucket = 'consumed') OR (entry_type = 'release' AND from_bucket = 'reserved' AND to_bucket IN ('available','expired','revoked')) OR (entry_type = 'expire' AND from_bucket = 'available' AND to_bucket = 'expired') OR (entry_type = 'revoke' AND from_bucket IN ('available','reserved') AND to_bucket = 'revoked') OR entry_type = 'adjustment'); UNIQUE (operation_id); UNIQUE (workspace_id, operation_group_id, operation_line); FOREIGN KEY (workspace_id, lot_id) REFERENCES credit_lots(workspace_id, id); FOREIGN KEY (workspace_id, reservation_id) REFERENCES credit_reservations(workspace_id, id)。

budget_reservations

采购预占绑定接受当天与稳定账户；连接与账户双锁。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
job_id	opaque_id NOT NULL
quote_id	opaque_id NOT NULL
billing_account_id	opaque_id NOT NULL
connection_id	opaque_id NOT NULL
provider_version_id	opaque_id NOT NULL
budget_day_id	opaque_id NOT NULL
provider_budget_day_id	opaque_id NOT NULL
reserved_fen	safe_uint NOT NULL
actual_fen	safe_uint NOT NULL DEFAULT 0
state	text NOT NULL DEFAULT 'held'
expires_at	timestamptz NOT NULL
settled_at	timestamptz

字段	SQL 类型与约束
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (state IN ('held','settled','released','expired')); CHECK (expires_at created_at); CHECK ((state = 'held') = (settled_at IS NULL)); CHECK (revision 0); UNIQUE (workspace_id, job_id); FOREIGN KEY (workspace_id, job_id, quote_id) REFERENCES jobs(workspace_id, id, quote_id); FOREIGN KEY (provider_version_id, connection_id, billing_account_id) REFERENCES ai_provider_versions(id, connection_id, billing_account_id); FOREIGN KEY (budget_day_id, billing_account_id) REFERENCES budget_days(id, billing_account_id); FOREIGN KEY (provider_budget_day_id, budget_day_id, billing_account_id, connection_id) REFERENCES provider_budget_days(id, budget_day_id, billing_account_id, connection_id)。

provider_cost_ledger

采购成本追加账本；查询费用与迟到费用单独对账。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
billing_account_id	opaque_id NOT NULL
budget_day_id	opaque_id NOT NULL
reservation_id	opaque_id
operation_id	opaque_id NOT NULL
entry_type	text NOT NULL
amount_fen	safe_uint NOT NULL
provider_charge_ref	text
reason_code	text NOT NULL

关系与唯一性：CHECK (entry_type IN ('reserve','release','settle','late_charge','reconcile')); UNIQUE (operation_id); FOREIGN KEY (budget_day_id, billing_account_id) REFERENCES budget_days(id, billing_account_id); FOREIGN KEY (reservation_id) REFERENCES budget_reservations(id)。

generation_provenance

生成来源不可变；输入授权、模型、标识、审核可追溯。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
job_id	opaque_id NOT NULL
asset_id	opaque_id NOT NULL
quote_id	opaque_id NOT NULL
consent_id	opaque_id NOT NULL
provider_version_id	opaque_id NOT NULL

字段	SQL 类型与约束
model_version_id	opaque_id NOT NULL
preset_version_id	opaque_id
input_hash	sha256_hex NOT NULL
provider_request_id	text NOT NULL
provider_job_id	text
seed	bigint
license_version_id	opaque_id NOT NULL
moderation_policy_ref	text NOT NULL
label_policy_ref	text NOT NULL
moderation_result	text NOT NULL
label_applied	boolean NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (moderation_result IN ('approved','rejected','quarantined')); UNIQUE (workspace_id, asset_id); FOREIGN KEY (workspace_id, job_id, quote_id) REFERENCES jobs(workspace_id, id, quote_id); FOREIGN KEY (workspace_id, asset_id) REFERENCES assets(workspace_id, id); FOREIGN KEY (workspace_id, consent_id) REFERENCES generation_consents(workspace_id, id); FOREIGN KEY (model_version_id, provider_version_id) REFERENCES ai_model_versions(id, provider_version_id); FOREIGN KEY (preset_version_id) REFERENCES ai_preset_versions(id); FOREIGN KEY (license_version_id) REFERENCES license_versions(id)。

outbox

主事务同时写出的可重投调度事实。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id
event_key	opaque_id NOT NULL
aggregate_kind	text NOT NULL
aggregate_id	opaque_id NOT NULL
event_type	text NOT NULL
payload	jsonb NOT NULL
available_at	timestamptz NOT NULL DEFAULT now()
claimed_by	opaque_id
claim_expires_at	timestamptz
delivery_attempts	integer NOT NULL DEFAULT 0
sent_at	timestamptz
last_error_code	text

关系与唯一性：CHECK (delivery_attempts BETWEEN 0 AND 1000000); CHECK (jsonb_typeof(payload) = 'object'); CHECK ((claimed_by IS NULL) = (claim_expires_at IS NULL)); UNIQUE (event_key); FOREIGN KEY (workspace_id) REFERENCES workspaces(id)。

idempotency_keys

工作区/身份作用域幂等；同 key 异 payload 409。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id
principal_scope	text NOT NULL
idempotency_key	text NOT NULL
request_hash	sha256_hex NOT NULL
operation	text NOT NULL
state	text NOT NULL DEFAULT 'processing'
response_status	integer
response_json	jsonb
resource_kind	text
resource_id	opaque_id
expires_at	timestamptz NOT NULL

关系与唯一性：CHECK (char_length(principal_scope) BETWEEN 1 AND 200); CHECK (char_length(idempotency_key) BETWEEN 1 AND 128); CHECK (state IN ('processing','completed')); CHECK (response_status IS NULL OR response_status BETWEEN 100 AND 599); CHECK (expires_at >= created_at + interval '7 days'); CHECK (state <'completed' OR response_status IS NOT NULL); FOREIGN KEY (workspace_id) REFERENCES workspaces(id)。

share_links

私有预览分享；撤销后不再发签名 URL。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
project_id	opaque_id NOT NULL
revision_id	opaque_id NOT NULL
created_by	opaque_id NOT NULL
token_hash	sha256_hex NOT NULL
password_hash	text
scope	text NOT NULL DEFAULT 'preview'

字段	SQL 类型与约束
state	text NOT NULL DEFAULT 'active'
expires_at	timestamptz NOT NULL
revoked_at	timestamptz
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (scope IN ('preview','preview_download')); CHECK (state IN ('active','revoked','expired')); CHECK (expires_at created_at); CHECK (revision 0); UNIQUE (token_hash); FOREIGN KEY (workspace_id, project_id, revision_id) REFERENCES project_revisions(workspace_id, project_id, id); FOREIGN KEY (created_by) REFERENCES users(id)。

share_access_events

分享限速与审计；不存原始 IP。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
share_link_id	opaque_id NOT NULL
outcome	text NOT NULL
request_id	opaque_id NOT NULL
visitor_digest	sha256_hex NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (outcome IN ('granted','denied','expired','revoked')); FOREIGN KEY (workspace_id, share_link_id) REFERENCES share_links(workspace_id, id)。

notification_preferences

用户按渠道/类别通知偏好。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
user_id	opaque_id NOT NULL
channel	text NOT NULL
category	text NOT NULL
enabled	boolean NOT NULL
revision	bigint NOT NULL DEFAULT 1

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (channel IN ('in_app','wechat','email_reserved')); CHECK (category IN ('job','billing','security','workspace','system')); CHECK (revision 0); UNIQUE (workspace_id, user_id, channel, category); FOREIGN KEY (user_id) REFERENCES users(id)。

notifications

站内通知稳定事实；不携带原图/秘密。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
recipient_user_id	opaque_id NOT NULL
event_key	opaque_id NOT NULL
category	text NOT NULL
template_key	text NOT NULL
template_version	integer NOT NULL
redacted_payload	jsonb NOT NULL
read_at	timestamptz
expires_at	timestamptz NOT NULL

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (category IN ('job','billing','security','workspace','system')); CHECK (template_version 0); CHECK (jsonb_typeof(redacted_payload) = 'object'); CHECK (expires_at created_at); UNIQUE (workspace_id, recipient_user_id, event_key); FOREIGN KEY (recipient_user_id) REFERENCES users(id)。

notification_deliveries

渠道发送重试去重；App 仅保留 future 通道。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id NOT NULL
notification_id	opaque_id NOT NULL
channel	text NOT NULL
attempt	integer NOT NULL DEFAULT 1
state	text NOT NULL DEFAULT 'pending'
provider_message_id	text
next_attempt_at	timestamptz
error_code	text

关系与唯一性：UNIQUE (workspace_id, id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); CHECK (channel IN ('in_app','wechat','email_reserved','app_reserved')); CHECK (attempt BETWEEN 1 AND 10); CHECK (state IN ('pending','sending','sent','failed','suppressed')); UNIQUE (workspace_id, notification_id, channel, attempt); FOREIGN KEY (workspace_id, notification_id) REFERENCES notifications(workspace_id, id)。

deletion_tombstones

备份恢复必须重放的删除标记，无原始 PII。 行不可变/追加。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
workspace_id	opaque_id
user_id	opaque_id
target_kind	text NOT NULL
target_id	opaque_id NOT NULL
object_locator_hash	sha256_hex
operation_id	opaque_id NOT NULL
deleted_at	timestamptz NOT NULL
retain_until	timestamptz NOT NULL
reason_code	text NOT NULL

关系与唯一性：CHECK (retain_until >= deleted_at); CHECK (num_nonnulls(workspace_id, user_id) >= 1); UNIQUE (operation_id, target_kind, target_id); FOREIGN KEY (workspace_id) REFERENCES workspaces(id); FOREIGN KEY (user_id) REFERENCES users(id)。

backup_runs

数据库和对象备份分开证据；不含凭据或声称已演练。 状态或元信息可变；CAS/权限按服务合同。

字段	SQL 类型与约束
id	opaque_id NOT NULL
created_at	timestamptz NOT NULL DEFAULT now()
environment	text NOT NULL
kind	text NOT NULL
state	text NOT NULL
started_at	timestamptz NOT NULL
completed_at	timestamptz
backup_reference	text NOT NULL
manifest_hash	sha256_hex
restored_tombstone_watermark	timestamptz
error_code	text

关系与唯一性：CHECK (environment IN ('dev','staging','prod')); CHECK (kind IN ('database','objects','restore_drill')); CHECK (state IN ('running','succeeded','failed')); CHECK (completed_at IS NULL OR completed_at >= started_at)。

C1 网页编辑与确定性导出 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (☐) syntax for tracking.

Goal: 交付可在本地启动的网页相册编辑器，从照片导入、图层编辑和保存恢复，到冻结文档、预检及实际 PNG／JPG／普通 PDF 导出。

Architecture: 自有文档模型是事实源，Fabric 只是画布适配器。网页与可信 headless 入口共用资源、滤镜、文字布局和渲染包；先实现本地仓库端口，后续 C3 替换为云端仓库，C4 接入持久化任务与权益。

Tech Stack: Node 24 LTS、npm workspaces、TypeScript、React、Vite、Fabric、IndexedDB、Vitest、Playwright 固定 Chromium、pdf-lib、fflate；精确候选版本见附录 A，实际安装后生成 lockfile。

Spec: v0.3 基线、PRD、架构、33 配方、测试运维。其他单元归属见交付路线。

状态：待评审的实施计划；下列测试代码与命令是实施时要执行的步骤，本次没有执行产品测试，也没有产品功能通过记录。用户确认全程 AI 开发；不按人工人数估算交期。

Global Constraints

- 文档坐标 px96，schemaVersion=1，colorSpace=srgb；单页为 pageSize，跨页宽度为单页两倍；封面正背面各为单页，另计。

- 首期草案允许 1 – 50 个内页跨页，即 2 – 100 个内页；封面正面、背面另存，不计内页数量。

- 单项目 500 张照片、单图 25MB、100 个图层／跨页；首期单图不超过 80MP。它们是待目标设备验证的容量上限，不是已通过的性能指标。

- 首期支持 JPEG、PNG；HEIC、RAW、PSD 暂不接受；不开放任意 SVG 上传。

- Group 禁止循环、重复 ID、NaN／Infinity 和超深嵌套，深度最多 5；图层数组从底到顶。

- 图片先校正 EXIF 方向，crop 为校正后源图的归一化矩形，范围必须位于 [0,1]，非空；原片不覆盖。

- 原图单列；F01 – F33 稳定 ID；强度 0 – 100%；配方顺序固定；强度在最终效果与原图的线性光域混合；seed 固定。

- 文档、素材、字体、模板和滤镜版本可追溯；缺原片不能回退缩略图；字体缺失／无授权、未知 schema、文字超框或不支持字符阻断成品导出。

- 裁剪后有效 PPI 100 – 199 警告；小于 100 强警告，普通数字导出可明确确认继续；不能靠 DPI 标签提升细节。

- 导出像素＝逻辑 px × DPI ÷96；600×300mm／300DPI 裁切区 7087×3543px；各边 3mm 出血后 7157×3614px。

- 普通 PDF 为 sRGB 栅格页，不承诺 PDF/X、CMYK、可搜索矢量文字、拼版或专业印厂交付。

- 网页完整编辑、小程序快速成册、App 预留；C1 仅交付网页与共享核心，其他首期模块仍在 C2 – C5。

Review Focus

- RF1：未知图层／扩展字段经过读取、局部编辑、保存后仍完整；未知层不能被静默省略导出。测试归属任务 1、8。

- RF2：IndexedDB 配额不足、并发标签页和保存冲突不毁坏上一版本，不把失败显示为“已保存”。测试归属任务 3。

- RF3：EXIF 6／8、透明 PNG、裁剪越界和极端长宽比不会误转方向、改变原片或虚报 PPI。测试归属任务 2、8。

- RF4：中文标点、组合字符、缺字与字体加载竞态不会以系统字体替代后悄悄导出。测试归属任务 2、5、8。

- RF5：React StrictMode、切换跨页、迟到图片加载和取消不会修改新页面或泄漏画布／对象 URL。测试归属任务 5、6、9。

交付边界与统一接口

C1 可在单用户本地浏览器完成完整闭环，界面显示“仅此浏览器保存”，提供可恢复的项目包。本地导出进度可取消，关闭页面不会假装仍有后台任务。正式登录、云存储、可靠导出队列和会员验证由 C3／C4 实现。C1 不调用生图、不展示虚构支付成功、不提供可绕过的假会员。

以下为本计划新增、待评审的工程决定，实施后不得暗改：

- MAX_IMPORT_BYTES=25_000_000，MAX_SOURCE_PIXELS=80_000_000，MAX_SURFACE_PIXELS=50_000_000；每跨页 100 层包含递归子层。DPI 仅接受 72、150、300。

- 本地浏览器预计工作集上限 768MiB，headless 2GiB；保守估算 12×最大源图像素＋12×输出面像素＋缓存实际字节＋128MiB。超预算明确拒绝并建议拆分或降规格，不暗降清晰度；不是实测内存保证。

- 同时解码／渲染 1 个输出面；图片缓存至多 128MiB；滤镜 tile 为 512×512，不创建整张 80MP 的浮点副本。每张 bitmap／canvas 在使用结束后释放。

- 所有层以左上为原点，x／y／width／height 为局部坐标，rotation 单位度；group 子层使用父坐标、组只允许统一缩放，避免首版引入不可表达的剪切。图片框缩放通过 crop／fitMode 表达；不支持的画布变换不能写回模型。

- fitMode='cover'|'contain'，默认 cover；cover 裁剪比例匹配图框，contain 明确留白；旋转保持源图方向归一化后的坐标。

- 保存后的 revision 为正整数；本地新项目 revision=1。保存用 expectedRevision 原子比较，成功加一；草稿的操作不擅自修改 revision。

- 自动保存空闲 2 秒、持续操作最多 10 秒触发；撤销至少 100 次操作，历史命令不含图片字节；本地保留最近 20 个恢复 revision，正在导出快照引用的资源不清理。

- C1 原创种子模板 5 个：婚礼、旅行、亲子、写真、国风各 1 个。它们是可编辑占位模板，不继承第三方图片或 Skill 输出。

- 关闭导出时释放所有临时资源；PNG 保留透明，JPEG 使用用户显式选择的底色（默认白色），JPG 质量 0.95。多面图片打 ZIP，PDF 每面一页。

共享类型由指定任务定义，任务外不要另起同义模型：

类型／端口	定义位置与最低字段			
DocumentModel, Layer, DocumentIssue, FilterRef, TemplateRef	任务 1；v0.1 示例为基础，known layers=image/text/shape/group；未知同版本层为 opaque 对象，保留全部 JSON；Issue 含 code、path、severity；FilterRef 含 presetId／recipeVersion／strength／seed，TemplateRef 含 id／version／sha256			
AssetRecord, FontRecord, ResourceManifest	任务 2；资源 id/version、文件 SHA-256、尺寸、来源／授权、状态；字体 face 可含多个本地 WOFF2 子集及各自 hash／字符覆盖			
ResourcePort	任务 2；read(id: string, variant: 'original'	'normalized'	'preview'	'font'): Promise<Uint8Array>；找不到则抛带 code 的错误，不能替代 variant
ResourceWriter, AssetBytes	任务 2 定义、任务 3 实现；putAsset(record:AssetRecord, data:AssetBytes):Promise<void>、putFont(record:FontRecord,files:ReadonlyMap<string, Uint8Array>):Promise<void>；AssetBytes 为 original／normalized／preview 三份字节；同 id 不同 hash 禁止覆盖			

类型／端口	定义位置与最低字段			
RevisionStore	任务 3；load(projectId: string, revision?: number): Promise<SavedRevision>；save(doc: DocumentModel, expectedRevision: number, resources: ResourceManifest): Promise<SaveResult>			
SavedRevision, SaveResult	任务 3；前者含 doc、docHash、resources、savedAt；后者为 {ok:true,value:SavedRevision} 或 {ok:false,code:'REVISION_CONFLICT'	'STORAGE_FULL'	'STORAGE_FAILED',recoveryDraft:DocumentModel}	
PixelTile, FilterRecipe	任务 4；tile 含 RGBA8 bytes、width、height、offsetX／Y、fullWidth／Height；FilterRef 从 document 导入			
RenderInput, RenderSurface, TextMetricsReport	任务 5；RenderInput 含 doc、surface:RenderSurface、manifest、port、signal、pixelScale；RenderSurface 含稳定 id、sourceSpreadId 或 cover、clip:{x,y,width,height}、bleedPx96、background、pixelWidth／Height；文字报告含 layerId、lines:string[]、overflow、unsupportedCodePoints			
ExportOptions, SurfaceSpec, ExportSnapshot	任务 8；options 含 format、dpi、view、pageIds、includeCover、bleedMm、jpegMatte；SurfaceSpec 扩展 RenderSurface 的物理 trim／bleed 元数据；snapshot 含已保存 revision、docHash、resources、options、optionsHash、createdAt			
PreflightReport	任务 8；含 snapshotHash、blockers、warnings、surfaces；每个 issue 有稳定 id，确认针对这些 id 和 snapshotHash，不接受全局“忽略全部”			
Artifact, ArtifactManifest	任务 9；文件名、MIME、bytes／SHA-256、surfaceId、像素／物理尺寸、DPI、revision、snapshotHash、renderBuild；不含原片 GPS			

文件结构

所有路径相对仓库根；每项任务列出具体文件，src/index.ts 仅导出公开接口。

workspace 包名固定为 @album/document、@album/assets、@album/storage、@album/render、@album/export、@album/web、@album/render-cli，依赖方向由上表决定，不跨包读取私有文件。

packages/document/src/	类型、校验、哈希、命令、模板
packages/assets/src/	导入、方向、字体、资源合同
packages/storage/src/	IndexedDB、revision、项目包
packages/render/src/	CPU 滤镜、字体布局、Fabric 映射
packages/export/src/	预检、快照、几何、编码与元数据
apps/web/src/	编辑器 UI、本地项目、导出面板
apps/render-cli/src/	可信本地快照渲染入口
tests/fixtures/	自制／明确授权的照片、字体、坏文档
tests/browser/	真实 Chromium 行为与导出检查
docs/implementation/c1/	实际环境、验收结果与导出样片

Task 1: 可校验、无损往返的文档模型

Files: Create package.json, package-lock.json, .nvmrc, tsconfig.json, vitest.config.ts, packages/document/package.json, packages/document/src/{model,validate,canonical,index}.ts, packages/document/src/document.test.ts, tests/fixtures/documents/minimal.json。

Interfaces: Consumes v0.1 JSON 字段及本计划 Global Constraints。Produces parseDocument(value: unknown): DocumentModel、validateDocument(doc: DocumentModel): DocumentIssue[]、canonicalDocument(doc: DocumentModel): string、hashDocument(doc: DocumentModel): Promise<string>。解析错误抛 DocumentValidationError，有 issues；未知同版本层不是解析错误，但返回 UNSUPPORTED_LAYER 导出阻断项。

☐ Step 1: 写失败用例 document.test.ts。 复制 minimal fixture 后增加未知层和 advancedBlend:{mode:'multiply'}；断言 parse→canonical→parse 保留字段；相同对象不同键顺序 hash 相等，数组顺序不同 hash 不同。再断言 NaN、Infinity、负宽高、重复层 ID、6 层嵌套、循环对象、schemaVersion=99 分别产生明确错误；封面与内页 ID 全局唯一。（RF1）

```
expect(parseDocument(JSON.parse(canonicalDocument(doc)))).toEqual(doc);
expect(validateDocument(doc).map(i => i.code)).toContain('UNSUPPORTED_LAYER');
expect(() => parseDocument({...doc, schemaVersion: 99})).toThrow(DocumentValidationError);
```

☐ Step 2: 建立仅供测试运行的 workspace 配置，执行 npm run test:unit -- packages/document/src/document.test.ts。预期 FAIL：缺少 parseDocument 实现；若只有依赖／配置错误，先修复测试环境，不把它算作有效失败。配置与最小可运行模型同项提交。

☐ Step 3: 实现上述四个签名。 已知字段严格校验、对象扩展字段透传；仅接收 JSON 值，禁止 prototype 污染键。canonical 键按 Unicode 码点排序、数组保持顺序、几何值保存为最多 6 位小数，hash 用 SHA-256。跨页及源图上限校验分开，未知层不能变成空层。生成 DocumentModel 的判别联合与 opaque 类型，不使用 Fabric JSON 作源数据。

☐ Step 4: 执行 npm run test:unit -- packages/document/src/document.test.ts 和 npm run typecheck，预期退出码均 0；记录 Node、npm、lockfile SHA 和兼容性结果。精确版本安装不兼容时停止扩展功能，依据最小复现选择受支持版本并记决策，不强制忽略 peer 依赖。

☐ Step 5: 提交 git add package.json package-lock.json .nvmrc tsconfig.json vitest.config.ts packages/document tests/fixtures/documents；git commit -m "feat(document): add lossless validated album model"。

Task 2: 不可变照片、EXIF 与本地字体

Files: Create packages/assets/package.json, packages/assets/src/{types,import-image,font-catalog,resource-port,index}.ts, packages/assets/src/assets.test.ts, scripts/{build-font-catalog,build-image-fixtures}.ts, tests/fixtures/images/{orientation-6,orientation-8,transparent,landscape,portrait}.json, tests/fixtures/images/README.md, apps/web/public/fonts/{manifest.json,OFL.txt}, tests/browser/assets.spec.ts, tests/browser/harness/{index.html,main.ts,host.ts}, playwright.config.ts。图像二进制由 fixture 脚本生成并加入同目录；字体子集文件由已锁依赖复制，禁止运行时下载字体。harness 只在测试启动 Vite 提供模块／fixture，后续浏览器核心测试复用；任务6开始另启动真实 web 做 UI 验证。

Interfaces: Consumes DocumentIssue。Produces资源共享类型和 ResourceWriter 接口、importImage(file: File, signal: AbortSignal): Promise<{record:AssetRecord,original:Uint8Array,normalized:Uint8Array,preview:Uint8Array}>、loadFont(record: FontRecord, port: ResourcePort, signal: AbortSignal): Promise<LoadedFont>、unsupportedGlyphs(text: string, font: LoadedFont): number[]。LoadedFont 含唯一 family、faces、codePoints、dispose()；字体集合内每个子集具有独立资源 id，可经 read(id,'font') 读取。

☐ Step 1: 写 assets.test.ts 与 assets.spec.ts。 构造带四角色块的 EXIF 6／8 JPEG，断言归一化宽高交换、角块位置正确、original SHA 不变，重复加载不再次旋转；透明 PNG alpha 保留；25_000_001 字节、80_000_001 像素、伪 JPEG、坏 PNG、超长宽比但像素合法输入均有明确接受／拒绝结果。字体读失败／hash 不符／授权禁止使用必须失败；我们A´ 中缺字返回具体码点，不以系统 fallback 当成功。（RF3、RF4）

```
expect([result.record.width,  result.record.height]).toEqual([200,  300]);  //  源为300×200，EXIF6
expect(result.original).toEqual(originalBytes);
expect(unsupportedGlyphs('我们  ',  font)).toContain(0x1f642);
```

☐ Step 2: 执行 npm run test:unit -- packages/assets/src/assets.test.ts 与 npm run test:browser -- tests/browser/assets.spec.ts，预期 FAIL：导入／字体入口未实现。

☐ Step 3: 实现三个函数与 ResourcePort。 检查 magic bytes 与图片头后解码；浏览器解码器统一应用 EXIF 一次，不再叠加手动旋转，归一化文件清除 EXIF orientation／GPS。保留原片字节独立存储；preview 长边 1600。seed 字体 Noto Sans SC／Noto Serif SC，各400／700真实字重，精确版本及 OFL 留档；fontkit 解析实际子集 cmap，按所需字符加载对应本地 faces，等待 FontFace.load 和 document.fonts.ready；family 用版本 hash 命名，不允许合成粗体冒充已有版本。abort 时释放 bitmap 和 ObjectURL。

☐ Step 4: 重跑上述两命令，预期全部 PASS。 记录 EXIF fixture 生成逻辑和颜色容差（JPEG 每通道 8）；核对 manifest 每份字体文件实际 SHA、来源与可用授权，禁止只读取 CSS unicode-range 当真实缺字证据。

☐ Step 5: 提交本项明确路径及新增 fixture、根依赖／lockfile；git commit -m "feat(assets): normalize imports and pin licensed fonts"。

Task 3: 原子保存、命令历史和恢复项目包

Files: Create packages/storage/package.json, packages/storage/src/{indexeddb-store,project-bundle,index}.ts, packages/document/src/commands.ts, apps/web/src/state/{history,autosave}.ts, tests/browser/storage.spec.ts。

Interfaces: Consumes文档哈希和 ResourceManifest。Produces IndexedDbRevisionStore implements RevisionStore, ResourcePort, ResourceWriter（构造参数 dbName:string）、applyCommand(doc: DocumentModel, command: DocumentCommand): DocumentModel、History(initial:DocumentModel)（execute(command:DocumentCommand)、undo()、redo() 均返回 DocumentModel）、Autosave(store:RevisionStore,getDraft:()=>DocumentModel,getResources:()=>ResourceManifest)（markDirty():void、flush():Promise<SaveResult>、dispose():void，status='dirty'|'saving'|'saved'|'conflict'|'error'）、exportProject(projectId:string,store:IndexedDbRevisionStore):Promise<Uint8Array>、importProject(bytes:Uint8Array,store:IndexedDbRevisionStore):Promise<SavedRevision>。DocumentCommand 为 insert/delete/update/reorder layer、add/delete/reorder spread、apply-template 的判别联合；update 只改已知允许字段，不替换整层对象。

☐ Step 1: 写 storage.spec.ts。 A／B 两标签页从 revision=1 保存，A 得 2，B 必须 REVISION_CONFLICT；注入事务 QuotaExceededError 后 load 仍返回 2、autosave.status='error'、恢复草稿存在，任务6再验证对应 UI 文案；用虚拟时钟验证 2 秒／10 秒；连续 100 次操作可全部撤销并重做；复制跨页后修改副本不改变原页；未知字段跨导出项目包／导入仍在。（RF1、RF2）

```
expect((await  store.save(draftA,  1,  resources)).ok).toBe(true);
expect(await  store.save(draftB,  1,  resources)).toMatchObject({ok:false,code:'REVISION_CONFLICT'});
expect((await  store.load(projectId)).doc.revision).toBe(2);
```

☐ Step 2: 执行 npm run test:browser -- tests/browser/storage.spec.ts，预期 FAIL：仓库和 history 未实现。

☐ Step 3: 实现上述端口与类。 revision 元信息、文档、资源引用与 current 指针在同一 IndexedDB 事务更新；资源 blobs 按 hash 不可变，未挂接资源可稍后回收。提供 create(doc:DocumentModel,resources:ResourceManifest):Promise<SavedRevision建立 revision=1，不复用 save 的冲突语义；本地类再提供 pinResources(ownerId:string,manifest:ResourceManifest):Promise<void>／releaseResources(ownerId:string):Promise<void>，清理器跳过活动导出引用。项目包 ZIP 含版本 manifest、文档、原图／归一图／字体字节、SHA 与许可证；导入逐项验证，拒绝路径穿越、重复路径、超资源上限、解压后总量超过 1GiB 的包，不从包执行脚本或解析任意 URL。导入用新 projectId，校验完成后文档与资源在同一事务落库，保留 sourceProjectId／sourceRevision 追溯；配额不足保留原库，不先清空。冲突返回可另存草稿，不自动合并；清理 20 份以外历史时保留当前和正在导出引用。

☐ Step 4: 重跑命令并增加坏 ZIP／hash 错误导入断言，预期 PASS。 npm run typecheck 退出 0。首次浏览器未授权持久存储／不支持 IndexedDB 时显示具体错误和项目包恢复建议，不声称自动云备份。

☐ Step 5: 提交本项源文件、测试及依赖变更；git commit -m "feat(storage): add atomic revisions and recoverable local projects"。

Task 4: 33 个确定性 CPU 滤镜

Files: Create packages/render/package.json, packages/render/src/filters/{types,recipes,kernel,index}.ts, packages/render/src/filters/kernel.test.ts, scripts/build-filter-manifest.ts, packages/render/manifests/filters-v1.json；Read docs/ai-album-v0.1/filters.csv，不修改历史配方状态。

Interfaces: Consumes CSV 的 F01 – F33。Produces getRecipe(ref: FilterRef): FilterRecipe、filterTile(tile: PixelTile, recipe: FilterRecipe, strength: number, seed: number): Uint8ClampedArray；manifest 带 CSV SHA、kernelVersion='cpu-srgb-v1' 与操作版本。原图使用 null FilterRef，不能占用 F01。

☐ Step 1: 写 kernel.test.ts。 断言 33 个唯一连续 ID、顺序／默认强度 .65／seed1729 与 CSV 一致；strength=0 完全返回原字节；单 exposure=1、strength=1 时 [128,64,32,128]→[255,128,64,128]；相同 seed 同字节，改 seed 的 grain 结果变化；四块 tile 拼接等于整图；12 种操作均有至少一组数值 fixture，alpha 逐像素不变。

```
expect([...filterTile(tile, exposureOneRecipe, 1, 1729)]).toEqual([255,128,64,128]);
expect(filterTile(tile, recipe, 0, 1729)).toEqual(tile.bytes);
```

☐ Step 2: 执行 npm run test:unit -- packages/render/src/filters/kernel.test.ts，预期 FAIL：kernel／manifest 未实现。

☐ Step 3: 按附录 B 实现签名。 manifest 由 CSV 构建且可重复生成；CPU 为唯一标准路径，不引入 CSS／GPU 第二套效果。滤镜在归一化源图坐标上计算，再裁剪和缩放；preview 也由这一处理结果缩小。强度／参数越界拒绝，未知 recipeVersion 阻断，不自动使用最新版本。

☐ Step 4: 重跑测试并执行 npm run build:filters -- --check，预期 PASS／退出 0；生成自制色卡的 33 效果联系表供后续视觉评审。数值通过只证明确定性，不把 CSV 的 visual_review_required 自动改成视觉已验收。

☐ Step 5: 提交本项路径、联系表及依赖变更；git commit -m "feat(render): add versioned deterministic filter recipes"。

Task 5: 共用的画布、裁剪与中文文字布局

Files: Create packages/render/src/{scene-adapter,text-layout,image-pipeline,index}.ts, packages/render/src/text-layout.test.ts, tests/browser/render.spec.ts, tests/fixtures/documents/render-contract.json。

Interfaces: Consumes模型、资源端口、filterTile。Produces共享 RenderInput／RenderSurface／TextMetricsReport 类型、layoutText(layer: TextLayer, font: LoadedFont, measure: TextMeasure): TextMetricsReport、createScene(canvas: HTMLCanvasElement, input: RenderInput): Promise<SceneHandle>；handle 为 {update(input:RenderInput):Promise<void>, render():Promise<void>, dispose():void}。TextMeasure 为 (text:string,font:LoadedFont,fontSize:number,letterSpacing:number)=>number；渲染与预检共用同份文字报告。RenderSurface 在此定义，不依赖后续 export 包，编辑器预览构造同型 surface。

☐ Step 1: 写 text-layout.test.ts 和 render.spec.ts。 连续中文、中文标点、中英混排、换行符、A＋组合重音和 emoji grapheme 不被拆坏；行首不出现，。！？、）》】，行尾不出现（《【，放不下时报告 overflow；100px 高框、fontSize40／lineHeight1.4 的三行判为溢出。验证 cover／contain、组旋转、透明度和层序；并发 update 后迟到解码不能覆盖新页面；dispose 两次安全，ObjectURL／bitmap 创建释放计数平衡。（RF4、RF5）

```
expect(layoutText(threeLineLayer, font, measure).overflow).toBe(true);
expect(layoutText(chineseLayer, font, measure).lines.every(s => !/^[，。！？、）》】]/u.test(s))).toBe(true);
```

☐ Step 2: 执行 npm run test:unit -- packages/render/src/text-layout.test.ts 和 npm run test:browser -- tests/browser/render.spec.ts，预期 FAIL：共享 scene／布局未实现。

☐ Step 3: 实现三个入口。 Intl.Segmenter 按 grapheme 分段、显式标点禁则、Canvas measureText 使用已加载精确字体；lineHeight 为 fontSize 倍数、letterSpacing 为 px96。将已确定的文字行分别映射 FabricText，不让交互 Textbox 和导出再各自换行。框为布局边界，报告实际字形墨迹及逻辑行框越界，文字编辑用 HTML textarea overlay 后提交模型。图片先共享 CPU pipeline 再交由 Fabric；禁止 SVG 外链／可执行对象。update 采用代次令牌＋AbortSignal，旧异步结果必须丢弃并释放。

☐ Step 4: 重跑两命令，预期 PASS。 render-contract 使用相同输入重复渲染像素 hash 相等；不同输出倍率的几何比例相符；缺字体时 FAIL FONT_UNAVAILABLE，不截图系统字体作为预期。

☐ Step 5: 提交本项路径与依赖；git commit -m "feat(render): share crop and CJK scene rendering"。

Task 6: 可操作的网页图层与跨页编辑器

Files: Create apps/web/{package.json,index.html,vite.config.ts}, apps/web/src/{main,App}.tsx, apps/web/src/editor/{Editor,CanvasView,LayerPanel,PropertiesPanel,SpreadStrip,AssetPanel}.tsx, apps/web/src/styles.css, tests/browser/editor.spec.ts；Modify packages/document/src/commands.ts。

Interfaces: Consumes RevisionStore、History、Autosave、createScene。Produces Editor({initial,store,resources,port}:EditorProps):React.JSX.Element；editor 只经 DocumentCommand 更新模型，Fabric selection／拖动反馈转换成同一命令。应用层创建 store／端口，不在共享渲染中引用 React 状态。

☐ Step 1: 写 editor.spec.ts。 导入两张照片→拖入跨页→移动、裁剪、对齐、旋转→添加文字与矩形／圆／线→改变层序／显隐／锁定→组合与取消组合→撤销重做→保存刷新，断言模型与画面一致。两种字体及400／700粗细、字号／颜色／对齐／字距／行距可改；画框边界和中心吸附按6屏幕像素阈值工作；换图保持图框几何；删除使用中素材显示引用数量并阻止直接破坏引用。删除跨页前确认数量；新增／复制／排序后封面不计入内页。StrictMode 双挂载后仍只有一个活跃画布监听器，切页无迟到图片污染。（RF5）

```
await page.getByRole('button', {name:'保存'}).click();
await expect(page.getByRole('status')).toHaveText('已保存至此浏览器');
await page.reload();
await expect(page.getByRole('button', {name:'跨页 1'})).toBeVisible();
```

☐ Step 2: 执行 npm run test:browser -- tests/browser/editor.spec.ts，预期 FAIL：编辑页与操作不存在。

☐ Step 3: 实现上述组件与命令。 页面三区为素材／画布／属性，底部跨页；1280px 完整布局，1024px 折叠侧栏。支持多选、复制、删除、上下层、6 个对齐方向、均匀分布、锁定、显隐；group 统一缩放和旋转，取消组合后绝对几何保持。吸附阈值为6屏幕像素、随缩放换算为逻辑距离，Alt 暂停吸附。双击裁剪，cover／contain 可切；换图保留框并重新计算合法 crop；导入逐项进度、失败重试、同 hash 重复提示，默认不新增重复素材。透明度 [0,1]，rotation 可规范化；命令校验失败不污染历史。离页有未保存提示，Ctrl/Cmd+Z 在输入框内优先原生文字撤销；空项目和资源错误显示操作路径。

☐ Step 4: 重跑测试及 npm run build、npm run typecheck，预期全部退出 0；实测键盘操作、1024／1280／1920 截图。没有可用真实浏览器时记录阻塞，不用静态 HTML 校验冒充交互通过。

☐ Step 5: 提交本项路径与依赖；git commit -m "feat(web): add editable multi-spread album workspace"。

Task 7: 原创模板与滤镜面板

Files: Create packages/document/src/{template,template.test}.ts, packages/document/templates/{wedding,travel,family,portrait,ink}-v1.json, apps/web/src/editor/{TemplatePanel,FilterPanel}.tsx, tests/browser/presets.spec.ts。

Interfaces: Consumes commands、filter manifest。Produces instantiateTemplate(template: TemplateVersion, bindings: Record<string,string>, idFactory:()=>string): Layer[]、applyTemplateCommand(spreadId:string,layers:Layer[],templateRef:TemplateRef):DocumentCommand；TemplateVersion 含 id／version、尺寸、slots／editableFields、licenseRef、原创来源；替换不删除素材仓库原片。

☐ Step 1: 写 template.test.ts 和 presets.spec.ts。 五类模板可筛选，预览不改文档；确认应用仅修改目标跨页，一次撤销恢复原层及引用；slot 不足／绑定素材不存在则明确失败，不截断照片。选中两张图片应用 F07、调整强度后一次撤销恢复两张；文字层不受滤镜影响；原图入口与 33 预设分开。模板复制层 ID 不冲突。

```
expect(history.undo()).toEqual(beforeTemplate);
const ids = instantiateTemplate(template, bindings, idFactory).map(l => l.id);
expect(new Set(ids).size).toBe(ids.length);
```

☐ Step 2: 执行 npm run test:unit -- packages/document/src/template.test.ts 和 npm run test:browser -- tests/browser/presets.spec.ts，预期 FAIL。

☐ Step 3: 实现签名与面板。 seed 模板只含原创几何、独立标题／日期字段、带 hash 的模板版本；不含购买的假授权标识或受限示例。模板单次作用于当前跨页，整本智能分配留在 C2；待选照片保持可见清单，不声称已自动全部入册。配方参数直接引用 manifest，不复制成 UI 私有值。

☐ Step 4: 重跑两命令，预期 PASS；核对 5 模板来源记录及 33 项原图／强度界面，联系表保留“ 待视觉确认 ”标记。

☐ Step 5: 提交本项路径；git commit -m "feat(editor): add original templates and filter controls"。

Task 8: 冻结快照、导出几何与阻断预检

Files: Create packages/export/package.json, packages/export/src/{types,geometry,snapshot,preflight,index}.ts, packages/export/src/preflight.test.ts, tests/browser/preflight.spec.ts, apps/web/src/export/PreflightPanel.tsx。

Interfaces: Consumes SavedRevision、ResourcePort、layoutText。Produces freezeSnapshot(saved:SavedRevision,options:ExportOptions,createdAt:string):Promise<ExportSnapshot>、planSurfaces(snapshot:ExportSnapshot):SurfaceSpec[]、preflight(snapshot:ExportSnapshot,port:ResourcePort,signal:AbortSignal):Promise<PreflightReport>、authorizeExport(snapshot:ExportSnapshot,report:PreflightReport,acknowledgedWarningIds:string[]):void。冻结后深拷贝且只读；snapshotHash 绑定 docHash、资源 manifest、optionsHash 和创建时间。

☐ Step 1: 写 preflight.test.ts 与 preflight.spec.ts。 原图 read 失败但 preview 存在仍 ASSET_MISSING；未知层、字体／授权失败、缺字、文字溢出、超像素预算各阻断；EXIF 后裁剪有效 PPI 与预期一致，199／100 警告、99 强警告，未确认不得导出。600×300mm、300DPI、0／3mm、view=spread／includeCover=false 分别断言 7087×3543／7157×3614。另用带封面的 fixture 验证单页导出顺序 front→内页1…N→back，spread 模式 front→跨页1…N→back；cover 各只占单页。snapshot=12 后保存13，快照仍 12；修改 DPI 后旧报告无效。（RF1、RF3、RF4）

```
expect(planSurfaces(snapshotWith3mm)[0]).toMatchObject({pixelWidth:7157,pixelHeight:3614});
expect(report.blockers.map(i => i.code)).toContain('ASSET_MISSING');
expect(() => authorizeExport(snapshot, warningReport, [])).toThrow('WARNINGS_UNACKNOWLEDGED');
```

☐ Step 2: 执行 npm run test:unit -- packages/export/src/preflight.test.ts 和 npm run test:browser -- tests/browser/preflight.spec.ts，预期 FAIL。

☐ Step 3: 实现四个入口。 从已保存记录构造快照，源图／字体读取核对 SHA；图片以 min(可用裁剪源宽÷绘制宽英寸,可用裁剪源高÷绘制高英寸) 算 PPI，contain 使用实际绘制区域，group 缩放纳入。旋转图片按未旋转框对应源尺寸算，不用旋转后外接框夸大 PPI。PNG／PDF 像素预算独立检查；几何只最终输出 round，出血扩展输出面，超出裁切区的图像原样延展、没有内容处用背景，不拉伸整幅页面冒充出血。资源预计超内存亦阻断。pageIds 取稳定逻辑页 ID（如 spr_01:left／right），spread 视图使用 spr_01；先按文档顺序生成输出面，再筛选所选 ID，拒绝与 view 不匹配或不存在的 ID。

☐ Step 4: 重跑两命令，预期 PASS。 预检面板逐项显示阻断／警告、实际像素和快照 revision；安全区 5mm／中缝关键文字警告。C1 输入若带尚未实现的 assignmentManifest 校验版本则阻断 UNSUPPORTED_LAYOUT_MANIFEST，C2 再接入重新派生照片清单的验证器；不把未知 manifest 当通过。含未支持的 AI 来源／标识要求时明确 UNSUPPORTED_PROVENANCE，C5 接入前不能忽略。为这两个拒绝行为各增加一条断言。

☐ Step 5: 提交本项路径与依赖；git commit -m "feat(export): freeze revisions and enforce preflight"。

Task 9: 实际 PNG／JPEG／PDF 编码与下载

Files: Create packages/export/src/{encode-png,encode-jpeg,encode-pdf,artifact-manifest,run-export}.ts, packages/export/src/metadata.test.ts, apps/web/src/export/ExportPanel.tsx, tests/browser/export.spec.ts。

Interfaces: Consumes ExportSnapshot、PreflightReport、authorizeExport、createScene。Produces runExport(snapshot:ExportSnapshot,report:PreflightReport,ack:string[],port:ResourcePort,signal:AbortSignal,onProgress:(done:number,total:number)=>void):AsyncGenerator<Artifact>、encodePng(canvas:HTMLCanvasElement,dpi:number):Promise<Uint8Array>、encodeJpeg(canvas:HTMLCanvasElement,dpi:number,matte:string):Promise<Uint8Array>、encodePdf(pages:EncodedPage[],snapshot:ExportSnapshot):Promise<Uint8Array>；EncodedPage 含 surface、mime、bytes。PDF 首期页面逐面渲染，累计编码数据达到 256MiB 时显式拒绝并建议拆分，不无界保留原始像素。

☐ Step 1: 写 metadata.test.ts 和 export.spec.ts。 校验 PNG pHYs、JPEG JFIF densityUnits=1／Xdensity=Ydensity=300；分辨率取 pHYs 反算容差 .02DPI，不能只读文件名。PDF pageCount、MediaBox、TrimBox／BleedBox 与物理尺寸一致；多面 ZIP 名称顺序 0001-front、0002-spread-01 等由 SurfaceSpec 固定；透明 PNG alpha 保留，JPEG 透明区用指定白色。编辑到13后导出快照12，标题和像素仍12；中途 cancel 不提供完整成功下载，资源计数归零。（RF5）

```
expect(decodedPng.width).toBe(7157);
expect(jfif).toMatchObject({densityUnits:1,xDensity:300,yDensity:300});
expect(pdf.getPageCount()).toBe(6);  //  4个内页＋封面正背面
```

☐ Step 2: 执行 npm run test:unit -- packages/export/src/metadata.test.ts 和 npm run test:browser -- tests/browser/export.spec.ts，预期 FAIL。

☐ Step 3: 实现编码与生成器。 PNG 插入／替换 pHYs 并重算 CRC；JPEG 写入／规范 JFIF 密度，不保留源 EXIF；PDF 使用 pdf-lib，MediaBox 包含出血、TrimBox 为裁切范围、BleedBox=MediaBox，宽高用 mm×72/25.4，按用户视图逐面一页。先 authorizeExport，再依序渲染／编码；每面完成释放 canvas。网页在调用前以 snapshotHash 执行 store.pinResources，finally 执行 releaseResources；CLI 的只读包天然冻结。ArtifactManifest 含实际文件 SHA、revision、资源 hash、renderBuild，不写用户位置／GPS。ZIP 使用 fflate 流式条目，累积下载超过 256MiB 提示拆分；浏览器不伪装持久后台队列。

☐ Step 4: 重跑两命令，预期 PASS；从导出字节重新解码检验尺寸、色块／文字截图和元数据，不只检验 mock。下载 PNG／JPG／PDF 与 JSON 清单作为程序样片候选；JPEG 视觉比较容许有损误差，不要求与 PNG 字节一致。

☐ Step 5: 提交本项路径与依赖；git commit -m "feat(export): encode real image and PDF artifacts"。

Task 10: 固定 Chromium 渲染入口、端到端验收与交付

Files: Create apps/render-cli/{package.json,src/main.ts,src/host.ts}, apps/web/src/render-entry.ts, tests/browser/c1-acceptance.spec.ts, tests/fixtures/documents/acceptance.json, scripts/verify-artifacts.ts, .github/workflows/c1.yml, docs/implementation/c1/{README.md,environment.json,acceptance-results.json}, docs/export-samples/program-c1/README.md；Modify 根 README.md、package.json。

Interfaces: Consumes共享 render／export，不重写 scene。Produces CLI npm run render:snapshot -- --bundle <project.zip--snapshot <snapshot.json--output <directory>（仅可信开发／worker 调用），成功退出0，参数／预检错误2，资源／执行失败3。浏览器 render-entry 暴露 renderSnapshot(request:TrustedRenderRequest):Promise<ArtifactManifest>，请求仅携带快照和 manifest 内资源 ID，不接受任意 URL 或代码。

☐ Step 1: 写 c1-acceptance.spec.ts。 从空浏览器开始导入授权照片→应用模板→编辑中文→设置滤镜→保存→项目包恢复→预检→导出三格式；同一 fixture 在网页和固定 Chromium render-entry 中输出 PNG 解码像素 hash 相同。HEADLESS 请求外部 HTTP、file://、../ 路径、未知资源 ID 被拒；50MP 边界与低内存预算有可重复拒绝结果，不能悄悄缩小。重复运行同快照像素一致，metadata／文件排序一致；ZIP 时间等容器字节不作为跨系统逐字节承诺。

```
expect(webPixelsHash).toBe(headlessPixelsHash);
expect(artifactManifest.revision).toBe(12);
expect(artifactManifest.snapshotHash).toBe(snapshotHash);
```

☐ Step 2: 执行 npm run test:browser -- tests/browser/c1-acceptance.spec.ts，预期 FAIL：CLI／端到端入口未接通。

☐ Step 3: 实现入口与验证脚本。 host 仅绑定 127.0.0.1 随机端口、每次随机 token，资源查表无任意路径；Playwright 拦截出站请求，仅允许本次受信 host／blob，固定 Chromium 版本、locale=zh-CN、timezone=UTC、DPR=1，超时120秒，关闭后清理临时目录。它不作为公网 API 发布。CI npm ci→npx playwright install --with-deps chromium→typecheck／unit／build／browser；记录 lockfile SHA、浏览器实际版本、OS、字体／kernel／build hash。无凭据也能运行 C1，不连微信／支付／AI。

☐ Step 4: 执行最终验证 npm ci、npm run typecheck、npm run test:unit、npm run build、npm run test:browser、npm run verify:artifacts -- docs/export-samples/program-c1，预期全部退出0。样片至少含600×300mm、300DPI、3mm出血的 PNG／JPG、4个内页＋独立封面正背面的单页PDF、快照／预检／资源manifest／实际环境与 SHA。记录耗时和峰值内存观察值，不把它升级为上线 SLA。按选择的执行方式安排独立复核，修复阻断问题后再认定 C1 完成。

☐ Step 5: 提交本项路径、实际程序样片与验收记录；git commit -m "feat(c1): verify reproducible editor-to-export delivery"。根 README 改成实际可运行入口，保留历史 AI 样片标注；推送实施分支、创建草稿实现 PR，不能直接宣称生产上线。

附录 A：依赖、命令与验证状态

官方 npm registry 元数据于 2026-10-21 查阅；以下为计划候选精确版本，尚未安装验证兼容性，不能把这张表当已生成 lockfile 或安全审计。

依赖	候选版本	许可元数据／用途
node／npm	24.19.0／11.9.0	当前执行环境；Node 24 LTS 系列
react／react-dom	19.3.1／19.3.1	MIT；编辑界面
vite／@vitejs/plugin-react	8.3.5／6.1.2	MIT；构建
typescript	7.0.2	Apache-2.0；类型检查
fabric	7.4.0	MIT；仅作为 adapter
vitest	5.0.5	MIT；纯逻辑测试
@playwright/test	1.64.1	Apache-2.0；绑定其浏览器修订版本
zod	4.6.7	MIT；严格已知字段＋透传扩展字段
idb	8.0.4	ISC；本地事务封装
pdf-lib／fflate	1.17.1／0.8.3	MIT；普通 PDF／ZIP

依赖	候选版本	许可元数据／用途
@fontsource/noto-sans-sc	5.3.1	OFL-1.1；本地可追溯字体子集
@fontsource/noto-serif-sc	5.3.1	OFL-1.1；第二字体、400／700真实字重
fontkit	2.0.4	MIT；读取真实字体 cmap／度量；浏览器兼容在任务2验证
tsx	4.23.15	MIT；可信 CLI 与构建脚本
@types/node／@types/react／@types/react-dom	24.19.2／19.3.0／19.3.0	MIT；类型版本对齐

新增依赖先说明用途，记录实际 LICENSE 与传递依赖，不无目的引入 Zustand／完整后端框架／图片生成 SDK。当前 C1 不需要数据库、Redis 或付款账号。

根脚本定义：test:unit=vitest run；typecheck=tsc --noEmit（项目引用可分包但总命令不变）；test:browser=playwright test；dev=vite --config apps/web/vite.config.ts；build 构建共享包／web／CLI；build:filters=tsx scripts/build-filter-manifest.ts；render:snapshot=tsx apps/render-cli/src/main.ts；verify:artifacts=tsx scripts/verify-artifacts.ts。脚本随其拥有任务落地，计划阶段不创建空实现来伪造通过。

附录 B：CPU 滤镜 v1 算法合同

这是对 v0.1 留待固定算法的本次工程建议，需与计划一起评审；所有操作按 CSV 顺序执行。RGB 归一化至 [0,1]，每步 clamp，内部 float64，最后量化 round(255×C)；alpha 原字节不变。亮度 Y=.2126R+.7152G+.0722B。

操作	定义
exposure:e	clamp(C×2^e)
contrast:k	clamp((C-.5)×(1+k)+.5)
saturation:s	clamp(Y+(C-Y)×(1+s))
temperature:t	clamp([R+.08t,G+.02t,B-.08t])
tint:q	clamp([R+.04q,G-.08q,B+.04q])
gamma:g	C^(1/g)，g>0
grayscale:a	C×(1-a)+Y×a
sepia:a	标准矩阵行 [.393,.769,.189]、[.349,.686,.168]、[.272,.534,.131] 后 clamp，再与输入按 a 混合
duotone:dark,light	dark×(1-Y)+light×Y，颜色由十六进制转归一化 sRGB
posterize:L	round(C×(L-1))/(L-1)，L 为2 – 256整数
vignette:v	源图像素中心归一化到 x,y∈[-1,1]；d=min(1,sqrt(x ² +y ²)/sqrt(2))；RGB乘 1-v×clamp((d-.2)/.8)^2
grain:a	源整数坐标 x,y 与 seed 混合：h=seed XOR imul(x+1,73856093) XOR imul(y+1,19349663)；MurmurHash3 fmix32 后转无符号 n=h/4294967296-.5；RGB各通道加 a×n

fmix32：h ^= h>>>16；h=imul(h,0x85ebca6b)；h ^= h>>>13；h=imul(h,0xc2b2ae35)；h ^= h>>>16。grain 在同像素使用同一个噪声值；tile 坐标必须加 offset，因此切块不改变颗粒。

最终强度 s 使用 IEC sRGB 转线性函数（阈值 .04045，低段除12.92，高段 ((C+.055)/1.055)^2.4），混合 linear(original)×(1-s)+linear(effect)×s，再转回 sRGB（阈值 .0031308，低段乘12.92，高段 1.055×L^(1/2.4)-.055）。s=0 必须字节直通。不得把 s 乘进每个配方参数来冒充此混合。

自审与验收归属

规格／风险	本计划对应项	说明
文档、方向、资源不可变与未知字段	1／2／3	RF1 – RF3 有指定失败用例
33滤镜、原图、强度与多选撤销	4／7	数值确定性与视觉确认分开
网页图层、文字、形状、模板、跨页、恢复	3／5／6／7	C1 完整基础编辑；C2 承担整本自动布局
V3-EXPORT-01／02／03	2／5／8／9	缺原图阻断、中文／字体／溢出、像素与元数据
V3-EXPORT-04	3／8／9／10	C1 验证冻结后继续编辑；持久队列／事务在 C4
RF4／RF5	2／5／6／8／9	字体竞态、StrictMode、取消和资源生命周期
小程序、登录、会员、队列、AI 与运营	路线 C3／C4／C5	明确后续验收，不作为 C1 已完成项

自审结论：10 项任务的接口名按统一类型表引用，失败／通过命令与文件归属齐全；没有把技术候选、计划用例或现有调研结果记作产品通过证据。下一步是评审本计划并确认 AI 执行方式，然后从任务1开始。

C2 整本排版、局部重排与照片完整性 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (☐) syntax for tracking.

Goal: 交付可解释且可撤销的整本分配、固定跨页构图、有效候选比较和真实文档完整性校验。

Architecture: 使用确定性的两层引擎：先解整本容量／顺序，再在固定页框匹配模板。候选及manifest与真实DocumentModel双向核对，应用是基于revision的一次文档命令；云端应用复用同一验证器。

Tech Stack: C1 TypeScript工作区、Vitest、React、Playwright；纯CPU规则引擎，无外部生图依赖。

Spec: v0.3 § §4 – 5／10、PRD §6、v0.4、C1接口。

Global Constraints

- 本文是待执行研发计划；测试片段和预期结果均不是当前已通过记录。只写本模块实现；真实微信、存储、支付、AI集成缺凭据时保持 unconfigured，本地模拟／契约测试可完整实施。

- 继承 共同合同：公开ID为1 – 128字符不透明字符串；SHA-256为64位小写十六进制；金额为人民币分整数、点数整数，JSON整数上限9007199254740991；UTC存储，展示和预算日为Asia/Shanghai。

- API根 /v1；成功 {data,requestId}，失败 {error:{code,message,retryable,details},requestId}；列表 {items,nextCursor,hasMore}。网页cookie＋CSRF，小程序短期bearer；所有资源逐次校验工作区权限。

- 可重放创建／财务／任务操作使用空间范围唯一的 Idempotency-Key，保留7天；相同key不同payload返回409。文档If-Match revision为字符串；草稿／目录更新有expectedRevision／If-Match，冲突409。

- 唯一产品文档模型由C1 @album/document提供，对应 document.schema.json；schemaVersion1、px96、sRGB；1 – 50内页跨页、500照片、100递归图层／跨页、组深度≤5。原片JPEG／PNG≤25_000_000字节且≤80MP；不接受任意外部URL素材引用。

- 平台统一存储／AI连接，不开放BYOS／BYOK；免费与付费能力都实现，真实价格／配额未经确认不得伪造。影楼首期单使用者，预留owner/admin/designer/viewer与seat_limit，不做邀请UI；App仅预留共享API。

- 测试命令沿用C1 npm run test:unit -- <file>／npm run test:browser -- <file>、npm run typecheck。集成测试使用临时PostgreSQL／Redis和本地受控模拟，禁止生产数据库；精确依赖版本在实施时锁入package-lock.json，Node 24 LTS。

- 默认 selectionPolicy=all、duplicatePolicy=forbid；只有allowSelection或明确列入repeat许可的照片可改变分配；每个输入照片必须在placements或unused中可核对。

- 推荐3个有效候选；不足3个展示真实数量，不用无效候选凑数。硬约束满足率100%；视觉候选可接受率≥80%是未来盲评目标，不能充当已测结果。

- 12张／4跨页／每页2 – 4张可行；50张／10跨页／每页最多2张无解，返回至少25跨页，不静默删片。

- 普通排版不调用生图、不消耗AI点数；主体分析仅服务当前项目，不建立跨项目人脸身份库。

Review Focus

- RF1：无EXIF、同时间、用户明确顺序和重复内容但不同assetId应保留稳定顺序，不默默折叠；任务1／2有断言。

- RF2：锁定框已超容量、锁定照片未在选片范围与局部重排的外部照片应产生可解释冲突，不偷改锁定；任务2／4有断言。

- RF3：极端横竖比例、0／NaN几何及图库行布局超高不能以评分掩盖固定页越界；任务3有断言。

- RF4：manifest宣称完整但真实文档漏片或混入装饰用途时不能应用／导出；任务4有断言。

- RF5：候选计算时用户编辑、网络重试与取消不能覆盖新revision或应用两次；任务5／6有断言。

文件与合同边界

下列产品路径均为未来实施时创建／修改，当前不生成产品代码。API只实现 OpenAPI 已定义操作与参数；DTO从合同生成到packages/contracts/src/api.ts（C3任务1），不自行维护第二套HTTP字段。五个schema使用固定文件名与各自$defs.root。数据库执行 001_initial.up.sql 与对应down迁移，并遵守 transaction-safety.md；发现合同和本计划有差异先修订合同并加反例，再实现。应用内部类型由本计划拥有它的任务定义并导出，测试数据在相应测试文件内构造，不依赖未定义的未来测试帮助函数。

类型与文件分工

packages/layout/src/types.ts只定义排版合同与引用C1层；不得另造文档schema。输入/输出按layout.schema.json校验。prepare.ts处理顺序与规范化；allocate.ts负责整本容量；compose.ts负责固定框几何；integrity.ts从真实层重建清单；apply.ts产生一个可撤销命令；网页只展示与发送选项，不自行推断完整性。纯包名 @album/layout。

Task 1: 排版输入、策略与可重复顺序

Files:

- Create: packages/layout/package.json

- Create: packages/layout/src/types.ts

- Create: packages/layout/src/prepare.ts

- Create: packages/layout/src/index.ts

- Create: packages/layout/src/prepare.test.ts

- Modify: package-lock.json

Interfaces:

- Consumes: C1 DocumentModel、Layer、TemplateRef；layout.schema.json。

- Produces: LayoutPhoto={assetId:string,width:number,height:number,captureTime?:string|null,groupId?:string,weight?:number}；LayoutConstraints={spreadCount:number,minPhotosPerSpread:number,maxPhotosPerSpread:number,selectionPolicy:"all"|"allowSelection",duplicatePolicy:"forbid"|"allowExplicit",orderPolicy:"user"|"captureTime",explicitRepeatAssetIds?:string[],requiredAssetIds?:string[],excludedAssetIds?:string[],lockedSpreadIds?:string[],lockedLayerIds?:string[],targetSpreadIds?:string[]}；LayoutInput={projectId:string,baseRevision:number,photos:LayoutPhoto[],constraints:LayoutConstraints,templateRef:TemplateRef,algorithmVersion:string,seed:number}；LayoutWeights={subject:number,timeline:number,whitespace:number,variety:number,color:number}；LayoutPolicy={algorithmVersion:string,candidateCount:1|2|3,weights:LayoutWeights}；PreparedLayout={input:LayoutInput,inputHash:string,orderedAssetIds:string[],policy:LayoutPolicy}；prepareLayout(input:LayoutInput,policy?:LayoutPolicy):Promise<PreparedLayout>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(prepared.orderedAssetIds).toEqual(['b','a','c']);  //  explicit  order,  no  EXIF  resort
expect(second.inputHash).toBe(first.inputHash);  //  same  canonical  payload
await  expect(prepareLayout({...input,photos:[photo,photo]})).rejects.toThrow('DUPLICATE_PHOTO_ID');
await  expect(prepareLayout({...input,  photos:[{...photo,width:NaN}]})).rejects.toThrow('INVALID_DIMENSIONS');
expect(prepared.orderedAssetIds).toHaveLength(3);  //  equal  photo  bytes,  distinct  IDs  remain
expect(noExifAndEqualTimestampIds).toEqual(['a','b','c']);  //  stable  input-index  tie  break
```

☐ Step 2: 运行失败用例。 npm run test:unit -- packages/layout/src/prepare.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在types.ts定义上述类型，prepare.ts实现prepareLayout(input:LayoutInput,policy?:LayoutPolicy):Promise<PreparedLayout>，index.ts导出。orderPolicy=user时photos数组顺序就是显式顺序；captureTime排序时有效时间在前、无EXIF放后并用输入索引打破平局；capture元数据的使用在UI说明。校验500输入、重复ID、required/excluded/explicitRepeat为输入ID子集、必选且禁用矛盾、min≤max、selection/duplicate枚举和seed为0 – 4294967295整数。policy省略时使用本地算法v1、candidateCount=3及任务3明确的五权重；云调用必须传冻结发布策略，校验policy.algorithmVersion=input.algorithmVersion且权重非负／和>0，PreparedLayout携带整份不可变policy。哈希绑定所有选项／锁定／模板／策略与算法版本，禁止把原始文件名或GPS写进报告。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- packages/layout/src/prepare.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add packages/layout/package.json packages/layout/src/types.ts packages/layout/src/prepare.ts packages/layout/src/index.ts packages/layout/src/prepare.test.ts package-lock.json；git commit -m "feat(layout): define deterministic input and assignment policy"。

Task 2: 整本容量求解与连续片段分配

Files:

- Create: packages/layout/src/allocate.ts

- Create: packages/layout/src/allocate.test.ts

Interfaces:

- Consumes: 任务1 PreparedLayout；C1 DocumentModel中现有锁定层。

- Produces: Allocation={spreads:ReadonlyArray<{spreadId:string,assetIds:string[]}>,unusedAssetIds:string[]}；LayoutConflict={code:string,assetIds:string[],spreadIds:string[],minimumRequiredSpreads:number|null,options:Array<"increaseSpreads"|"raiseCapacity"|"allowSelection"|"unlock">}；AllocationResult={ok:true,value:Allocation}|{ok:false,conflicts:LayoutConflict[]}；allocateBook(prepared:PreparedLayout,doc:DocumentModel):AllocationResult。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(allocateBook(twelveOnFour,doc).ok).toBe(true);
expect(fiftyOnTenResult).toMatchObject({ok:false,conflicts:[{code:'CAPACITY_EXCEEDED',minimumRequiredSpreads:25}]});
expect(allocated.spreads.flatMap(s => s.assetIds)).toEqual(explicitOrder);
expect(lockedOverflow.conflicts.map(c => c.code)).toContain('LOCKED_CAPACITY');
expect(lockedNotSelected.conflicts.map(c => c.code)).toContain('LOCKED_ASSET_NOT_SELECTED');
expect(allocated.unusedAssetIds).toEqual([]); // all/forbid
```

☐ Step 2: 运行失败用例。 npm run test:unit -- packages/layout/src/allocate.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在allocate.ts实现allocateBook(prepared:PreparedLayout,doc:DocumentModel):AllocationResult。锁定跨页／图层先扣容量，targetSpreadIds外跨页完全固定；锁定照片纳入全书计数。按显式顺序连续分段，用动态规划最小化章节切断和密度变化，整数可行性先于美学评分；打平按seed和稳定ID。allowSelection才可把可选片列unused；allowExplicit只重复explicitRepeatAssetIds中的照片。返回可操作冲突及最小页数，不运行无界搜索。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- packages/layout/src/allocate.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add packages/layout/src/allocate.ts packages/layout/src/allocate.test.ts；git commit -m "feat(layout): allocate complete photo sets within book constraints"。

Task 3: 固定跨页构图、主体安全与候选评分

Files:

- Create: packages/layout/src/compose.ts

- Create: packages/layout/src/score.ts

- Create: packages/layout/src/compose.test.ts

Interfaces:

- Consumes: 任务1 PreparedLayout、任务2 Allocation；C1 instantiateTemplate(template:TemplateVersion,bindings:Record<string,string>,idFactory:()=>string):Layer[]。

- Produces: SubjectHint={assetId:string,rect:{x:number,y:number,width:number,height:number},confidence:number}；ComposedSpread={spreadId:string,layers:Layer[],hardViolations:string[],warnings:string[],score:{subject:number,timeline:number,whitespace:number,variety:number,color:number,total:number}}；composeSpread(prepared:PreparedLayout,allocation:Allocation,doc:DocumentModel,template:TemplateVersion,hints:SubjectHint[]):ComposedSpread[]。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(composed.every(s => s.hardViolations.length===0)).toBe(true);
expect(allPhotoFrames.every(f => Number.isFinite(f.x) && f.width>0 && f.x+f.width<=spreadWidth)).toBe(true);
expect(panoramic.crop.x).toBeGreaterThanOrEqual(0);
expect(overflow.hardViolations).toContain('FRAME_OUT_OF_BOUNDS');
expect(protectedSubject.warnings).toContain('SUBJECT_CROP_RISK');
expect(invalid.score.total).toBeLessThan(valid.score.total); // invalid is still excluded, never eligible
```

☐ Step 2: 运行失败用例。 npm run test:unit -- packages/layout/src/compose.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在compose.ts实现Interfaces中完整composeSpread签名，在score.ts消费任务1 LayoutWeights并实现scoreSpread(spread:ComposedSpread,weights:LayoutWeights):ComposedSpread["score"]；无云配置的本地算法v1权重为0.30/0.20/0.20/0.20/0.10并记录算法版本，云候选冻结有效layout配置权重。模板槽位固定在页面；若用justified几何只能作为内部候选，必须重新适配固定框且验证坐标／正尺寸／明示重叠规则。cover/contain生成合法crop；无可信主体提示时显示未知，不声称保护成功。五项评分均为[0,1]：subject取提示框保留比例，无提示为0.5；timeline取1 − 输入顺序逆序对比例；whitespace取接近模板声明留白比例的程度；variety取与相邻已固定跨页的模板／框位差异；color无已授权本地颜色摘要时为0.5。total为冻结五权重的归一化加权和；权重和必须>0，硬错误total=-1且直接淘汰，不接收用户任意脚本；新图片／文字保持独立可编辑层。主体提示来自当前项目局部元数据端口，不上传照片、不进行身份识别。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- packages/layout/src/compose.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add packages/layout/src/compose.ts packages/layout/src/score.ts packages/layout/src/compose.test.ts；git commit -m "feat(layout): compose editable fixed-page candidates"。

Task 4: 从真实图层核对manifest与候选硬约束

Files:

- Create: packages/layout/src/integrity.ts

- Create: packages/layout/src/candidates.ts

- Create: packages/layout/src/integrity.test.ts

Interfaces:

- Consumes: 任务1 – 3类型及C1文档；layout.schema.json定义的AssignmentManifest。

- Produces: Placement={assetId:string,spreadId:string,layerId:string}；LayoutViolation={code:string,message:string,assetIds?:string[],layerIds?:string[]}；AssignmentManifest={version?:1,selectionPolicy:LayoutInput["constraints"]["selectionPolicy"],duplicatePolicy:LayoutInput["constraints"]["duplicatePolicy"],inputAssetIds:string[],placements:Placement[],unusedAssetIds:string[],repeatedAssetIds:string[],violations:LayoutViolation[]}；LayoutCandidate={candidateId:string,baseRevision:number,inputHash:string,templateVersion:number,algorithmVersion:string,seed:number,spreads:DocumentModel["spreads"],assignmentManifest:AssignmentManifest,constraintReport:{valid:boolean,violations:LayoutViolation[]},warnings:LayoutViolation[]}；deriveAssignment(input:LayoutInput,doc:DocumentModel):AssignmentManifest、verifyCandidate(input:LayoutInput,before:DocumentModel,candidate:LayoutCandidate):string[]、generateCandidates(prepared:PreparedLayout,doc:DocumentModel,template:TemplateVersion,hints:SubjectHint[]):LayoutCandidate[]。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(valid.assignmentManifest.placements).toHaveLength(12);
expect(verifyCandidate(input,before,elevenClaimingTwelve)).toContain('MANIFEST_DOCUMENT_MISMATCH');
expect(deriveAssignment(input,docWithDecorativeImage).placements).toHaveLength(12);
expect(verifyCandidate(input,before,changedLockedText)).toContain('LOCKED_LAYER_CHANGED');
expect(generateCandidates(prepared,doc,template,[])).toHaveLength(2); // fixture has only 2 valid variants
expect(allowSelectionManifest.unusedAssetIds.sort()).toEqual(['optional-1','optional-2']);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- packages/layout/src/integrity.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在integrity.ts实现派生／复验函数，在candidates.ts实现generateCandidates(...)。照片层用layoutPhotoRole="selection"扩展字段和稳定inputHash追溯，仅此用途计选片，装饰图片不计；已锁定旧层通过任务1准备时显式

映射，不能靠文件名猜测。ComposedSpread是内部评分记录，输出candidate.spreads严格转成C1实际spread，不把score/hardViolations塞进HTTP候选；约束和warnings按LayoutViolation输出。生成后、应用前和应用后的真实文档各校验一次；总清单包括targetSpreadIds外已分配选片。候选以prepared.policy.candidateCount为上限且最多3个，去重相同规范几何，允许0／1／2个并返回不可行原因；manifest不能代替真实层校验。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- packages/layout/src/integrity.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add packages/layout/src/integrity.ts packages/layout/src/candidates.ts packages/layout/src/integrity.test.ts；git commit -m "feat(layout): verify assignment manifests against actual documents"。

Task 5: revision守卫、局部应用与导出完整性接入

Files:

- Create: packages/layout/src/apply.ts

- Create: packages/layout/src/apply.test.ts

- Modify: packages/export/src/preflight.ts

- Modify: packages/export/package.json

Interfaces:

- Consumes: 任务4 verifyCandidate／deriveAssignment、C1 DocumentCommand／DocumentModel；C1预检 preflight(snapshot:ExportSnapshot,port:ResourcePort,signal:AbortSignal):Promise<PreflightReport>。

- Produces: createLayoutCommand(input:LayoutInput,candidate:LayoutCandidate,current:DocumentModel):DocumentCommand；validateDocumentAssignment(doc:DocumentModel):string[]。排版上下文与manifest作为文档扩展元数据保存，导出校验无需再次调用算法。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(() => createLayoutCommand(input,candidateAt12,docAt13)).toThrow('REVISION_CONFLICT');
expect(after.spreads.find(s=>s.id==='outside')).toEqual(before.spreads.find(s=>s.id==='outside'));
expect(afterLockedLayer).toEqual(beforeLockedLayer);
expect(history.undo()).toEqual(before); // one command restores entire local re-layout
expect(validateDocumentAssignment(tampered)).toContain('MISSING_SELECTED_PHOTO');
expect(exportReport.blockers.map(i=>i.code)).toContain('LAYOUT_INTEGRITY_FAILED');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- packages/layout/src/apply.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在apply.ts实现两个签名；同时修改C1 packages/export/src/preflight.ts接入validateDocumentAssignment，修改packages/export/package.json声明@album/layout依赖（layout只依赖document，禁止反向依赖export）。应用先比baseRevision及inputHash，再验证完整片段、合并targetSpreadIds范围并重新派生；任何失败整个命令不执行。未知manifest版本仍阻断。云端C3在CAS事务中调用同一函数，前端命令不能授权覆盖并发变更。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- packages/layout/src/apply.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add packages/layout/src/apply.ts packages/layout/src/apply.test.ts packages/export/src/preflight.ts packages/export/package.json；git commit -m "feat(layout): apply revision-safe layouts and enforce export integrity"。

Task 6: 网页选片、候选核对与可恢复取消

Files:

- Create: apps/web/src/layout/LayoutWizard.tsx

- Create: apps/web/src/layout/CandidateComparison.tsx

- Create: apps/web/src/layout/layout-controller.ts

- Create: tests/browser/layout.spec.ts

- Modify: apps/web/src/editor/EditorPage.tsx

Interfaces:

- Consumes: 任务1 – 5接口；C1编辑器历史与当前保存revision。

- Produces: LayoutWizard(props:{projectId:string,onClose:()=>void}):React.JSX.Element；CandidateComparison(props:{candidates:LayoutCandidate[],onApply:(candidateId:string)=>void}):React.JSX.Element；computeLayouts(input:LayoutInput,doc:DocumentModel,template:TemplateVersion,signal:AbortSignal):Promise<LayoutCandidate[]>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
await expect(page.getByText('12 / 12 张已使用')).toBeVisible();
await expect(page.getByText('至少需要 25 个跨页')).toBeVisible();
await expect(page.getByRole('button',{name:'应用候选'})).toBeDisabled(); // invalid or stale candidate
await expect(page.getByText('候选已过期，请重新生成')).toBeVisible();
expect(generationNetworkRequests).toHaveLength(0);
expect(documentAfterCancel).toEqual(documentBeforeCompute);
```

☐ Step 2: 运行失败用例。 npm run test:browser -- tests/browser/layout.spec.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在三个新UI文件实现签名，EditorPage.tsx挂载入口。页面依次选片／顺序／场景／跨页数／每跨页范围／锁定与策略；全部使用和禁重为默认，允许筛选／明确重复必须主动选择。展示used/unused/repeated及每次出现位置、裁剪／主体安全警告；显式选择候选后才应用，一次撤销恢复。计算通过Web Worker或可中断分段任务执行，AbortSignal和请求序号丢弃迟到结果；取消保留现有文档。保存成功revision变化使旧候选不可用；空候选显示调整操作；仅实际候选数可见。

☐ Step 4: 验证通过。 重跑 npm run test:browser -- tests/browser/layout.spec.ts 与 npm run typecheck；预期PASS、退出码0。另执行 npm run build，预期退出0；以50张旅行与200张锁定样本检查真实UI，不宣称已达盲评指标。

☐ Step 5: 限定提交。 git add apps/web/src/layout/LayoutWizard.tsx apps/web/src/layout/CandidateComparison.tsx apps/web/src/layout/layout-controller.ts tests/browser/layout.spec.ts apps/web/src/editor/EditorPage.tsx；git commit -m "feat(web): compare layout candidates with complete photo accounting"。

自审与执行交接

已逐节对照v0.3 § 4 – 5、PRD LAY-01/02与v0.4编辑器配置：任务1／2覆盖排序、分组、必选禁用和容量，任务3覆盖比例／裁剪／主体提示与可编辑构图，任务4／5覆盖全量manifest、局部锁定、CAS和导出核对，任务6覆盖三候选、实际不足与取消。RF1→1/2，RF2→2/4，RF3→3，RF4→4，RF5→5/6均有具体断言。类型只在拥有任务定义；任务3引用C1 TemplateVersion，任务5不让layout依赖export，已检查依赖无环。6项均为完整功能增量，每项恰有5个可勾步骤，无通用“ 补测试 ”任务。

当前覆盖缺口：无首期功能缺项。视觉≥80%目标、真实浏览器性能与接入后的云端CAS仍须实施时实测；它们不是文档验收通过。后续按已确认的执行方式逐任务开发，本次仅交付计划，无需为继续编写其他模块等待增量审批。

C3 统一身份、私有素材、云revision与小程序成册 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (☐) syntax for tracking.

Goal: 交付网页与微信小程序统一身份、工作区权限、私有云素材／文件夹、跨端保存、分享通知和小程序快速成册闭环。

Architecture: NestJS模块化API使用FastifyAdapter承载HTTP，PostgreSQL作为身份、权限和revision事实源；存储对象与素材ID／目录解耦。两端调用同一合同，mini仅发白名单patch，云仓库适配C1端口；集成服务通过可替换适配器实现且默认禁用未验证能力。

Tech Stack: Node 24 LTS、TypeScript、NestJS＋FastifyAdapter、pg、OpenAPI类型生成、Vitest、Playwright；原生微信小程序TypeScript与wx APIs；私有OSS/COS/S3按实际已实现适配器启用。

Spec: v0.3 §8、账号与权益、v0.4配置、存储与文件夹、API合同。

Global Constraints

- 本文是待执行研发计划；测试片段和预期结果均不是当前已通过记录。只写本模块实现；真实微信、存储、支付、AI集成缺凭据时保持 unconfigured，本地模拟／契约测试可完整实施。

- 继承 共同合同：公开ID为1 – 128字符不透明字符串；SHA-256为64位小写十六进制；金额为人民币分整数、点数整数，JSON整数上限9007199254740991；UTC存储，展示和预算日为Asia/Shanghai。

- API导出格式枚举固定为png／jpeg／pdf，产物容器可为zip；.jpg只作为文件扩展名或用户文案，不能用作API格式枚举。

- API根 /v1；成功 {data,requestId}，失败 {error:{code,message,retryable,details},requestId}；列表 {items,nextCursor,hasMore}。网页cookie＋CSRF，小程序短期bearer；所有资源逐次校验工作区权限。

- 可重放创建／财务／任务操作使用空间范围唯一的 Idempotency-Key，保留7天；相同key不同payload返回409。文档If-Match revision为字符串；草稿／目录更新有expectedRevision／If-Match，冲突409。

- 唯一产品文档模型由C1 @album/document提供，对应 document.schema.json；schemaVersion1、px96、sRGB；1 – 50内页跨页、500照片、100递归图层／跨页、组深度≤5。原片JPEG／PNG≤25_000_000字节且≤80MP；不接受任意外部URL素材引用。

- 平台统一存储／AI连接，不开放BYOS／BYOK；免费与付费能力都实现，真实价格／配额未经确认不得伪造。影楼首期单使用者，预留owner/admin/designer/viewer与seat_limit，不做邀请UI；App仅预留共享API。

- 测试命令沿用C1 npm run test:unit -- <file>／npm run test:browser -- <file>、npm run typecheck。集成测试使用临时PostgreSQL／Redis和本地受控模拟，禁止生产数据库；精确依赖版本在实施时锁入package-lock.json，Node 24 LTS。

- 微信身份键为provider＋app_id＋openid；只在同开放平台可信来源的UnionID关联，不因手机号／昵称相同合并资产或订单。无UnionID仍可登录。

- 环境固定dev/staging/prod，由部署注入，不接受业务请求切换env。正式凭据仅秘密引用，平台管理员不自动获得用户原片权限；短信备用只在已开通且验证的配置下启用。

- 文件夹名称1 – 80个Unicode码点、NFC＋大小写折叠、最多10级；禁止/、\、控制字符、.、..；basePrefix为1 – 256字符相对前缀，keyPatternVersion=v1，publicAccess=false。

- 上传／下载凭据默认900／300秒，正整数≤3600；回收期建议30天，导出产物7天、未完成上传临时对象24小时，但不清理活动pin；所有期限来自已发布政策。

- 云保存至少最近20个恢复revision；自动保存空闲2秒／连续最多10秒沿用C1；小程序不提供自由移动图层／加形状／完整网页编辑器。

Review Focus

- RF1：OpenID复用到不同app、UnionID缺失、手机相同和扫码重放不得误合并账号；任务1有断言。

- RF2：跨工作区ID混用、角色刚被撤销、普通管理员读取原片须拒绝；任务2／3有断言。

- RF3：并发预占、伪造图片、prefix改名和迁移校验失败不越配额或断旧资源；任务3／4有断言。

- RF4：离线双端提交、mini未知层字段、迟到保存响应不得静默覆盖或丢草稿；任务5／6／8有断言。

- RF5：分享撤销、签名URL存续、注销和通知重放不能继续新授权或重复发消息；任务1／7有断言。

文件与合同边界

下列产品路径均为未来实施时创建／修改，当前不生成产品代码。API只实现 OpenAPI 已定义操作与参数；DTO从合同生成到packages/contracts/src/api.ts（C3任务1），不自行维护第二套HTTP字段。五个schema使用固定文件名与各自$defs.root。数据库执行 001_initial.up.sql 与对应down迁移，并遵守 transaction-safety.md；发现合同和本计划有差异先修订合同并加反例，再实现。应用内部类型由本计划拥有它的任务定义并导出，测试数据在相应测试文件内构造，不依赖未定义的未来测试帮助函数。

文件与端口分工

apps/api/src/auth拥有身份／会话与注销；workspaces是逐请求授权边界；assets拥有上传／对象locator；folders只管理逻辑目录；projects拥有CAS与mini白名单；sharing／notifications管理外发授权。packages/server-core只提供数据库事务、幂等记录与时钟，禁止放入会员或供应商逻辑。C4权益通过本模块定义的只读QuotaPolicyPort注入，首期本地策略也必须有真实边界；不依赖尚未实现的会员函数。

Task 1: 合同驱动API、微信身份、绑定与注销

Files:

- Create: packages/contracts/package.json

- Create: packages/contracts/src/api.ts

- Create: scripts/generate-api-types.ts

- Create: packages/server-core/package.json

- Create: packages/server-core/src/database.ts

- Create: packages/server-core/src/idempotency.ts

- Create: packages/server-core/src/index.ts

- Create: apps/api/package.json

- Create: apps/api/src/main.ts

- Create: apps/api/src/auth/types.ts

- Create: apps/api/src/auth/identity.ts

- Create: apps/api/src/auth/sessions.ts

- Create: apps/api/src/auth/account-deletion.ts

- Create: apps/api/src/auth/privacy-export.ts

- Create: apps/api/src/auth/auth.test.ts

- Modify: package.json

- Modify: package-lock.json

- Create: packages/server-core/src/actor.ts

- Create: apps/api/src/app.module.ts

- Create: apps/api/src/auth/auth.module.ts

- Create: apps/api/src/auth/controller.ts

- Create: apps/api/src/http/contract-http.ts

Interfaces:

- Consumes: OpenAPI及database/001_initial.up.sql；部署注入的dev/staging/prod和秘密适配器。

- Produces: Database={transaction<T>(work:(tx:PoolClient)=>Promise<T>):Promise<T>}；Actor={userId:string,sessionId:string,client:"web"|"mini",platformPermissions:string[]}（由server-core/actor.ts定义并在auth/types.ts re-export，避免共享jobs反向依赖API）；VerifiedIdentity={provider:"wechat"|"sms",appId:string,subject:string,unionId:string|null,openPlatformId:string|null}；IdentityAdapter={exchange(code:string,redirectUri:string|null):Promise<VerifiedIdentity>}；AuthService.login(adapter:IdentityAdapter,code:string,client:"web"|"mini"):Promise<{actor:Actor,accessToken:string,refreshToken:string}>、beginWebLogin(browserNonce:string):Promise<{loginId:string,expiresAt:string}>、consumeWebLogin(loginId:string,nonce:string):Promise<Actor>、bind(actor:Actor,verified:VerifiedIdentity):Promise<void>、rotateRefresh(token:string):Promise<{accessToken:string,refreshToken:string}>、requestAccountDeletion(actor:Actor,input:components["schemas"]["DeletionRequest"]):Promise<components["schemas"]["DeletionStatus"]>；cancelAccountDeletion(actor:Actor):Promise<components["schemas"]["DeletionStatus"]>；requestPrivacyExport(actor:Actor,key:string):Promise<{jobId:string}>；runPrivacyExport(lease:JobLease):Promise<void>（privacy-export.ts，JobLease由C4任务2提供）。runIdempotent<T>(tx:PoolClient,scope:string,key:string,payloadHash:string,work:()=>Promise<T>):Promise<T>定义在server-core。HTTP组合接口bindContractHttp(app:NestFastifyApplication):Promise<void>定义在http/contract-http.ts；领域端口不依赖Nest装饰器。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(await auth.login(noUnionIdAdapter,'verified-code','mini')).toMatchObject({actor:{userId:expect.any(String)}});
expect(userForAppA).not.toBe(userForAppB); // same openid, different appId
expect(userWithSamePhone).not.toBe(existingUser);
await expect(auth.consumeWebLogin(loginId,nonce)).rejects.toThrow('LOGIN_ALREADY_CONSUMED');
await expect(auth.rotateRefresh(reusedToken)).rejects.toThrow('REFRESH_REUSE');
expect(afterDeletion).toMatchObject({activeSessions:0,activeShares:0,canCreateJob:false});
expect(privacyExportUserIds).toEqual([actor.userId]);
expect(deletionWithPendingRefund.blockers).toContain('pending_refund');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/auth/auth.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在所列文件实现上述接口；generate-api-types.ts从OpenAPI生成paths/components/operations类型，禁止手改生成DTO。database.ts实现PoolClient事务rollback/finally释放；idempotency.ts落实7天payload绑定。main.ts使用NestFactory.create<NestFastifyApplication>(AppModule,new FastifyAdapter())启动Nest应用并设置/v1前缀；app.module.ts组合领域模块，auth/auth.module.ts注册身份端口providers与AuthController，auth/controller.ts按生成OpenAPI operationId绑定登录／会话／绑定／注销／隐私导出路由，领域服务仍使用Interfaces中的原签名。http/contract-http.ts定义并实现bindContractHttp(app:NestFastifyApplication):Promise<void>，把生成合同请求／响应校验、Actor认证Guard、CSRF Guard、统一envelope／异常过滤器、requestId与受控CORS安装到Nest/Fastify生命周期；微信登录与独立验签回调按合同声明的security例外处理，不用客户端参数跳过Guard。登录code只在服务器交换，二维码请求按合同由浏览器创建browserNonce，原浏览器cookie与nonce双绑定且一次消费，确认终端不能直接拿走会话；工程默认二维码120秒、mini access 15分钟、refresh 7天且轮换，实际值受已发布auth配置限制。绑定需两个已验证主体和重新登录，已有不同账户时返回人工合并冲突，不自动转移订单。刷新重用撤销会话族，退出／禁用撤销会话。注销以已验证会话发起和二次确认；确认事务立即封禁新登录／任务／分享并写删除计划，冻结取消／财务保留引用，按合同requested/confirmed/blocked/executing/completed/cancelled与eligibleAt执行，owned_studio/pending_payment/pending_refund/active_job/legal_retention阻塞须明示；符合条件时confirmed即排期，不另发明scheduled状态，取消只能在执行前且不自动复活旧会话／分享；按已发布政策异步清理；必要订单审计脱敏保留。privacy-export.ts中requestPrivacyExport按POST /me/privacy-exports创建privacy_export任务，runPrivacyExport以fence执行、只用个人工作区授权范围，导出本人可访问数据／素材清单，私有短期下载并剔除秘密／其他用户数据；调度由C4任务2提供，接口未连接时不显示已生成。短信能力adapter默认unconfigured，实际短信资质／模板／频控验证后才开放。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/auth/auth.test.ts 与 npm run typecheck；预期PASS、退出码0。附带检查生成DTO无diff及cookie Secure/HttpOnly/SameSite、日志无code/token；提供本地测试身份模式但prod启动拒绝它。

☐ Step 5: 限定提交。 git add packages/contracts/package.json packages/contracts/src/api.ts scripts/generate-api-types.ts packages/server-core/package.json packages/server-core/src/database.ts packages/server-core/src/idempotency.ts packages/server-core/src/index.ts apps/api/package.json apps/api/src/main.ts apps/api/src/auth/types.ts apps/api/src/auth/identity.ts apps/api/src/auth/sessions.ts apps/api/src/auth/account-deletion.ts apps/api/src/auth/privacy-export.ts apps/api/src/auth/auth.test.ts

package.json package-lock.json packages/server-core/src/actor.ts apps/api/src/app.module.ts apps/api/src/auth/auth.module.ts apps/api/src/auth/controller.ts apps/api/src/http/contract-http.ts；git commit -m "feat(auth): add unified verified identities and deletion lifecycle"。

Task 2: 工作区、逐资源授权与席位预留

Files:

- Create: apps/api/src/workspaces/types.ts

- Create: apps/api/src/workspaces/authorization.ts

- Create: apps/api/src/workspaces/service.ts

- Create: apps/api/src/workspaces/workspaces.test.ts

- Create: apps/api/src/workspaces/workspaces.module.ts

- Create: apps/api/src/workspaces/controller.ts

- Modify: apps/api/src/app.module.ts

Interfaces:

- Consumes: 任务1 Actor、Database、runIdempotent；SQL工作区／成员表。

- Produces: WorkspaceRole="owner"|"admin"|"designer"|"viewer"；WorkspaceAction="read"|"edit"|"manageAssets"|"deleteAssets"|"purchase"|"share"|"manageSettings"；WorkspaceContext={workspaceId:string,userId:string,role:WorkspaceRole}；authorizeWorkspace(actor:Actor,workspaceId:string,action:WorkspaceAction,tx?:PoolClient):Promise<WorkspaceContext>；createWorkspace(actor:Actor,kind:"personal"|"studio",key:string):Promise<{workspaceId:string,seatLimit:number}>；updateWorkspaceSettings(actor:Actor,workspaceId:string,expectedRevision:number,settings:{defaultTemplateId:string|null,defaultSpreadCount:number,defaultWidthMm:number,defaultHeightMm:number}):Promise<{revision:number}>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
await expect(authorizeWorkspace(viewer,wid,'edit')).rejects.toThrow('FORBIDDEN');
await expect(authorizeWorkspace(admin,wid,'purchase')).rejects.toThrow('FORBIDDEN'); // purchase/redeem/refund share owner-only financial gate
expect((await authorizeWorkspace(admin,wid,'manageSettings')).role).toBe('admin');
await expect(authorizeWorkspace(ownerA,workspaceB,'read')).rejects.toThrow('NOT_FOUND');
await expect(authorizeWorkspace(removedDesigner,wid,'read')).rejects.toThrow('FORBIDDEN');
await expect(authorizeWorkspace(platformAdmin,wid,'read')).rejects.toThrow('NOT_FOUND');
expect(studio).toMatchObject({seatLimit:1});
expect(projectAfterChangingDefaults).toEqual(projectBeforeChangingDefaults);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/workspaces/workspaces.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在types.ts定义角色／动作／上下文，authorization.ts实现authorizeWorkspace；service.ts实现创建与设置CAS。个人工作区唯一本人；studio初始化seatLimit=1但保留成员／权益seat_limit字段，与设备数无关，不提供邀请入口。owner/admin可管理设置与素材；purchase动作仅owner，购买／兑换／申请退款的API共用此owner专属财务授权，admin不得由管理设置权限推导财务权限；designer编辑及素材整理，删除由已发布角色策略授权；viewer只读。每个资源查找用workspace_id＋id，数据库事务内重新检查撤销状态；不以JWT缓存角色作为最终授权。默认值仅用于新项目；不能修改平台配额、endpoint或价格。workspaces.module.ts注册WorkspaceController和授权providers，controller.ts按合同映射工作区／设置端点，AppModule导入该模块，HTTP wrapper负责认证与输入校验、领域层仍逐资源授权。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/workspaces/workspaces.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/workspaces/types.ts apps/api/src/workspaces/authorization.ts apps/api/src/workspaces/service.ts apps/api/src/workspaces/workspaces.test.ts apps/api/src/workspaces/workspaces.module.ts apps/api/src/workspaces/controller.ts apps/api/src/app.module.ts；git commit -m "feat(workspaces): enforce resource ownership and reserved studio roles"。

Task 3: 私有存储适配、上传验证与原子配额

Files:

- Create: apps/api/src/assets/types.ts

- Create: apps/api/src/assets/object-store.ts

- Create: apps/api/src/assets/upload.ts

- Create: apps/api/src/assets/resource-port.ts

- Create: apps/api/src/assets/upload.test.ts

- Create: apps/api/src/assets/assets.module.ts

- Create: apps/api/src/assets/controller.ts

- Modify: apps/api/src/app.module.ts

Interfaces:

- Consumes: 任务1 – 2端口；C1 importImage归一化规则、ResourcePort／ResourceWriter／AssetRecord；config.schema.json storage配置。

- Produces: StorageLocator={storageProfileId:string,profileVersion:number,bucket:string,key:string,objectVersionId:string|null,sha256:string,variant:string}；ObjectStore={put(locator:StorageLocator,bytes:Uint8Array):Promise<void>,read(locator:StorageLocator):Promise<Uint8Array>,head(locator:StorageLocator):Promise<{bytes:number,sha256:string}>,remove(locator:StorageLocator):Promise<void>,sign(locator:StorageLocator,mode:"upload"|"download",ttlSeconds:number):Promise<string>}；QuotaPolicyPort={readQuota(workspaceId:string):Promise<{limitBytes:number,maxProjectPhotos:number}>}；beginUpload(actor:Actor,wid:string,input:{bytes:number,sha256:string,mime:"image/jpeg"|"image/png",folderId:string|null,projectId:string|null},key:string):Promise<{uploadId:string,url:string,expiresAt:string}>；completeUpload(actor:Actor,wid:string,uploadId:string,key:string):Promise<AssetRecord>；createResourcePort(actor:Actor,wid:string):ResourcePort。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(await Promise.allSettled(twoConcurrentUploads)).toMatchObject([{status:'fulfilled'},{status:'rejected'}]); // quota fits exactly one
await expect(completeUpload(actor,wid,fakeJpegUploadId,key)).rejects.toThrow('INVALID_IMAGE');
await expect(beginUpload(actor,wid,{...uploadInput,bytes:25_000_001},key)).rejects.toThrow('UPLOAD_TOO_LARGE');
await expect(completeUpload(actor,wid,over80MpUploadId,key)).rejects.toThrow('IMAGE_PIXELS_EXCEEDED');
expect(await port.read(oldId,'original')).toEqual(originalBytes); // prefix changed
expect(usageAfterRejectedUpload.reservedBytes).toBe(0);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/assets/upload.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在object-store.ts定义ObjectStore并提供本地私有fake及已选择供应商adapter；未实现oss/cos/s3不得标ready。upload.ts实现begin/complete签名，事务预占原片及派生图保守上界、解码后原子调整实际字节；完成前HEAD、magic、解码像素、hash全部匹配，EXIF校正后存normalized与preview，原片不可变。服务端生成含UTC年月的固定token key，不使用文件夹／姓名／文件名；profile只影响新写入，旧资源按locator。下载每次授权再签300秒URL，资源端口绑定actor/wid且核对hash，禁止preview替代original。限额包含回收／历史／预占，工作区内去重不泄露跨区hash；会员过期超额只阻止新增，保留读／下载。活动上传24小时清理例外与幂等complete要有CAS。assets.module.ts注册素材／存储providers与AssetsController，controller.ts映射上传签发／complete、素材查询／下载／用量端点，AppModule导入该模块；不能让直接Fastify路由绕过Nest的合同／安全wrapper。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/assets/upload.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/assets/types.ts apps/api/src/assets/object-store.ts apps/api/src/assets/upload.ts apps/api/src/assets/resource-port.ts apps/api/src/assets/upload.test.ts apps/api/src/assets/assets.module.ts apps/api/src/assets/controller.ts apps/api/src/app.module.ts；git commit -m "feat(assets): validate private uploads with atomic storage reservations"。

Task 4: 逻辑文件夹、回收恢复与可恢复迁移

Files:

- Create: apps/api/src/folders/service.ts

- Create: apps/api/src/assets/migration.ts

- Create: apps/api/src/folders/folders.test.ts

- Create: apps/api/src/folders/folders.module.ts

- Create: apps/api/src/folders/controller.ts

- Create: apps/api/src/assets/migration-controller.ts

- Modify: apps/api/src/assets/assets.module.ts

- Modify: apps/api/src/app.module.ts

Interfaces:

- Consumes: 任务2授权、任务3 StorageLocator／ObjectStore；SQL asset_folders、storage_migration_items。

- Produces: AssetFolder={id:string,workspaceId:string,parentId:string|null,name:string,normalizedName:string,revision:number,deletedAt:string|null}；FolderMutation={name?:string,parentId?:string|null}；createFolder(actor:Actor,wid:string,parentId:string|null,name:string,key:string):Promise<AssetFolder>、updateFolder(actor:Actor,wid:string,fid:string,expectedRevision:number,patch:FolderMutation):Promise<AssetFolder>、batchMoveAssets(actor:Actor,wid:string,assetIds:string[],targetFolderId:string|null,key:string):Promise<void>、trashFolder(actor:Actor,wid:string,fid:string,expectedRevision:number,key:string):Promise<void>、restoreFolder(actor:Actor,wid:string,fid:string,targetParentId:string|null,name:string,key:string):Promise<AssetFolder>；migrateObject(itemId:string,store:ObjectStore):Promise<{state:"copied"|"verified"|"switched"|"failed",oldLocatorRetained:boolean}>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
await expect(updateFolder(actor,wid,parentId,1,{parentId:descendantId})).rejects.toThrow('FOLDER_CYCLE');
await expect(createFolder(actor,wid,tenthLevelId,'child',key)).rejects.toThrow('FOLDER_DEPTH_EXCEEDED');
await expect(updateFolder(actor,wid,folderId,1,{name:'CAF É '})).rejects.toThrow('FOLDER_NAME_CONFLICT'); // sibling caf é exists
expect(assetKeysAfterRename).toEqual(assetKeysBeforeRename);
await expect(trashFolder(actor,wid,referencedFolderId,1,key)).rejects.toThrow('ASSET_IN_USE');
expect(failedHashMigration).toMatchObject({state:'failed',oldLocatorRetained:true});
expect(locatorAfterFailedMigration).toEqual(oldLocator);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/folders/folders.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在folders/service.ts定义AssetFolder并实现五个目录／批量签名；同父活跃名称使用NFC＋大小写折叠唯一键，移动整棵子树算最大10级，不仅检查目标深度。批量操作同事务全成功／全拒绝，引用项目／任务的树删除列出影响并整体拒绝；回收不释放物理配额，恢复同名冲突不覆盖。assets/migration.ts实现migrateObject(itemId,store)：复制→SHA/尺寸核验→原locator revision条件更新→记录检查点；旧locator受活动snapshot pin保护直到释放／保留期，失败不切换。后台迁移创建需权限、估算费用和显式确认，取消只停止未复制项；目录操作永不触发对象迁移。folders.module.ts注册FoldersController和目录服务并由AppModule导入；folders/controller.ts映射目录／批量移动／回收恢复合同端点。assets/migration-controller.ts承载后台存储迁移创建／进度／取消路由，由AssetsModule注册并施加平台运维权限，不借工作区目录权限执行物理迁移。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/folders/folders.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/folders/service.ts apps/api/src/assets/migration.ts apps/api/src/folders/folders.test.ts apps/api/src/folders/folders.module.ts apps/api/src/folders/controller.ts apps/api/src/assets/migration-controller.ts apps/api/src/assets/assets.module.ts apps/api/src/app.module.ts；git commit -m "feat(storage): add transactional folders and checked object migration"。

Task 5: 云端revision仓库、恢复与mini白名单patch

Files:

- Create: apps/api/src/projects/revisions.ts

- Create: apps/api/src/projects/mini-patch.ts

- Create: apps/api/src/projects/layout-service.ts

- Create: packages/storage/src/cloud-revision-store.ts

- Create: apps/api/src/projects/revisions.test.ts

- Create: apps/api/src/projects/projects.module.ts

- Create: apps/api/src/projects/controller.ts

- Modify: apps/api/src/app.module.ts

Interfaces:

- Consumes: C1 RevisionStore、SavedRevision、SaveResult、DocumentModel；任务1 – 3授权／资源端口；C2 verifyCandidate、createLayoutCommand。

- Produces: saveCloudRevision(actor:Actor,wid:string,doc:DocumentModel,expectedRevision:number,resources:ResourceManifest,key:string):Promise<SavedRevision>；MiniPatch=components["schemas"]["SetText"]|components["schemas"]["SetCrop"]|components["schemas"]["ReplaceAsset"]|components["schemas"]["ReorderSpreads"]（使用合同type判别与spreadId，不另设op字段）；applyMiniPatch(actor:Actor,wid:string,projectId:string,expectedRevision:number,patches:MiniPatch[],key:string):Promise<SavedRevision>；CloudRevisionStore implements RevisionStore，构造器(baseUrl:string,workspaceId:string,fetcher:typeof fetch)，保持C1 load/save签名。createCloudLayoutJob(actor:Actor,wid:string,projectId:string,input:components["schemas"]["LayoutJobCreate"],key:string):Promise<JobRecord>、runCloudLayoutJob(lease:JobLease):Promise<void>、applyCloudLayout(actor:Actor,wid:string,projectId:string,candidateId:string,expectedRevision:number,key:string):Promise<SavedRevision>定义在layout-service.ts；JobRecord/JobLease由C4任务2提供。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(concurrentSaveResults.filter(r=>r.ok)).toHaveLength(1);
expect(conflict.status).toBe(409);
expect(advancedLayerAfterMini).toEqual(advancedLayerBeforeMini);  //  opaque  metadata  untouched
expect(changedText.text).toBe('新标题');
await  expect(applyMiniPatch(actor,wid,projectId,12,unsupportedPatch,key)).rejects.toThrow('MINI_PATCH_FORBIDDEN');
expect(recoverableRevisionCount).toBeGreaterThanOrEqual(20);
expect(cloudSaveFailure.recoveryDraft).toEqual(unsavedLocalDoc);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/projects/revisions.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在revisions.ts实现saveCloudRevision并按项目锁＋revision CAS保存规范JSON/hash和manifest；验证每个asset/font版本归属、ready、不可变hash及C2清单，引用对象pin／retention不被回收。mini-patch.ts只允许模板editableFields、允许替换槽位和合法crop；拒绝自由几何／字体／未知操作，使用C1模型局部更新保留未知同版层及扩展字段。cloud-revision-store.ts实现HTTP适配，将If-Match字符串revision与409映射为C1 SaveResult恢复草稿；离线先落IndexedDB恢复副本，不显示已云保存。候选apply在相同CAS事务验证baseRevision，不信客户端文档。至少20恢复revision和活动任务引用例外。layout-service.ts按精确OpenAPI挂接POST /projects/{projectId}/layout-jobs、GET /jobs/{jobId}/layout-candidates、POST /projects/{projectId}/apply-layout：LayoutJobCreate只有layoutSpec字段，路由projectId必须等于layoutSpec.projectId，不复制baseRevision/assetIds/seed到顶层；job冻结布局输入/权重，worker调用C2引擎、保存真实候选清单，应用在同一项目CAS事务调用C2校验。此云调度接线在C4任务2就绪后完成，不提前引用不存在的队列帮助函数。mini外部commands只接受合同setText/setCrop/replaceAsset/reorderSpreads，平台取会话；内部MiniPatch统一为这四种合同命令，replaceDocument仅web。projects.module.ts注册revision／mini／layout领域providers，controller.ts导出ProjectsController与LayoutCandidatesController映射项目保存／恢复、mini patch及上述layout合同端点，由AppModule导入；业务CAS在领域事务内完成。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/projects/revisions.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/projects/revisions.ts apps/api/src/projects/mini-patch.ts apps/api/src/projects/layout-service.ts packages/storage/src/cloud-revision-store.ts apps/api/src/projects/revisions.test.ts apps/api/src/projects/projects.module.ts apps/api/src/projects/controller.ts apps/api/src/app.module.ts；git commit -m "feat(projects): add cloud compare-and-swap revisions and safe mini patches"。

Task 6: 小程序快速成册、轻编辑与跨端任务中心

Files:

- Create: apps/mini/project.config.json

- Create: apps/mini/app.ts

- Create: apps/mini/app.json

- Create: apps/mini/src/api.ts

- Create: apps/mini/pages/album/index.ts

- Create: apps/mini/pages/album/index.wxml

- Create: apps/mini/pages/album/index.wxss

- Create: apps/mini/pages/tasks/index.ts

- Create: apps/mini/pages/tasks/index.wxml

- Create: apps/mini/src/album.test.ts

Interfaces:

- Consumes: 生成HTTP类型、任务1微信登录、任务3上传、任务4目录、任务5 MiniPatch、C2布局API；任务列表来自C4合同，未就绪只显示unconfigured。

- Produces: MiniAlbumState={projectId:string|null,revision:number|null,phase:"select"|"upload"|"layout"|"edit"|"preview",pendingPatches:MiniPatch[],recoverable:boolean}；loadMiniAlbum(projectId:string):Promise<MiniAlbumState>、submitMiniEdits(state:MiniAlbumState,key:string):Promise<MiniAlbumState>；listMiniTasks(cursor:string|null):Promise<{items:Array<{jobId:string,state:string}>,nextCursor:string|null,hasMore:boolean}>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(travelAlbum.selectedPhotoCount).toBe(50);
expect(travelAlbum.spreadCount).toBe(10);
expect(offlineState).toMatchObject({recoverable:true,pendingPatches:expectedPatches});
expect(after409.pendingPatches).toEqual(expectedPatches);
expect(advancedFieldAfterMiniSave).toEqual(advancedFieldBeforeMiniSave);
expect(taskSeenOnMini.jobId).toBe(taskCreatedOnWeb.jobId);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/mini/src/album.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在api.ts实现统一envelope、bearer轮换、过期登录和HTTP错误；相册页实现选片逐项上传重试、简易目录选择／新建、主题／页数／候选对比、允许文字替换／单图裁剪、预览／保存和导出任务入口。状态机函数按上述签名放pages/album/index.ts并可在Node测试；小程序wx存储只保存恢复patch和revision，不保存refresh明文日志。409显示保留本地草稿、复制恢复或重新加载，不自动重放到新revision。需要高级编辑显示“ 可在电脑网页继续编辑 ”；不把提示变成未开通渠道购买引导。任务页按服务端事实源分页查询，页面关闭不取消后台任务，未理解图层不重序列化。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/mini/src/album.test.ts 与 npm run typecheck；预期PASS、退出码0。真实微信开发者工具与iOS/Android授权设备流程留实施证据，缺AppID只标联调待接入，不阻断本地状态机与mock合同验收。

☐ Step 5: 限定提交。 git add apps/mini/project.config.json apps/mini/app.ts apps/mini/app.json apps/mini/src/api.ts apps/mini/pages/album/index.ts apps/mini/pages/album/index.wxml apps/mini/pages/album/index.wxss apps/mini/pages/tasks/index.ts apps/mini/pages/tasks/index.wxml apps/mini/src/album.test.ts；git commit -m "feat(mini): build quick-album workflow with safe cross-device edits"。

Task 7: 私有分享、下载授权与用户授权通知

Files:

- Create: apps/api/src/sharing/service.ts

- Create: apps/api/src/notifications/service.ts

- Create: apps/api/src/sharing/sharing.test.ts

- Create: apps/api/src/sharing/sharing.module.ts

- Create: apps/api/src/sharing/controller.ts

- Create: apps/api/src/notifications/notifications.module.ts

- Create: apps/api/src/notifications/controller.ts

- Modify: apps/api/src/app.module.ts

Interfaces:

- Consumes: 任务1 Actor／注销、任务2权限、任务3ObjectStore；SQL分享／通知／outbox。

- Produces: createShare(actor:Actor,wid:string,projectId:string,expiresAt:string,key:string):Promise<{shareId:string,url:string}>；revokeShare(actor:Actor,wid:string,shareId:string):Promise<void>；authorizeShare(token:string,resourceId:string):Promise<{workspaceId:string,resourceId:string}>；NotificationAdapter={send(input:{recipientId:string,templateVersion:string,fields:Record<string,string>}):Promise<{providerMessageId:string}>}；deliverNotification(eventId:string,adapter:NotificationAdapter):Promise<"sent"|"skipped"|"retry">。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
await expect(authorizeShare(revokedToken,resourceId)).rejects.toThrow('SHARE_REVOKED');
expect(existingSignedUrlExpirySeconds).toBe(300);
await expect(authorizeShare(revokedToken,resourceId)).rejects.toThrow('SHARE_REVOKED'); // no new signed URL issued
expect(messageCountAfter100Replays).toBe(1);
expect(notificationAfterOptOut).toBe('skipped');
expect(smsUnconfiguredResult).toBe('skipped');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/sharing/sharing.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在sharing/service.ts实现三签名，随机高熵token仅存hash，默认私有，分享范围明确到项目／冻结预览资源，不泄露原片、GPS或其他项目；到期／撤销／注销后拒绝新签名。UI准确说明直连300秒窗口，需即时撤销时切网关逐次鉴权且无公开缓存。notifications/service.ts定义adapter并实现deliverNotification：站内记录＋可选微信订阅／短信，检查实际授权、模板版本、退订及工作区访问；eventId＋recipient＋channel唯一，Outbox投递重试不重复发送。短信默认disabled；未配置通知不使任务失败。不可让管理员随请求传收件地址绕过已验证目的地。SharingModule／NotificationsModule各注册对应controller.ts和服务providers并由AppModule导入，分别映射私有分享／公开token访问／撤销与通知列表／偏好合同操作；公开分享路径仅使用合同token授权，其他路由继续使用统一Actor Guard。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/sharing/sharing.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/sharing/service.ts apps/api/src/notifications/service.ts apps/api/src/sharing/sharing.test.ts apps/api/src/sharing/sharing.module.ts apps/api/src/sharing/controller.ts apps/api/src/notifications/notifications.module.ts apps/api/src/notifications/controller.ts apps/api/src/app.module.ts；git commit -m "feat(sharing): enforce revocable access and consented notifications"。

Task 8: 网页云工作区、目录界面与恢复状态整合

Files:

- Create: apps/web/src/auth/LoginPanel.tsx

- Create: apps/web/src/workspaces/WorkspaceSwitcher.tsx

- Create: apps/web/src/assets/FolderBrowser.tsx

- Create: apps/web/src/projects/CloudProjectProvider.tsx

- Create: apps/web/src/account/AccountSettings.tsx

- Create: tests/browser/cloud-workspace.spec.ts

- Modify: apps/web/src/editor/EditorPage.tsx

Interfaces:

- Consumes: 任务1 – 7服务和CloudRevisionStore；C1本地仓库／恢复项目包；C2候选核对。

- Produces: CloudProjectProvider(props:{workspaceId:string,children:React.ReactNode}):React.JSX.Element；FolderBrowser(props:{workspaceId:string,onSelectAsset:(assetId:string)=>void}):React.JSX.Element；WorkspaceSwitcher(props:{onChange:(workspaceId:string)=>void}):React.JSX.Element；AccountSettings():React.JSX.Element。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
  await    expect(page.getByText('云端保存失败，本地草稿已保留')).toBeVisible();
  await    expect(page.getByText('该版本已由其他设备修改')).toBeVisible();
  await    expect(page.getByText('移入回收站，不会立即删除云对象')).toBeVisible();
  await    expect(page.getByRole('button',{name:'邀请成员'})).toHaveCount(0);
  expect(documentAfterLateSaveResponse).toEqual(currentWorkspaceDocument);
  await    expect(page.getByText('短信登录未开通')).toBeVisible();
```

☐ Step 2: 运行失败用例。 npm run test:browser -- tests/browser/cloud-workspace.spec.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 实现上述组件与LoginPanel／AccountSettings并接入EditorPage。登录支持微信扫码过期刷新与明确平台状态；只有短信能力available才显示可用入口。工作区切换绑定项目／资源／购买上下文并清理迟到响应；云保存中／成功／离线／失败文案与真实状态一致。目录面包屑、名称／时间排序、搜索、批量移动、当前目录上传、回收恢复、引用影响和存储用量分类完整呈现；无权操作不显示成功按钮。个人设置包括语言／展示时区／快捷键，工作区默认只影响新建相册；账号注销展示实际政策和订单保留说明后请求删除。分享显示有效期／撤销与签名窗口；保留C1本地项目包导入／恢复入口。

☐ Step 4: 验证通过。 重跑 npm run test:browser -- tests/browser/cloud-workspace.spec.ts 与 npm run typecheck；预期PASS、退出码0。运行npm run build并验证网页→mini改标题／crop→网页恢复高级字段的合同回放，实际设备另记。

☐ Step 5: 限定提交。 git add apps/web/src/auth/LoginPanel.tsx apps/web/src/workspaces/WorkspaceSwitcher.tsx apps/web/src/assets/FolderBrowser.tsx apps/web/src/projects/CloudProjectProvider.tsx apps/web/src/account/AccountSettings.tsx tests/browser/cloud-workspace.spec.ts apps/web/src/editor/EditorPage.tsx；git commit -m "feat(web): integrate cloud workspaces folders and recovery states"。

自审与执行交接

已复核NestJS＋FastifyAdapter组合、模块／controller与统一合同安全wrapper，领域公开端口保持原签名；admin购买拒绝及owner专属购买／兑换／退款有任务2失败断言，机器格式统一jpeg。8项覆盖AUTH-01/02、AST-01、DOC-01、V3-CROSS-01、CFG-STORAGE/FOLDERS/AUTH/NOTIFY、SHARE-01、ACCOUNT-DELETE／PRIVACY-01与APP-RESERVE。RF1→1、RF2→2/3、RF3→3/4、RF4→5/6/8、RF5→1/7都有失败断言。已检查ResourcePort与RevisionStore保持C1签名，MiniPatch不另定义产品文档；云仓库把数值revision转If-Match字符串；应用服务都在拥有任务定义。迁移、目录与账号删除都具备引用／pin保护，不用测试任务代替功能。

当前覆盖缺口：无首期功能缺项；真实微信扫码／小程序／短信和云厂商适配仅在经营者确认接入后做真实联调，不宣称模拟即生产可用。通知的可靠调度与账本依赖C4任务事实源，C3先以相同数据库outbox接口实施；允许本地完整验证，不等待真实凭据。本轮不编码，不逐模块请求增量批准。

C4 配置发布、可靠任务、导出与商业账本 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (☐) syntax for tracking.

Goal: 交付原子配置发布、持久任务／云导出、真实服务端订单状态、会员／AI点数、兑换退款与对账运营闭环。

Architecture: PostgreSQL事务和Outbox是任务及商业事实源，BullMQ仅调度；租约与fence隔离迟到worker。订单、entitlement_grants、credit_lots及不可变ledger分离，支付回调／查单共用履约入口；配置release与权益／导出snapshot在接受任务时冻结。

Tech Stack: Node 24 LTS、TypeScript、pg、Redis／BullMQ、固定Playwright Chromium、C1 render-cli、React admin/web、Vitest；渠道adapter与密钥服务端口。

Spec: 会员支付设计、v0.3 § § 6 – 7、v0.4配置发布、配置验收、测试运维。

Global Constraints

- 本文是待执行研发计划；测试片段和预期结果均不是当前已通过记录。只写本模块实现；真实微信、存储、支付、AI集成缺凭据时保持 unconfigured，本地模拟／契约测试可完整实施。

- 继承 共同合同：公开ID为1 – 128字符不透明字符串；SHA-256为64位小写十六进制；金额为人民币分整数、点数整数，JSON整数上限9007199254740991；UTC存储，展示和预算日为Asia/Shanghai。

- API根 /v1；成功 {data,requestId}，失败 {error:{code,message,retryable,details},requestId}；列表 {items,nextCursor,hasMore}。网页cookie＋CSRF，小程序短期bearer；所有资源逐次校验工作区权限。

- 可重放创建／财务／任务操作使用空间范围唯一的 Idempotency-Key，保留7天；相同key不同payload返回409。文档If-Match revision为字符串；草稿／目录更新有expectedRevision／If-Match，冲突409。

- 唯一产品文档模型由C1 @album/document提供，对应 document.schema.json；schemaVersion1、px96、sRGB；1 – 50内页跨页、500照片、100递归图层／跨页、组深度≤5。原片JPEG／PNG≤25_000_000字节且≤80MP；不接受任意外部URL素材引用。

- 平台统一存储／AI连接，不开放BYOS／BYOK；免费与付费能力都实现，真实价格／配额未经确认不得伪造。影楼首期单使用者，预留owner/admin/designer/viewer与seat_limit，不做邀请UI；App仅预留共享API。

- 测试命令沿用C1 npm run test:unit -- <file>／npm run test:browser -- <file>、npm run typecheck。集成测试使用临时PostgreSQL／Redis和本地受控模拟，禁止生产数据库；精确依赖版本在实施时锁入package-lock.json，Node 24 LTS。

- 任务事实源PostgreSQL，Outbox＋BullMQ仅调度；取消与成功只能一个终态；租约／fence守卫所有结果与账本终结。重试建议3次，退避10／30／90秒；OOM不盲重试相同超预算任务。

- 配置draft→validated→published→retired；测试绑定配置hash＋secretVersion，5分钟有效。回滚产生新release，禁止复活已撤销密钥，已有订单／快照／账本不回滚。

- 导出DPI72／150／300、50MP／输出面，headless预算2GiB；PPI100 – 199警告，<100强警告需确认。600×300mm、300DPI、四边3mm出血→7157×3614px；普通PDF为sRGB栅格页。

- 首期无自动续费；期限SKU按duration_days（30／365天等）计算，同档start=max(now,current_end)。价格以经营者发布的真实SKU为准；未发布不显示虚构价。

- 支付回调重放100次仅发放一次；待支付5分钟仍未知进入查单，关单前先查询。网页微信支付与小程序虚拟权益支付独立adapter／准入capability，不默认复用wx.requestPayment。

- 点数报价不预占，接受任务原子预占，成功一次结算，明确失败／取消一次释放到原批次。状态未知建议30分钟终局，超时释放后迟到结果不再扣点／发布；退款冻结额度不得消费。

Review Focus

- RF1：配置并发编辑、失效测试、旧密钥回滚和进程刷新失败不混用release或泄密；任务1有断言。

- RF2：Redis丢失、重复派发、worker失租、取消／成功竞态不能产生两个终态或可下载半成品；任务2／3有断言。

- RF3：回调100次、回调与查单并发、金额／应用被篡改只履约合法订单一次；任务4／5有断言。

- RF4：点数刚好够用、批次在预占期间到期、退款冻结和并发兑换不能负余额或错误恢复有效期；任务5／6／7有断言。

- RF5：排队时会员到期、退款迟到、已扣款本地关单和跨日对账不能抹去收款或重复补发；任务3／7／8有断言。

文件与合同边界

下列产品路径均为未来实施时创建／修改，当前不生成产品代码。API只实现 OpenAPI 已定义操作与参数；DTO从合同生成到packages/contracts/src/api.ts（C3任务1），不自行维护第二套HTTP字段。五个schema使用固定文件名与各自$defs.root。数据库执行 001_initial.up.sql 与对应down迁移，并遵守 transaction-safety.md；发现合同和本计划有差异先修订合同并加反例，再实现。应用内部类型由本计划拥有它的任务定义并导出，测试数据在相应测试文件内构造，不依赖未定义的未来测试帮助函数。

文件与服务分工

apps/api/src/config拥有所有typed config草稿／发布与秘密引用；packages/jobs提供持久任务状态机，apps/export-worker只执行冻结快照。payments规范化渠道事件，commerce实现订单／权益／点数／兑换／退款／对账；共同事务使用C3 Database，不复制第二套账本。配置中心负责20个业务域的校验／权限／未配置能力；schema中的albumDefaults作为独立typed payload也必须覆盖。AI模型／预设内容和样图测试UI归C5，平台账本及发布机械归本模块。

Task 1: 类型化配置中心、秘密轮换、原子release与能力

Files:

- Create: apps/api/src/config/types.ts

- Create: apps/api/src/config/secrets.ts

- Create: apps/api/src/config/network-policy.ts

- Create: apps/api/src/config/service.ts

- Create: apps/api/src/config/capabilities.ts

- Create: apps/api/src/config/config.test.ts

Interfaces:

- Consumes: C3 Database／Actor／runIdempotent；config.schema.json全部root种类；001_initial.up.sql配置实体。

- Produces: ConfigPayload为config.schema.json root生成的判别联合；ConfigVersion={id:string,namespace:string,version:number,environment:"dev"|"staging"|"prod",payload:ConfigPayload,revision:number,state:"draft"|"validated"|"published"|"retired",payloadHash:string}；ConfigRelease={releaseId:string,versionIds:string[],versions:Readonly<Record<string,ConfigVersion>>,effectiveAt:string}（内部有效配置快照，HTTP只投影生成ConfigRelease DTO字段）；SecretPort={write(name:string,value:Uint8Array):Promise<{secretRef:string,secretVersion:string}>,read(ref:string,version:string):Promise<Uint8Array>,isRevoked(ref:string,version:string):Promise<boolean>}；createConfigDraft(actor:Actor,namespace:string,payload:ConfigPayload,baseVersion:number|null):Promise<ConfigVersion>、updateConfigDraft(actor:Actor,id:string,expectedRevision:number,payload:ConfigPayload):Promise<ConfigVersion>、publishConfig(actor:Actor,versionIds:string[],testIds:string[],reason:string,expectedCurrentRelease:string|null):Promise<ConfigRelease>、rollbackConfig(actor:Actor,releaseId:string,expectedCurrentRelease:string,reason:string):Promise<ConfigRelease>、loadRelease(releaseId?:string):Promise<Readonly<ConfigRelease>>；assertApprovedHttps(url:string,allowedHosts:string[]):Promise<void>；fetchApprovedHttps(url:string,init:RequestInit,allowedHosts:string[],limits:{maxBytes:number,timeoutMs:number}):Promise<Uint8Array>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(concurrentDraftWrites.filter(r=>r.status===200)).toHaveLength(1);
await  expect(publishConfig(ops,versions,[expiredTestId],reason,current)).rejects.toThrow('CONFIG_TEST_EXPIRED');
await  expect(rollbackConfig(ops,oldRelease,current,reason)).rejects.toThrow('SECRET_REVOKED');
expect(snapshotAfterRefreshFailure.releaseId).toBe(previousRelease.releaseId);
expect(JSON.stringify(publicCapabilities)).not.toContain('secretRef');
```

```
await expect(assertApprovedHttps('https://127.0.0.1/model',['provider.example'])).rejects.toThrow('ENDPOINT_FORBIDDEN');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/config/config.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在所列文件实现签名。每种payload注册schema、命名空间角色、秘密字段、依赖／有效版本、默认禁用、功能测试级别、审计与snapshot影响，不只做通用JSON表单。平台运维／财务／内容分权，发布需重新验证会话；secret只写新版本／返回掩码，日志不记明文。network-policy.ts实现两个网络签名：DNS解析、IPv4/6私网／loopback／link-local／metadata、逐跳重定向都验证，fetchApprovedHttps用固定已验证地址的连接lookup/SNI校验封住DNS重绑定窗口，不是校验完再任意fetch；生产只允许部署批准HTTPS主机，本地模拟隔离。发布事务验证5分钟内测试绑定hash／secretVersion、依赖和未撤销状态后CAS切指针，worker一次加载完整release失败沿用旧版告警。回滚重新验证并生成新release，不回滚账本。能力仅available/disabled/unconfigured/degraded与通用原因；支持紧急停新请求而保留查询／终结。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/config/config.test.ts 与 npm run typecheck；预期PASS、退出码0。遍历全部schema root kind验证注册覆盖，含albumDefaults；模拟本地对象PUT/GET/hash/DELETE测试的删除残留明确告警，不删既有key。

☐ Step 5: 限定提交。 git add apps/api/src/config/types.ts apps/api/src/config/secrets.ts apps/api/src/config/network-policy.ts apps/api/src/config/service.ts apps/api/src/config/capabilities.ts apps/api/src/config/config.test.ts；git commit -m "feat(config): publish validated releases with secret-safe capabilities"。

Task 2: 数据库事实源、Outbox、租约fence与取消

Files:

- Create: packages/jobs/package.json

- Create: packages/jobs/src/types.ts

- Create: packages/jobs/src/repository.ts

- Create: packages/jobs/src/outbox.ts

- Create: packages/jobs/src/index.ts

- Create: packages/jobs/src/jobs.test.ts

- Modify: package-lock.json

Interfaces:

- Consumes: C3 Database与幂等事务；任务1 ConfigRelease；SQL jobs/job_attempts/outbox。

- Produces: JobState="queued"|"running"|"packaging"|"retry_wait"|"cancel_requested"|"succeeded"|"failed"|"cancelled"；JobRecord={jobId:string,workspaceId:string,kind:"export"|"generation"|"layout"|"storage_migration"|"privacy_export"|"config_test"|"reconciliation"|"backup_restore_drill",state:JobState,configReleaseId:string,attempt:number,fence:number,cancelRequested:boolean,providerSubmissionState:"not_submitted"|"submitting"|"accepted"|"unknown"|"terminal",stage:string}；JobLease={job:JobRecord,leaseOwner:string,fence:number,expiresAt:string}；createJob(tx:PoolClient,input:{workspaceId:string,kind:JobRecord["kind"],configReleaseId:string,inputHash:string,payload:unknown},key:string):Promise<JobRecord>、claimJob(jobId:string,workerId:string,now:string):Promise<JobLease|null>、renewLease(lease:JobLease,now:string):Promise<JobLease>、finishJob(lease:JobLease,outcome:{state:"succeeded"|"failed"|"cancelled",result:unknown}):Promise<boolean>、requestCancel(actor:Actor,wid:string,jobId:string):Promise<JobRecord>、dispatchOutbox(limit:number):Promise<number>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(await dispatchOutbox(100)).toBeGreaterThanOrEqual(1); // rebuild Redis solely from DB
expect(await finishJob(staleLease,{state:'succeeded',result:artifact})).toBe(false);
expect(terminalTransitionsForJob).toHaveLength(1);
expect(uniqueJobsForSameKey).toHaveLength(1);
expect(otherWorkspaceFirstDispatchIndex).toBeLessThan(hugeWorkspaceAllJobs.length);
expect(await claimJob(cancelledJobId,workerId,now)).toBeNull();
```

☐ Step 2: 运行失败用例。 npm run test:unit -- packages/jobs/src/jobs.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在repository.ts实现任务／租约函数；API与SQL同用queued/running/packaging/retry_wait/cancel_requested/succeeded/failed/cancelled；未知上游使用state=running、providerSubmissionState=unknown、stage=confirming_upstream，不能发明独立持久态；接口JobRecord.jobId映射HTTP Job.id。outbox.ts实现派发；PG行锁／条件更新校验当前fence+未终态，续租有界且失租即停止写结果。取消queued原子终局，running标请求并由同一终局CAS决定赢家；结果对象上传完成也不能绕过fence。Outbox至少一次投递，唯一jobId和调度去重，Redis重建扫描未终态PG记录，不依赖队列历史。全局／工作区并发取部署／发布配置／权益最严值，轮转工作区公平调度；10/30/90秒最多3次仅对可重试明确错误，未知付费提交保持running＋providerSubmissionState=unknown＋stage=confirming_upstream而非重提。attempt记录脱敏原因、心跳和队列延迟，OOM标资源失败建议降规格／拆分。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- packages/jobs/src/jobs.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add packages/jobs/package.json packages/jobs/src/types.ts packages/jobs/src/repository.ts packages/jobs/src/outbox.ts packages/jobs/src/index.ts packages/jobs/src/jobs.test.ts package-lock.json；git commit -m "feat(jobs): add durable fenced jobs and replayable outbox"。

Task 3: 服务端导出快照、冻结权益与私有产物

Files:

- Create: apps/api/src/exports/service.ts

- Create: apps/export-worker/package.json

- Create: apps/export-worker/src/main.ts

- Create: apps/export-worker/src/render-job.ts

- Create: apps/export-worker/src/export.test.ts

- Modify: package-lock.json

Interfaces:

- Consumes: C1 freezeSnapshot／preflight／authorizeExport／render:snapshot CLI；C3 ResourcePort／ObjectStore／授权；任务1 loadRelease、任务2 JobLease／createJob；任务5 getEntitlements（本项执行顺序在任务5核心接口就绪后接入，先用显式EntitlementPort）。

- Produces: EntitlementPort={getEntitlements(workspaceId:string,at:string):Promise<EntitlementSnapshot>}；EntitlementSnapshot={workspaceId:string,at:string,grantIds:string[],maxDpi:72|150|300,maxConcurrentExports:number,brandWatermark:boolean,storageLimitBytes:number}；acceptExport(actor:Actor,wid:string,input:{projectId:string,revision:number,options:ExportOptions,snapshotHash:string,acknowledgedWarningIds:string[]},key:string):Promise<JobRecord>；renderExportJob(lease:JobLease,signal:AbortSignal):Promise<void>；authorizeArtifactDownload(actor:Actor,wid:string,artifactId:string):Promise<{url:string,expiresAt:string}>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(artifact.revision).toBe(12);  //  save13  while12  queued
await  renderExportJob(leaseAcceptedBeforeExpiry,signal);
expect(jobAfterWorker.state).toBe('succeeded');
expect(renderedSize).toEqual({width:7157,height:3614});
expect(artifactVisibleAfterLostFence).toBe(false);
expect(artifactVisibleAfterCancel).toBe(false);
await  expect(acceptExport(actor,wid,staleWarningAck,key)).rejects.toThrow('PREFLIGHT_STALE');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/export-worker/src/export.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在exports/service.ts定义EntitlementPort/Snapshot并实现acceptExport／download；在render-job.ts实现renderExportJob。接受时重新鉴权／原图可读／预检hash与逐项警告确认，事务冻结saved revision+manifest+configReleaseId+entitlement_snapshot+资源locator pins并建job/outbox；权益检查只限提交，排队期间到期不撤销

快照。worker使用C1固定Chromium CLI，单面顺序处理、2GiB和50MP硬限，不允许外部HTTP／任意路径；真实PNG/JPG/PDF校验实际像素／DPI／页序后私有上传，fence成功后才公开artifact记录。取消／失败清临时对象，pins至可安全终结后释放；7天保留期到期不删除原片，下载URL过期重新鉴权签发。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/export-worker/src/export.test.ts 与 npm run typecheck；预期PASS、退出码0。同时执行C1程序化样片校验；不能以mock证明7157×3614，需从真实产物重新解码。

☐ Step 5: 限定提交。 git add apps/api/src/exports/service.ts apps/export-worker/package.json apps/export-worker/src/main.ts apps/export-worker/src/render-job.ts apps/export-worker/src/export.test.ts package-lock.json；git commit -m "feat(exports): render immutable entitled snapshots in durable workers"。

Task 4: 独立支付渠道adapter、订单与幂等收款确认

Files:

- Create: apps/api/src/payments/types.ts

- Create: apps/api/src/payments/adapters.ts

- Create: apps/api/src/payments/orders.ts

- Create: apps/api/src/payments/confirm.ts

- Create: apps/api/src/payments/payments.test.ts

Interfaces:

- Consumes: C3 Actor／授权／Database，任务1已发布SKU／payment配置，SQL orders/payment_attempts/payment_events；任务5 fulfillOrder(tx,orderId)事务函数。

- Produces: PaymentEvent={channel:"wechat_pay"|"wechat_virtual",eventId:string,transactionId:string,orderId:string,appId:string,merchantId:string,amountFen:number,currency:"CNY",state:"paid"|"closed"|"refunded",fulfillable:boolean}；PaymentAdapter={createPayment(orderId:string):Promise<{clientPayload:unknown}>,verifyEvent(rawBody:Uint8Array,headers:Record<string,string>):Promise<PaymentEvent>,query(orderId:string):Promise<PaymentEvent>,refund(orderId:string,refundId:string,amountFen:number):Promise<{providerRefundId:string,state:"submitted"|"succeeded"|"unknown"}>,normalizeStatus(raw:unknown):PaymentEvent}；createOrder(actor:Actor,wid:string,skuVersionId:string,channel:PaymentEvent["channel"],key:string):Promise<{orderId:string,amountFen:number,clientPayload:unknown}>；confirmPayment(event:PaymentEvent):Promise<{accepted:boolean,fulfillmentState:string}>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
  expect(grantsAfter100Callbacks).toHaveLength(1);
  expect(fulfillmentsAfterConcurrentQueryAndCallback).toHaveLength(1);
  await    expect(confirmPayment({...paidEvent,amountFen:paidEvent.amountFen-1})).rejects.toThrow('PAYMENT_MISMATCH');
  await    expect(confirmPayment({...paidEvent,appId:'wrong-app'})).rejects.toThrow('PAYMENT_MISMATCH');
  expect(orderAfterClientSuccessOnly.state).toBe('pending');
  expect(miniUnavailable.clientPayload).toBeUndefined();
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/payments/payments.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在types.ts定义PaymentEvent/Adapter，adapters.ts实现本地签名模拟与实际已批准渠道；orders.ts实现createOrder，confirm.ts实现confirmPayment。服务端读取SKU不可变价／期限／权益／退款规则和渠道商品映射，不信客户端金额；mini数字权益按真实虚拟支付权限、OS／版本capability独立启用，不伪造微信支付资格。验签在原始body上完成，校验商户／应用／商品／金额／币种／业务号，支付event和transaction双唯一；回调／查询共用加锁确认和履约事务。订单规范态pending/paid/closed/refund_pending/partially_refunded/refunded，fulfillmentState仅unfulfilled/fulfilled/reversed/manual_review；等待履约用paid＋unfulfilled，不持久化fulfilling。渠道若要求同步发货应答，必须履约提交后按官方合同ACK，不能通用先ACK；client success仅触发查单。5分钟未知进入查单队列，关单前查询，已扣款本地关单转异常补偿保留收款事实。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/payments/payments.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/payments/types.ts apps/api/src/payments/adapters.ts apps/api/src/payments/orders.ts apps/api/src/payments/confirm.ts apps/api/src/payments/payments.test.ts；git commit -m "feat(payments): confirm channel payments through idempotent orders"。

Task 5: 会员grants、批次点数与原子预占结算

Files:

- Create: apps/api/src/commerce/entitlements.ts

- Create: apps/api/src/commerce/credits.ts

- Create: apps/api/src/commerce/fulfillment.ts

- Create: apps/api/src/commerce/credits.test.ts

Interfaces:

- Consumes: 任务3 EntitlementSnapshot／EntitlementPort、C3 QuotaPolicyPort；任务4订单／PaymentEvent；SQL entitlement_grants/credit_lots/credit_reservations/credit_ledger。

- Produces: getEntitlements(workspaceId:string,at:string):Promise<EntitlementSnapshot>；readQuota(workspaceId:string):Promise<{limitBytes:number,maxProjectPhotos:number}>；fulfillOrder(tx:PoolClient,orderId:string):Promise<void>；CreditReservation={reservationId:string,workspaceId:string,jobId:string,points:number,state:"reserved"|"settled"|"released"}；reserveCredits(tx:PoolClient,wid:string,jobId:string,points:number):Promise<CreditReservation>、settleCredits(tx:PoolClient,reservationId:string,eventId:string):Promise<void>、releaseCredits(tx:PoolClient,reservationId:string,eventId:string):Promise<void>；creditBalance(wid:string,at:string):Promise<{available:number,reserved:number,frozen:number,expired:number}>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(acceptedExactlyEnoughRequests).toHaveLength(1);
expect(await creditBalance(wid,now)).toMatchObject({available:0,reserved:quotedPoints});
expect(settlementEntriesForRepeatedSuccess).toHaveLength(1);
expect(releaseTargetLotId).toBe(originalLotId);
expect(releasedExpiredLot.expiresAt).toBe(originalExpiry); // no extension
expect(nextSameTierGrant.startsAt).toBe(previousGrant.endsAt);
expect(otherGrantAfterRefund.revokedAt).toBeNull();
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/commerce/credits.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在entitlements.ts实现getEntitlements/readQuota并适配C3／导出端口；在fulfillment.ts实现fulfillOrder，以source_order＋grant_type唯一创建会员grant／赠点／购买lot，不直接覆盖用户会员标签。免费权益来自有效free配置，付费只按未撤销grants当前时间计算，不依赖定时任务。相同档续购start=max(now,current_end)，按duration_days真实天数；首期跨档购买禁用或受审计人工处理，不隐式折算。credits.ts定义Reservation并实现四函数，按到期时间／lot ID稳定顺序锁批次，非负约束、reserved/frozen隔离；预占／结算／释放／过期均追加流水。同reservation内部重试不再预占，结算和释放终局CAS只能一次；失败回原lot不延有效期，特殊补偿新建lot并关联工单。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/commerce/credits.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/commerce/entitlements.ts apps/api/src/commerce/credits.ts apps/api/src/commerce/fulfillment.ts apps/api/src/commerce/credits.test.ts；git commit -m "feat(commerce): grant memberships and maintain atomic credit lots"。

Task 6: 高熵兑换批次、限速与唯一权益来源

Files:

- Create: apps/api/src/commerce/redemption.ts

- Create: apps/api/src/commerce/redemption.test.ts

Interfaces:

- Consumes: C3授权／幂等；任务5 fulfillOrder所用同一grant/lot发放事务语义，任务1 SKU版本；SQL redemption_batches/codes/redemptions。

- Produces: RedemptionBatchInput={skuVersionId:string,startsAt:string,expiresAt:string,count:number,maxUsesPerCode:number,workspaceKinds:Array<"personal"|"studio">,distributionChannel:string}；createRedemptionBatch(actor:Actor,input:RedemptionBatchInput,reason:stri

ng):Promise<{batchId:string,oneTimeCodes:string[]}>；redeemCode(actor:Actor,wid:string,rawCode:string,key:string):Promise<{redemptionId:string,grantIds:string[]}>；disableRedemptionBatch(actor:Actor,batchId:string,reason:string):Promise<void>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(successfulConcurrentRedemptions).toHaveLength(1);
expect(await  redeemCode(actor,wid,code,sameKey)).toEqual(firstRedemption);
expect(rawCodeInDatabaseOrLogs).toBe(false);
expect(genericErrorForUsedByOther).toBe(genericErrorForUnknownCode);
expect(previouslyGrantedEntitlementsAfterDisable).toEqual(entitlementsBeforeDisable);
await  expect(redeemCode(viewer,wid,code,key)).rejects.toThrow('FORBIDDEN');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/commerce/redemption.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在redemption.ts实现三签名，使用crypto随机至少128bit不可枚举code，数据库只存带服务端pepper的不可逆hash，原码仅生成时受控导出一次，日志只掩码。首期每码maxUses=1，未来上限仍事务计数；锁code检查状态／时效／工作区种类，redemption唯一来源调用同一发放语义而非创建假支付订单。按用户／IP／设备信号限速且错误通用。批次停用只阻止新兑换，不撤销既有grant；追回单独受审计工单，角色／理由／操作摘要必须记录。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/commerce/redemption.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/commerce/redemption.ts apps/api/src/commerce/redemption.test.ts；git commit -m "feat(redemption): issue hashed codes with atomic entitlement claims"。

Task 7: 退款冻结、渠道回执与订单来源撤销

Files:

- Create: apps/api/src/commerce/refunds.ts

- Create: apps/api/src/commerce/refunds.test.ts

Interfaces:

- Consumes: 任务4 PaymentAdapter／规范事件；任务5 reservation／credit_lot／grants；任务2取消API。

- Produces: RefundState="requested"|"submitted"|"unknown"|"succeeded"|"failed"|"manual_review"；requestRefund(actor:Actor,wid:string,orderId:string,reason:string,key:string):Promise<{refundId:string,state:RefundState}>；processRefund(refundId:string,adapter:PaymentAdapter):Promise<RefundState>；confirmRefund(event:PaymentEvent,refundId:string):Promise<RefundState>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(balanceDuringRefund.frozen).toBe(refundablePoints);
await  expect(reserveCredits(tx,wid,newJobId,refundablePoints)).rejects.toThrow('INSUFFICIENT_CREDITS');
expect(revokedGrantIds).toEqual([grantFromRefundedOrder]);
expect(orderAfterRefund.paidAmountFen).toBe(originalPaidAmountFen);
expect(orderAfterRefund.refundedAmountFen).toBe(confirmedRefundAmountFen);
expect(forcedRefundAfterConsumption).toMatchObject({state:'manual_review',negativeBalance:false});
expect(refundEntriesAfterDuplicateCallback).toHaveLength(1);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/commerce/refunds.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在refunds.ts定义RefundState并实现三签名；API／SQL统一requested/submitted/unknown/succeeded/failed/manual_review；客户端以中文标签展示，不增加另一套业务状态。按购买时冻结退款政策受理，首期自动规则仅未消耗点数包／未使用会员权益；其他进入人工且不覆盖消费者／渠道强制退款要求。事务冻结来源order的未消耗lot／grant，活动预占任务先按规则取消或结算，不能挪其他订单余额。调用refund使用稳定refundId；未知回执查询原退款，不重复调用制造二次退款。确认退款保留付款历史与累计金额，撤销只限来源grant、未耗lot；强制退款但已消费记录平台财务损失／复核，余额不改负数。退款失败按终局释放冻结，迟到成功仍走唯一事件与审计；注销账户受限保留处理，不丢订单事实。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/commerce/refunds.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/commerce/refunds.ts apps/api/src/commerce/refunds.test.ts；git commit -m "feat(refunds): freeze and reverse source entitlements safely"。

Task 8: 商业UI、任务中心、管理对账与差异复核

Files:

- Create: apps/web/src/commerce/MembershipPage.tsx

- Create: apps/web/src/commerce/OrderPage.tsx

- Create: apps/web/src/jobs/TaskCenter.tsx

- Create: apps/admin/src/commerce/CommerceConsole.tsx

- Create: apps/api/src/commerce/reconciliation.ts

- Create: apps/api/src/commerce/reconciliation.test.ts

- Create: tests/browser/commerce.spec.ts

Interfaces:

- Consumes: 任务1 – 7服务、C3当前工作区／用户；渠道只读账单adapter。

- Produces: StatementLine={channel:string,transactionId:string,orderId:string,paidFen:number,refundedFen:number}；reconcileStatement(businessDate:string,lines:StatementLine[]):Promise<{matched:number,reviewIds:string[]}>；MembershipPage(props:{workspaceId:string}):React.JSX.Element；TaskCenter(props:{workspaceId:string}):React.JSX.Element；CommerceConsole():React.JSX.Element。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(report.reviewIds).toHaveLength(1);  //  provider  paid,  local  closed
expect(grantCountAfterRepeatedReconciliation).toBe(grantCountBeforeReconciliation);
expect(financeViewerCanSeeRawPhoto).toBe(false);
expect(visiblePriceWithoutPublishedSku).toBeNull();
expect(frontendSuccessWithoutConfirmation).toBe('待确认');
expect(manualAdjustmentAudit).toMatchObject({reason:'工单复核',ticketId:expect.any(String)});
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/commerce/reconciliation.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在reconciliation.ts实现reconcileStatement，以Asia/Shanghai businessDate核对渠道收款／退款和本地履约，每日差异建唯一复核记录，禁止靠重复补发修正。MembershipPage显示免费／个人／影楼／点数包真实已发布价格、天数、范围、工作区、点数有效期／退款规则；未配价显示未上架，未开通mini渠道不出可用购买按钮。OrderPage查询服务端状态、订单／退款／兑换记录，TaskCenter显示排队／处理／打包／确认中／完成／失败／取消与jobId、可追溯重试，不伪装浏览器任务。CommerceConsole含SKU版本、兑换批次受控下载、订单查询、差异复核、额度／会员人工调整；财务修改要角色、理由、工单、前后摘要，退款／返还双人复核策略可配置，客服不看密钥／支付token／原片。任务进度和通知共享outbox事实。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/commerce/reconciliation.test.ts 与 npm run typecheck；预期PASS、退出码0。另运行npm run test:browser -- tests/browser/commerce.spec.ts，预期PASS；用模拟签名渠道走购买→回调100次→会员→导出→退款→对账的真实本地数据库闭环；真实商户联调独立待接入。

☐ Step 5: 限定提交。 git add apps/web/src/commerce/MembershipPage.tsx apps/web/src/commerce/OrderPage.tsx apps/web/src/jobs/TaskCenter.tsx apps/admin/src/commerce/CommerceConsole.tsx apps/api/src/commerce/reconciliation.ts apps/api/src/commerce/reconciliation.test.ts tests/browser/commerce.spec.ts；git commit -m "feat(commerce): expose truthful purchases tasks and reconciliation"。

自审与执行交接

8项覆盖CFG-CENTER/BILLING/EXPORT/OPS及所有配置注册、EXP-01/02、PAY-01、LIC-01、SHARE下载产物、ADM-01财务权限与账本。RF1→1、RF2→2/3、RF3→4/5、RF4→5/6/7、RF5→3/7/8均有明确断言。每任务拥有五个步骤；任务3使用本项EntitlementPort解耦任务5，任务4与5的收款／发放在同一数据库事务调用，实施可先完成5的独立账本签名再连4，不引入第二账本。C5报价／预算只能调用reserve/settle/release而不能直接改余额。所有DTO引用OpenAPI，JobState／Order／Refund／channel与SQL使用上述同一规范枚举，progress对外0 – 100。

当前覆盖缺口：无首期功能缺项。价格／资质／渠道ACK／退款政策要经营者接入时核验，当前可以用标明测试SKU与本地签名渠道完成状态机和竞态；不能给测试SKU伪造对外价格。部署备份恢复／告警属于上线手册与OPS合同，本模块实现其监控数据与可恢复事实源；真实生产上线另外验收。本轮只编写计划，不执行支付／付费供应商请求。

C5 AI创作、模型内容版本与运营后台 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (☐) syntax for tracking.

Goal: 交付有许可和用途确认的AI艺术素材，从能力验证／报价预算到异步生成、审核标识、私有入库、可编辑标题入册与运营后台。

Architecture: 供应商能力与模型／路由／预设均不可变版本化，报价固定输入和费用合同。接受任务在C4同一数据库事务预占点数／预算并入队，输出先隔离校验和审核标识再入素材库，成功终局才结算；网页和mini只使用已发布能力。

Tech Stack: TypeScript、C3 API与PostgreSQL、C4 jobs／credits／config、Node隔离下载、供应商adapter、React web/admin、Vitest／Playwright；不采用非商业Zine Skill。

Spec: v0.3 § § 2／7、AI服务配置、配置验收、PRD GEN-01/02、provider合同。

Global Constraints

- 本文是待执行研发计划；测试片段和预期结果均不是当前已通过记录。只写本模块实现；真实微信、存储、支付、AI集成缺凭据时保持 unconfigured，本地模拟／契约测试可完整实施。

- 继承 共同合同：公开ID为1 – 128字符不透明字符串；SHA-256为64位小写十六进制；金额为人民币分整数、点数整数，JSON整数上限9007199254740991；UTC存储，展示和预算日为Asia/Shanghai。

- API根 /v1；成功 {data,requestId}，失败 {error:{code,message,retryable,details},requestId}；列表 {items,nextCursor,hasMore}。网页cookie＋CSRF，小程序短期bearer；所有资源逐次校验工作区权限。

- 可重放创建／财务／任务操作使用空间范围唯一的 Idempotency-Key，保留7天；相同key不同payload返回409。文档If-Match revision为字符串；草稿／目录更新有expectedRevision／If-Match，冲突409。

- 唯一产品文档模型由C1 @album/document提供，对应 document.schema.json；schemaVersion1、px96、sRGB；1 – 50内页跨页、500照片、100递归图层／跨页、组深度≤5。原片JPEG／PNG≤25_000_000字节且≤80MP；不接受任意外部URL素材引用。

- 平台统一存储／AI连接，不开放BYOS／BYOK；免费与付费能力都实现，真实价格／配额未经确认不得伪造。影楼首期单使用者，预留owner/admin/designer/viewer与seat_limit，不做邀请UI；App仅预留共享API。

- 测试命令沿用C1 npm run test:unit -- <file>／npm run test:browser -- <file>、npm run typecheck。集成测试使用临时PostgreSQL／Redis和本地受控模拟，禁止生产数据库；精确依赖版本在实施时锁入package-lock.json，Node 24 LTS。

- 规则排版／文字编辑不消耗AI生图点数；textSuggestion为首期RESERVED/OFF，路由不能启用、公开图片GenerationInput不能报价文字；未来需独立输入／输出／报价／结算合同，不作为整本成册依赖。AI初期每个计费子任务1张图，imageCount=1，不承诺人脸完全一致或seed逐像素复现。

- 初始原创／明确授权预设至少山水水墨、人像淡彩、花鸟写意3个；非商业、许可不明、过期、依赖未验证均不可上架；会员去品牌水印不移除AI来源标识。

- provider默认连接超时5000ms、单次请求60000ms；异步总时长不等于HTTP超时。并发／RPM取平台、供应商、工作区最严；dailyBudgetFen=0停止新付费任务。

- 预算按稳定accountBillingId、环境、Asia/Shanghai预算日预占＋已发生成本；同采购账户多个连接共享账户上限并叠加各自上限。80%告警，100%停新任务，跨日保留旧日未终局预占。

- 报价默认5分钟有效，未接受不预占；冻结configReleaseId、route/model/preset/costRule版本、inputHash、引用授权及policy。成功且审核通过结算一次，明确失败释放一次；状态未知先查询，建议30分钟终局，迟到不补扣／发布。

- 参考图只传用户选中且授权的图，默认剥离GPS／原文件名，不训练／公开复用；结果服务端受控拉取，quarantine→审核／显隐标识→ready；不把供应商临时URL当永久asset引用。

Review Focus

- RF1：“ 聊天兼容 ”冒充图生图、未知参数和供应商超时已接受不得静默忽略或重复提交；任务1／4有断言。

- RF2：许可证过期、已停用模型、配置／密钥轮换不能改变已报价任务或让无权预设上架；任务2／3／7有断言。

- RF3：10并发仅够一次预算、共享采购账户连接与北京时间午夜不能穿透预算／清零预占；任务3有断言。

- RF4：恶意URL／DNS重绑定／坏图、审核或标识失败、迟到结果不能成为ready素材或结算成功；任务4／5有断言。

- RF5：回到相册定位、重复插入、改标题和跨端结果恢复不能覆盖原片／重复扣点／丢未知层；任务6有断言。

文件与合同边界

下列产品路径均为未来实施时创建／修改，当前不生成产品代码。API只实现 OpenAPI 已定义操作与参数；DTO从合同生成到packages/contracts/src/api.ts（C3任务1），不自行维护第二套HTTP字段。五个schema使用固定文件名与各自$defs.root。数据库执行 001_initial.up.sql 与对应down迁移，并遵守 transaction-safety.md；发现合同和本计划有差异先修订合同并加反例，再实现。应用内部类型由本计划拥有它的任务定义并导出，测试数据在相应测试文件内构造，不依赖未定义的未来测试帮助函数。

文件与服务分工

packages/providers只做供应商协议与能力合同；apps/api/src/ai/catalog.ts拥有模型／路由／预设／许可版本；quotes.ts与budget.ts负责输入冻结和平台采购预占；apps/ai-worker按C4fence执行；output.ts隔离下载审核标识并注册C3素材。apps/admin提供分权配置与测试证据，不能在客户端持有平台秘密或通过手改job状态绕过账本。DTO复用C3生成的components，不定义第二种GenerationQuote；供应商Input与用户GenerationInput是两个明确边界。

Task 1: 供应商能力合同、受控端点与可重放适配器

Files:

- Create: packages/providers/package.json

- Create: packages/providers/src/types.ts

- Create: packages/providers/src/registry.ts

- Create: packages/providers/src/fake-adapter.ts

- Create: packages/providers/src/index.ts

- Create: packages/providers/src/provider.test.ts

- Modify: package-lock.json

Interfaces:

- Consumes: provider.schema.json全部定义；C4 SecretPort／assertApprovedHttps(url,allowedHosts)／fetchApprovedHttps(url,init,allowedHosts,limits)。

- Produces: ProviderCapability="textToImage"|"imageToImage"|"textSuggestion"|"asyncQuery"|"cancel"|"idempotentSubmit"；ProviderInput={operationId:string,modelVersionId:string,prompt:string,negativePrompt?:string,width:number,height:number,count:1,referenceObjectIds?:string[],referenceStrength?:number,seed?:number,consentRef?:string}；SubmitResult={state:"accepted",providerRequestId:string,acceptedAt:string}|{state:"rejected",code:string,message:string}|{state:"unknown",operationId:string,retryAfterSeconds:number}；QueryResult精确对应provider.schema.json/$defs/queryResult，由本任务生成到types.ts；ProviderContext={connectionVersionId:string,secretRef:string,secretVersion:string,endpoint:string,allowedResultHosts:string[]}；ProviderAdapter={id:string,version:string,capabilities:ReadonlySet<ProviderCapability>,submit(input:ProviderInput,context:ProviderContext):Promise<SubmitResult>,query(identity:{operationId:string,providerRequestId:string|null},context:ProviderContext):Promise<QueryResult>,cancel(identity:{operationId:string,providerRequestId:string|null},context:ProviderContext):Promise<QueryResult>}；TextSuggestionInput={operationId:string,modelVersionId:string,prompt:string,maxCharacters:number}；TextSuggestionResult={state:"succeeded",text:string,costFen:number}|{state:"rejected",code:string}|{state:"unknown",operationId:string}；TextSuggestionAdapter={suggest(input:TextSuggestionInput,context:ProviderContext):Promise<TextSuggestionResult>}；getProviderAdapter(id:string,version:string):ProviderAdapter。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(chatOnly.capabilities.has('imageToImage')).toBe(false);
await expect(adapter.submit(unsupportedSeedInput,context)).rejects.toThrow('UNSUPPORTED_PARAMETER');
expect(submitTimeoutAfterAcceptance.state).toBe('unknown');
expect((await adapter.query(originalIdentity,context)).state).toBe('succeeded');
expect(providerPaidSubmitCount).toBe(1);
await expect(adapter.submit(input,privateEndpointContext)).rejects.toThrow('ENDPOINT_FORBIDDEN');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- packages/providers/src/provider.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在types.ts定义上述类型及schema精确QueryResult，registry.ts按id/version注册已实现adapter；fake-adapter.ts提供同步归一到accepted→query、异步、429、拒绝、认证失败、已接受后超时、重复回调／取消竞态／坏结果场景。adapter只能映射白名单参数；unsupported拒绝，不丢字段。默认5秒连接／60秒单请求受配置，所有网络通过fetchApprovedHttps逐次DNS／重定向和固定连接地址验证，秘密只发批准host。query支持operationId定位未知submit，不支持查询的非幂等供应商只能人工复核，不能伪造幂等。textSuggestion为独立预留类型，首期RESERVED/OFF；publishModel可保留能力元数据，但路由发布与公开报价拒绝启用textSuggestion，不能把文字结果伪装成图片QueryResult，不成为成册强依赖。真实adapter只在后续选择供应商后实现同一合同，模型发现能力为false时手动登记，不声称自动发现。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- packages/providers/src/provider.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add packages/providers/package.json packages/providers/src/types.ts packages/providers/src/registry.ts packages/providers/src/fake-adapter.ts packages/providers/src/index.ts packages/providers/src/provider.test.ts package-lock.json；git commit -m "feat(providers): define capability-aware safe generation adapters"。

Task 2: 不可变模型、路由、预设与商业许可发布

Files:

- Create: apps/api/src/ai/catalog-types.ts

- Create: apps/api/src/ai/catalog.ts

- Create: apps/api/src/content/versioned-assets.ts

- Create: apps/api/src/ai/catalog.test.ts

Interfaces:

- Consumes: 任务1 ProviderCapability与registry；C4ConfigPayload／release，C1 TemplateVersion／FilterRecipe／FontRecord；SQL ai_model_versions/ai_route_versions/ai_preset_versions/template_versions/font_versions/filter_versions。

- Produces: LicenseRecord={id:string,version:number,source:string,author:string,termsVersion:string,commercialAllowed:boolean,allowReference:boolean,allowModification:boolean,allowRedistribution:boolean,attribution:string|null,expiresAt:string|null,territories:string[]}；ModelVersion={id:string,version:number,connectionVersionId:string,providerModelId:string,providerModelVersion:string,capabilities:ProviderCapability[],parameterSchema:unknown,licenseRef:{id:string,version:number},testReportId:string,state:"unverified"|"ready"|"disabled"|"retired"}；PresetVersion={id:string,version:number,name:string,category:string,modelVersionIds:string[],parameterTemplate:Record<string,string|number|boolean>,editableParameters:string[],licenseRef:{id:string,version:number},exampleAssetIds:string[],pricingVersionId:string,moderationPolicyVersion:string,labelPolicyVersion:string,state:"draft"|"published"|"retired"}；publishModel(actor:Actor,model:ModelVersion):Promise<void>、publishPreset(actor:Actor,preset:PresetVersion):Promise<void>；publishContentVersion(actor:Actor,kind:"template"|"font"|"filter",id:string,version:number,license:LicenseRecord,reason:string):Promise<void>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
await expect(publishPreset(contentActor,nonCommercial)).rejects.toThrow('LICENSE_NOT_COMMERCIAL');
await expect(publishPreset(contentActor,expired)).rejects.toThrow('LICENSE_EXPIRED');
await expect(publishModel(opsActor,authOnlyUntested)).rejects.toThrow('MODEL_UNVERIFIED');
expect(initialPresetNames).toEqual(expect.arrayContaining(['山水水墨','人像淡彩','花鸟写意']));
expect(acceptedJob.modelVersionId).toBe(oldModelVersionId); // new route does not mutate it
await expect(publishContentVersion(financeActor,'font',id,1,license,reason)).rejects.toThrow('FORBIDDEN');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/ai/catalog.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在catalog-types.ts定义上述版本／LicenseRecord，在catalog.ts实现publishModel/publishPreset及能力路由版本引用；本任务定义路由RouteVersion={id:string,version:number,capability:ProviderCapability,primaryModelVersionId:string,fallbackModelVersionIds:string[]}并导出，quote消费固定版本。参数schema只声明adapter支持的尺寸、长度、格式、参考张数与seed等，校验后发布。预设至少三类，示例只原创／明确覆盖商用用途授权，不能复制受限Zine；用途许可／地域／有效期／署名均可追溯。versioned-assets.ts统一管理模板／字体／33滤镜版本许可及hash，公开版本不可覆盖；停用只影响新请求，旧作品保留版本和适用历史条款。字段插值只白名单变量，不执行模型代码或任意表达式。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/ai/catalog.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/ai/catalog-types.ts apps/api/src/ai/catalog.ts apps/api/src/content/versioned-assets.ts apps/api/src/ai/catalog.test.ts；git commit -m "feat(content): publish licensed versioned models presets and assets"。

Task 3: 五分钟报价、共享账户预算与接受事务

Files:

- Create: apps/api/src/ai/quotes.ts

- Create: apps/api/src/ai/budget.ts

- Create: apps/api/src/ai/quotes.test.ts

Interfaces:

- Consumes: C3 components["schemas"]["GenerationInput"]／GenerationQuote／GenerationJobCreate、授权／资源；任务2版本，C4 reserveCredits(tx,wid,jobId,points)／createJob／loadRelease。

- Produces: GenerationInput=components["schemas"]["GenerationInput"]、GenerationQuote=components["schemas"]["GenerationQuote"]；BudgetReservation={id:string,accountBillingId:string,connectionVersionId:string,businessDate:string,estimatedCostFen:number,state:"reserved"|"settled"|"released"}；createGenerationQuote(actor:Actor,input:GenerationInput):Promise<GenerationQuote>；acceptGenerationQuote(actor:Actor,input:components["schemas"]["GenerationJobCreate"],key:string):Promise<JobRecord>；reserveProviderBudget(tx:PoolClient,accountBillingId:string,connectionVersionId:string,estimatedCostFen:number,jobId:string,at:string):Promise<BudgetReservation>、closeProviderBudget(tx:PoolClient,reservationId:string,actualCostFen:number,state:"settled"|"released"):Promise<void>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(creditsReservedBeforeQuoteAcceptance).toBe(0);
expect(quote.expiresAt).toBe('2026-10-21T08:05:00.000Z');  //  injected  clock  08:00  UTC
expect(acceptedUnderTenConcurrentRequests).toHaveLength(1);  //  shared  budget  fits  one
expect(oldDayReservationAfterShanghaiMidnight.state).toBe('reserved');
expect(secondConnectionBudgetAvailable).toBe(0);  //  same  accountBillingId
await  expect(acceptGenerationQuote(actor,changedInputHash,key)).rejects.toThrow('QUOTE_INPUT_MISMATCH');
expect(JSON.stringify(quote)).not.toContain('estimatedCostFen');
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/ai/quotes.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在quotes.ts实现两个报价签名，在budget.ts实现reserve/close；报价校验model/preset许可与能力、白名单参数／尺寸、引用图当前权限／ready／用途consent，固定全部版本、normalizedInput/hash、credits和5分钟expiry，采购成本只存服务端。输入改变必须新报价；接受事务锁quote、检查未接受／未过期／配置仍安全、引用授权与预算，创建稳定jobId、同事务预占用户点数及account＋connection预算并写job/outbox。预算行按稳定accountBillingId与北京时间日上锁，共享采购账户跨连接／密钥版本不重置；连接上限再叠加，0停新付费，80%事件告警／100%拒绝，跨日未终局仍在原日账。query若计费单独记cost ledger；actualCost超预估平台承担并触发复核，不补扣用户点数。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/ai/quotes.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/ai/quotes.ts apps/api/src/ai/budget.ts apps/api/src/ai/quotes.test.ts；git commit -m "feat(ai): freeze quotes and reserve credits with shared provider budgets"。

Task 4: 异步生成worker、未知上游查询与终局竞争

Files:

- Create: apps/ai-worker/package.json

- Create: apps/ai-worker/src/main.ts

- Create: apps/ai-worker/src/generate-job.ts

- Create: apps/ai-worker/src/provider-events.ts

- Create: apps/ai-worker/src/generate.test.ts

- Modify: package-lock.json

Interfaces:

- Consumes: 任务1 ProviderAdapter，任务3quote／budget；C4 JobLease／renewLease／finishJob／settleCredits／releaseCredits，C3通知outbox。

- Produces: runGenerationJob(lease:JobLease,adapter:ProviderAdapter,signal:AbortSignal):Promise<void>；processProviderEvent(adapterId:string,rawBody:Uint8Array,headers:Record<string,string>):Promise<{accepted:boolean}>；reconcileUnknownGeneration(jobId:string,now:string):Promise<"query_scheduled"|"failed_released"|"already_terminal">。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(paidSubmitCountAfterTimeoutAndNewDefaultModel).toBe(1);
expect(queriedConnectionVersionId).toBe(frozenConnectionVersionId);
expect(reservationAfterUnknown.state).toBe('reserved');
expect(await    reconcileUnknownGeneration(jobId,after30Minutes)).toBe('failed_released');
expect(creditsAfterLateSuccess).toEqual(creditsAfterTerminalTimeout);
expect(publishedOutputsAfterCancelWins).toHaveLength(0);
expect(terminalLedgerEvents).toHaveLength(1);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/ai-worker/src/generate.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在generate-job.ts实现run/reconcile，在provider-events.ts实现adapter专用验签与事件去重。持久化提交意图／operationId后才网络提交；accepted记录providerRequestId；unknown记录running＋providerSubmissionState=unknown＋stage=confirming_upstream并查询原operation/provider ID，禁止切供应商重新付费。自动fallback只在明确未接受且满足已报价能力／质量／费用合同，其他重报价重新确认。所有查询用冻结连接／模型／secretVersion，即使后台发布新默认；密钥紧急撤销进入查询／人工终结流程，不当重提理由。取消先请求adapter，终局CAS赢家决定一次settle/release；超过冻结终局期限默认30分钟fail+release，迟到输出记录采购成本但不发布、不再扣点。worker崩溃重启从PG提交状态恢复，不从“没有内存结果”推断未提交。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/ai-worker/src/generate.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/ai-worker/package.json apps/ai-worker/src/main.ts apps/ai-worker/src/generate-job.ts apps/ai-worker/src/provider-events.ts apps/ai-worker/src/generate.test.ts package-lock.json；git commit -m "feat(ai-worker): reconcile unknown provider work without duplicate charges"。

Task 5: 隔离输出、审核申诉、来源链与强制标识

Files:

- Create: apps/api/src/ai/output.ts

- Create: apps/api/src/ai/moderation.ts

- Create: apps/api/src/ai/provenance.ts

- Create: packages/document/src/generated-provenance.ts

- Create: apps/api/src/ai/output.test.ts

- Modify: packages/export/src/preflight.ts

Interfaces:

- Consumes: 任务1QueryResult／ProviderContext；C3 ObjectStore／AssetRecord／ResourcePort；C4fence／credits，C1导出预检；任务2moderation/label政策版本。

- Produces: ModerationDecision={state:"approved"|"rejected"|"review",reasonCode:string,policyVersion:string,reviewId:string|null}；ModerationAdapter={review(bytes:Uint8Array,policyVersion:string):Promise<ModerationDecision>}；GenerationProvenance={jobId:string,modelVersionId:string,presetVersionId:string,inputHash:string,referenceAssetIds:string[],consentRef:string|null,licenseRef:string,labelPolicyVersion:string,moderationPolicyVersion:string,outputSha256:string}；ingestGenerationOutput(lease:JobLease,result:Extract<QueryResult,{state:"succeeded"}>,context:ProviderContext,moderator:ModerationAdapter):Promise<{assetId:string,state:"ready"|"rejected"|"review"}>；validateGeneratedProvenance(doc:DocumentModel,manifest:ResourceManifest):Promise<DocumentIssue[]>；submitModerationAppeal(actor:Actor,wid:string,jobId:string,reason:string):Promise<{reviewId:string}>。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
  await    expect(ingestGenerationOutput(lease,privateRedirectResult,context,moderator)).rejects.toThrow('RESULT_URL_FORBIDDEN');
  expect(assetStateForInvalidMagic).toBe('rejected');
  expect(assetStateForMissingLabelPolicy).not.toBe('ready');
  expect(creditsForRejectedOutput.state).toBe('released');
  expect(preflightForRemovedRequiredLabel.blockers.map(i=>i.code)).toContain('AI_LABEL_MISSING');
  expect(publicRejectedResponse).not.toHaveProperty('imageBytes');
  expect(appeal).toMatchObject({reviewId:expect.any(String)});
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/ai/output.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在output.ts实现ingestGenerationOutput：通过C4 fetchApprovedHttps限定批准结果host并逐跳DNS／HTTPS验证、固定连接地址及流式字节上限／超时、magic／实际解码像素／hash；超出模型限制即reject，先写quarantine私有对象，不用临时URL作assetId。moderation.ts实现adapter及申诉记录；审核／适用标识政策未配置或失败时不ready不结算成功，明确拒绝释放一次，人工review保持限时可终结状态。packages/document/src/generated-provenance.ts定义Interfaces中的GenerationProvenance纯JSON类型，API provenance.ts导入并re-export；provenance.ts定义来源链校验显式视觉与可追溯元数据标识；导出preflight调用validateGeneratedProvenance，移除必要标识阻断，会员去品牌水印不移AI标识。审核通过、标识处理成功、私有素材写入+hash验证后以同fence事务ready与settle，失败补偿清隔离对象；迟到或失租只记审计不发布。申诉只记录复核，不允许管理员直接把job改成功跳账本。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/ai/output.test.ts 与 npm run typecheck；预期PASS、退出码0。

☐ Step 5: 限定提交。 git add apps/api/src/ai/output.ts apps/api/src/ai/moderation.ts apps/api/src/ai/provenance.ts packages/document/src/generated-provenance.ts apps/api/src/ai/output.test.ts packages/export/src/preflight.ts；git commit -m "feat(ai-output): quarantine moderate and label generated private assets"。

Task 6: 艺术创作入口、结果素材与独立可编辑文字

Files:

- Create: apps/web/src/ai/ArtStudio.tsx

- Create: apps/web/src/ai/GenerationResult.tsx

- Create: packages/document/src/generated-insertion.ts

- Create: packages/document/src/generated-insertion.test.ts

- Create: tests/browser/art-to-album.spec.ts

- Modify: apps/web/src/editor/EditorPage.tsx

- Modify: apps/mini/pages/album/index.ts

- Modify: apps/mini/pages/album/index.wxml

Interfaces:

- Consumes: 任务2已发布preset列表、任务3quote、任务4任务中心、任务5ready素材／provenance；C1 Layer／DocumentCommand、C3 MiniPatch／云revision。

- Produces: GenerationReturnTarget={projectId:string,spreadId:string,baseRevision:number}；ArtStudio(props:{workspaceId:string,returnTarget:GenerationReturnTarget|null}):React.JSX.Element；createGeneratedInsertionCommand(doc:DocumentModel,input:{assetId:string,spreadId:string,provenance:GenerationProvenance,title:string|null,dateText:string|null},idFactory:()=>string):DocumentCommand；GenerationResult(props:{jobId:string,onInsert:(assetId:string)=>void}):React.JSX.Element。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(originalPhotoRecordAfterInsertion).toEqual(originalPhotoRecordBeforeInsertion);
expect(newLayers.map(l=>l.type)).toEqual(['image','text','text']);
expect(chargeCountAfterTitleEdit).toBe(chargeCountBeforeTitleEdit);
expect(advancedExistingLayerAfterInsertion).toEqual(advancedExistingLayerBeforeInsertion);
expect(history.undo()).toEqual(beforeInsertion);
expect(currentSpreadAfterReturn).toBe(savedReturnSpreadId);
expect(repeatedInsertWithSameCommandId.layerCount).toBe(firstInsertion.layerCount);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- packages/document/src/generated-insertion.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在generated-insertion.ts实现签名但不让@album/document依赖服务端模块：GenerationProvenance从任务5创建的packages/document/src/generated-provenance.ts导入（仅JSON类型，无网络）。插入ready asset为一图片层，title/dateText独立text层，保留其他层与原片，一次可撤销命令、稳定commandId防网络重放；禁止把AI图伪装成已拆分可编辑图层。ArtStudio区分照片忠实排版／艺术改写，显示不承诺人脸一致、实际用途／区域／保留政策和选中参考图授权，提交前明确点数与有效期。只显示支持参数，切风格新报价；结果可加入指定文件夹或当前跨页，不自动覆盖旧结果。返回保存的project/spread，revision过期提示选择目标且不覆盖；任务跨端可找回。mini只允许模板允许的插入槽位／文字字段，完整自由插入引导网页继续，实际生成任务仍统一报价／任务API。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- packages/document/src/generated-insertion.test.ts 与 npm run typecheck；预期PASS、退出码0。另执行npm run test:browser -- tests/browser/art-to-album.spec.ts，预期PASS；确认文字修改没有generation quote/job网络请求。

☐ Step 5: 限定提交。 git add apps/web/src/ai/ArtStudio.tsx apps/web/src/ai/GenerationResult.tsx packages/document/src/generated-insertion.ts packages/document/src/generated-insertion.test.ts tests/browser/art-to-album.spec.ts apps/web/src/editor/EditorPage.tsx apps/mini/pages/album/index.ts apps/mini/pages/album/index.wxml；git commit -m "feat(ai-ui): insert generated art with separately editable text"。

Task 7: 分权运营后台、真实样图测试与成本监控

Files:

- Create: apps/admin/package.json

- Create: apps/admin/src/main.tsx

- Create: apps/admin/src/config/ConfigOverview.tsx

- Create: apps/admin/src/ai/ProviderConsole.tsx

- Create: apps/admin/src/content/ContentConsole.tsx

- Create: apps/api/src/ai/config-tests.ts

- Create: apps/api/src/ai/config-tests.test.ts

- Create: tests/browser/ai-admin.spec.ts

- Modify: package-lock.json

Interfaces:

- Consumes: C4草稿／测试／发布／审计／SecretPort与预算，任务1 – 5能力／许可／样图输出；C3后台actor权限。

- Produces: ConfigTestReport=components["schemas"]["ConfigTest"]；runAiConfigTest(actor:Actor,draftId:string,input:components["schemas"]["ConfigTestCreate"]):Promise<ConfigTestReport>（level为schema/connection/storage_roundtrip/capability/sample；sample必须有paidTestConfirmation，预算从服务器平台测试账户读取）；ConfigOverview():React.JSX.Element、ProviderConsole():React.JSX.Element、ContentConsole():React.JSX.Element。

☐ Step 1: 写失败断言。 测试文件内构造下述输入、受控时钟和端口fake；并发／SQL行为在临时PostgreSQL执行，不用只返回预定结果的mock证明事务。

```
expect(authSuccessReport.verifiedChecks).not.toContain('imageToImage');  //  network/auth  alone  is  not  inference
await  expect(runAiConfigTest(financeActor,draftId,sampleInput)).rejects.toThrow('FORBIDDEN');
await  expect(runAiConfigTest(opsActor,draftId,unconfirmedSample)).rejects.toThrow('TEST_COST_CONFIRMATION_REQUIRED');
expect(customerCreditBalanceAfterSample).toBe(customerCreditBalanceBeforeSample);
expect(report).toMatchObject({configHash:frozenDraftHash,credentialVersionIds:[frozenSecretVersion]});
expect(platformTestCost.actualCostFen).toBeGreaterThanOrEqual(0);
expect(testAfterCredentialRotation.state).toBe('expired');  //  immutable  old  result  is  not  rewritten
expect(logContainsFullPromptOrKey).toBe(false);
```

☐ Step 2: 运行失败用例。 npm run test:unit -- apps/api/src/ai/config-tests.test.ts；预期FAIL，错误指向本项缺失的接口或行为断言；依赖／测试配置错误须先排除，不能算有效红灯。

☐ Step 3: 实现本项接口。 在config-tests.ts实现runAiConfigTest，schema／connection／storage_roundtrip／capability／sample五个合同level分开，UI可按连接／能力／样图分组但不改HTTP枚举，hash＋秘密版本绑定且5分钟失效；样图先显示估算费用并确认，从平台独立testBudget预占，不扣客户点数，使用自制描述／授权测试图。采集实际模型／参数／release／耗时／providerRequestId／输出hash／采购成本，失败保留脱敏原因；契约未通过保持unverified。admin页面含20业务域及albumDefaults公开摘要、未配置项／负责人／有效版本／最近测试；AI连接／模型／路由／预设／报价／预算／用量／成本／轮换，内容模板／字体／滤镜／示例许可与审计全部可操作。财务改价格不改秘密、内容改许可不发金融权益，发布复验登录，误权限拒绝。80%/100%预算、未知上游、审核失败和队列延迟以已验证告警目的地发送运营事件；不把内部endpoint／采购成本暴露客户。

☐ Step 4: 验证通过。 重跑 npm run test:unit -- apps/api/src/ai/config-tests.test.ts 与 npm run typecheck；预期PASS、退出码0。另运行 npm run test:browser -- tests/browser/ai-admin.spec.ts与build，预期PASS；真实样图必须经营者确认实际供应商、费用和用途后才执行，本地合同失败／成功回放不需要真实key。

☐ Step 5: 限定提交。 git add apps/admin/package.json apps/admin/src/main.tsx apps/admin/src/config/ConfigOverview.tsx apps/admin/src/ai/ProviderConsole.tsx apps/admin/src/content/ContentConsole.tsx apps/api/src/ai/config-tests.ts apps/api/src/ai/config-tests.test.ts tests/browser/ai-admin.spec.ts package-lock.json；git commit -m "feat(admin): manage AI releases evidence licenses and test budgets"。

自审与执行交接

7项覆盖GEN-01/02、V3-MODE-01、V3-ART-01、V3-LICENSE-01、CFG-AI/CONTENT/CENTER/OPS和ADM-01。任务1解决供应商差异，2解决版权／模型内容版本，3解决报价预算，4解决未知状态／终局，5解决隔离／审核申诉／标识，6完成跨路径用户体验，7完成运营及测试费用。RF1→1/4、RF2→2/3/7、RF3→3、RF4→4/5、RF5→6有明确断言。Task5的来源JSON类型在共享document拥有，API只re-export，避免document反向依赖API；GenerationInput／Quote使用生成DTO，用户模型和provider模型不混用。所有任务均有独立功能与五个步骤，没有泛化测试尾项。

当前覆盖缺口：无首期功能缺项。真实模型、商业示例、许可证、适用审核／标识法规及采购价格在接入时核验；尚未确认的能力保持unconfigured／unverified，示例不伪造商用授权。可先完成全部本地可重放测试、UI和状态机；本轮只交付研发计划，不调用付费生成，不逐模块等待文档批准。

C6 全模块验收与运维发布 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (☐) syntax for tracking.

Goal: 为C1 – C5建立可重复构建、集成验证、恢复演练和受控发布证据，使真实上线条件可核对。

Architecture: 重用C3 API、C4任务／账本及C5供应商契约，不另建业务实现。环境、秘密、镜像、数据库与配置版本共同构成release manifest；先隔离预发，再完成平台／经营者所需真实接入，最后灰度发布。

Tech Stack: Node24 LTS、npm lockfile、Playwright固定Chromium、PostgreSQL16/17目标、Redis/BullMQ、Docker/Compose集成、云秘密服务、指标与日志。

Spec: 统一基线、运行手册、测试策略、环境schema。

Global Constraints

- 用户已明确全程AI开发；此为待执行计划，不是部署记录。dev/staging/prod隔离，不将生产照片复制到测试。

- 公开ID1 – 128字符不透明字符串；金额人民币分、点数整数；数据库UTC，预算日Asia/Shanghai。

- 1 – 50内页跨页、500照片／项目、100递归图层／面、源图25_000_000字节／80MP、输出面50MP，容量需实测。

- 固定Chromium／字体／滤镜／模板及renderer hash；缺原片不回退缩略图；缺字体／授权／文字超框阻断导出。

- 生产镜像须实际构建并锁定digest；真实支付／AI调用只使用已确认账户／预算，秘密不入Git或报告。

Review Focus

- RF1：环境变量看似存在但值为空／秘密引用越界，必须启动失败且无明文日志；任务1。

- RF2：配置刷新部分失败或新镜像schema不兼容，禁止混合版本接受新写；任务2。

- RF3：跨端切换、支付重复回调、供应商未知状态同时出现，作品／账本仍一致；任务3。

- RF4：恢复旧库使已注销照片复活或重复履约，必须重放删除标记并先对账；任务4。

- RF5：无实际流量／无真机／无真实渠道证明的发布不能被标成全通过，失败回滚保留新业务数据；任务5。

Task 1: 可重复构建与启动配置

Files: Create scripts/release/build-manifest.ts, scripts/release/check-startup.ts, deploy/compose.integration.yaml, .github/workflows/release.yml, tests/ops/startup.spec.ts；Modify root package.json。

Interfaces: Consumes环境schema、实际lockfile和C1 – C5产物。Produces parseStartup(env:Record<string,string|undefined>,role:ServiceRole):StartupConfig、buildManifest(input:BuildInputs):Promise<ReleaseManifest>。ServiceRole为api/export-worker/ai-worker/web/admin；BuildInputs含commit、imageDigests、schemaVersion、migrationVersion、rendererBuild、font/filter/template hashes；manifest不含秘密。

☐ Step 1: 写startup.spec.ts，空DATABASE_URL_FILE、路径越出/run/secrets、prod使用localFiles、未锁镜像digest均拒绝；expect(()=>parseStartup(invalid,'api')).toThrow('STARTUP_CONFIG_INVALID')。（RF1）

☐ Step 2: npm run test:ops -- tests/ops/startup.spec.ts，预期FAIL缺parseStartup实现，不能只把配置错误算有效失败。

☐ Step 3: 实现两个签名，数值环境字符串先转类型，只校验选定键；按角色读取必要_FILE，worker不索取会话签名秘密；镜像／SBOM／许可随产物归档，Compose从本包模板适配实际构建路径。

☐ Step 4: 同命令及npm run typecheck预期退出0；docker compose --env-file deploy/.env.integration -f deploy/compose.integration.yaml config --quiet预期0，检查输出不含秘密值，不执行生产连接。

☐ Step 5: 提交上述路径；git commit -m "build: verify role-based startup and release manifests"。

Task 2: 健康检查、配置陈旧和兼容迁移

Files: Create apps/api/src/health/readiness.ts, scripts/release/check-compatibility.ts, tests/ops/readiness.spec.ts。

Interfaces: ConsumesC3 DB端口、C4配置快照／任务claim端口。Produces readiness(deps:HealthDependencies,now:Date):Promise<ReadyStatus>、assertCompatible(current:ReleaseManifest,target:ReleaseManifest):void。ReadyStatus含ready、checks[{name,state,reason}]；仅脱敏类别。

☐ Step 1: 断言DB不可达readiness503，AI未配置只关闭AI能力；配置陈旧>300秒不接受新写／claim但可终结在途；expect(()=>assertCompatible(newSchema,oldReader)).toThrow('INCOMPATIBLE_READER')。（RF2）

☐ Step 2: npm run test:ops -- tests/ops/readiness.spec.ts预期FAIL缺入口。

☐ Step 3: 实现上述签名，完整config release原子刷新；live只检测进程，ready检测当前角色必要依赖；迁移采用expand→兼容→回填→contract，不自动执行破坏性down。

☐ Step 4: 同命令PASS；隔离环境执行健康探针／迁移兼容样本，保留HTTP状态和schema版本，不用单个进程live当全站就绪。

☐ Step 5: git add apps/api/src/health/readiness.ts scripts/release/check-compatibility.ts tests/ops/readiness.spec.ts；git commit -m "feat(ops): gate readiness and schema compatibility"。

Task 3: 全链路、跨端与竞态验收

Files: Create tests/e2e/album-commerce-ai.spec.ts, tests/e2e/mini-contract.spec.ts, scripts/release/collect-evidence.ts。

Interfaces: ConsumesOpenAPI、测试工作区、可重放adapter、真实浏览器和mini真机记录。Produces collectEvidence(run:AcceptanceRun):Promise<EvidenceIndex>；AcceptanceRun含commit、environment、testIds、results、attachments、actualServiceModes，明示mock／sandbox／live。

☐ Step 1: 同一run覆盖mini上传→规则成册→网页改中文→revision冻结导出；支付回调100次、10并发余额仅够1次、AI提交超时迟到、取消／成功竞态，expect(grantsForOrder).toHaveLength(1)、expect(acceptedGenerationCount).toBe(1)、expect(exportRevision).toBe(12)。（RF3）

☐ Step 2: npm run test:e2e -- tests/e2e/album-commerce-ai.spec.ts tests/e2e/mini-contract.spec.ts预期FAIL缺串联／证据收集，不以缺凭据错误冒充功能失败。

☐ Step 3: 完成现有模块串联与证据收集，不为通过测试创建第二套账本；mock服务有固定故障脚本。真实微信／支付／AI测试在已具备准入和预算后执行，其失败不能用mock结果覆盖。

☐ Step 4: 同命令PASS，实际程序样片验证像素／DPI／中文／页序；小程序真机矩阵逐项记录。无真机／服务资质时对应case记blocked_external，不能写通过。

☐ Step 5: 提交本项路径；git commit -m "test: cover cross-platform album commercial and AI flows"。

Task 4: 备份恢复和队列重建

Files: Create scripts/ops/restore-drill.ts, scripts/ops/rebuild-jobs.ts, tests/ops/restore.spec.ts。

Interfaces: Consumes数据库／对象备份、tombstone、jobs/outbox和渠道只读查询。Produces restoreDrill(input:RestoreInput):Promise<RestoreReport>、rebuildJobs(scope:RecoveryScope):Promise<RecoveryReport>；input必须isolatedTarget=true、backupPoint、objectManifest、reconcileMode='queryOnly'，禁止默认生产目标。

☐ Step 1: 恢复点后删除的素材仍不可见，已付未履约只按同operation处理一次，旧worker fence不能提交；expect(restoredAsset.visible).toBe(false)、expect(repeatedGrantCount).toBe(0)。非隔离目标拒绝。（RF4）

☐ Step 2: npm run test:ops -- tests/ops/restore.spec.ts预期FAIL缺恢复编排。

☐ Step 3: 实现签名，先冻结新金融／生成写、恢复隔离目标、hash抽查、重放删除、只读对账、重建队列使旧租约失效；没有对账证据不恢复写入口。

☐ Step 4: 同命令PASS；实际隔离恢复记录RPO／RTO、账本汇总／资源引用／未完任务；建议RPO15分钟／RTO4小时只是目标，不自动出具已达标结论。

☐ Step 5: 提交本项路径；git commit -m "feat(ops): add isolated restore and queue recovery drills"。

Task 5: 容量、发布准出与回滚验证

Files: Create scripts/release/evaluate-gates.ts, scripts/release/rollback-dryrun.ts, tests/ops/release-gates.spec.ts, docs/implementation/release/README.md。

Interfaces: Consumes所有需求追踪和真实证据。Produces evaluateGates(evidence:EvidenceIndex,policy:ReleasePolicy):GateReport、rollbackDryRun(current:ReleaseManifest,target:ReleaseManifest):Promise<RollbackReport>；GateReport含passed、blocked、waived及明确原因，不能把skipped映射passed。

☐ Step 1: mock支付、空流量、未测真机、缺恢复报告均不能满足各自真实准出；expect(evaluateGates(incomplete,production).passed).toBe(false)；回滚不能执行数据删表或重复账本写。（RF5）

☐ Step 2: npm run test:ops -- tests/ops/release-gates.spec.ts预期FAIL缺gate计算。

☐ Step 3: 实现签名，按测试策略跑1→10→50→100会话阶梯及耐久，渲染逐面／预算限制；把实际CPU／内存／耗时／费用与目标对比。发布灰度按真实观测推进，生成可审查发布说明和回滚方案后再处理生产发布授权。

☐ Step 4: 同命令PASS；运行npm run release:check -- --evidence docs/implementation/release/evidence.json，仅全部规定门槛有真实证据才退出0，否则非0并列明阻断。验收后按执行方法安排独立复核。

☐ Step 5: 提交本项路径；git commit -m "feat(ops): enforce evidence-based release and rollback gates"。

自审

5项任务覆盖构建／配置、健康迁移、全链路、恢复、容量发布；5个Review Focus都有失败断言和命令。命令随实施任务写入根package scripts，当前文档包不创建空脚本冒充运行。所有真实凭据与生产发布点有具体输入和证据要求，不阻断可离线研发。